

Unmodified Half-Gates is Adaptively Secure So is Unmodified Three-Halves

Xiaojie Guo^{*,†,‡} Kang Yang^{*} Xiao Wang[§] Yu Yu^{¶,‡} Zheli Liu[†]

July 29, 2025

Abstract

Circuit garbling is a crucial cryptographic tool in many practical privacy-preserving applications due to two features: efficiency — it can be constructed in the random permutation model, allowing hardware acceleration; adaptivity — the majority of the communication can be transmitted offline before inputs are known. However, existing adaptive garbling schemes can only be proven in the random oracle model at best, leading to $20\times$ slowdown.

In this work, we apply analysis often used for symmetric-key primitives to adaptive garbling and show that two practically deployed selective-secure schemes, half-gates and three-halves, already satisfy adaptive security *without* any modification to their implementations or security assumptions. We show how to bound an adaptive advantage via an *adversary-dependent* statistical distance and analyze this distance by adapting the H-coefficient technique to remove this adversary dependence. For real-life systems, our result solves the security concern about the heuristic of using the two schemes with offline communication. As a byproduct, we discuss when we can further offload the decoding information of garbled outputs to the offline phase, leading to a separation result.

1 Introduction

Garbled circuits (GCs) [Yao86, BHR12b] provide practical solutions to a great number of privacy-preserving systems, e.g., COTI [COT24], TLSNotary [TLS], neural network inference [JVC18, RSC⁺19], private contact discovery [KRS⁺19]. They use GCs to implement secure two-party computation (2PC) to process their sensitive data. In a garbling scheme that creates GCs, a garbler can compute a garbled circuit $\hat{f}$ and a garbled input $\hat{x}$ for circuit f and input x , respectively. An evaluator, given $\hat{f}$ and $\hat{x}$, can compute a garbled output, which can be decoded into $f(x)$.

Practical garbled circuits are often proven under *selective security*. In this setting, the adversary chooses (f, x) *simultaneously* and distinguishes between a real-world execution consisting of honestly created $(f, \hat{x})$ and an ideal-world execution where these values are simulated using only $f(x)$. The concrete efficiency of garbling schemes has been greatly improved over the past decades [Yao86, BMR90, NPS99, KS08, PSSW09, KMR14, GLNP15, ZRE15, RR21], among which the state-of-the-art selective garblings (i.e., half-gates [ZRE15] and three-halves [RR21]) has a communication complexity of 2λ or $1.5\lambda + o(1)$ bits per AND gate for a computational security parameter λ , while XOR gates are free using free-XOR [KS08] optimization. To reduce computation, almost all implementations assume the random permutation model (RPM), which is commonly instantiated by fixed-key AES and accelerated using AES-NI [BHKR13, GKWY20, GKW⁺20]. This leads to selective garbling schemes producing more than 20 million garbled AND gates per second [WMK16]. However, selective-secure garbling schemes have a major drawback: the evaluator must receive the garbled circuit along with its garbled input;

^{*}State Key Laboratory of Cryptology. **Email:** yangk@sklc.org

[†]Nankai University. **Email:** liuzheli@nankai.edu.cn

[‡]Shanghai Qi Zhi Institute. **Email:** guoxiaojie@sqz.ac.cn

[§]Northwestern University. **Email:** wangxiao@northwestern.edu

[¶]Shanghai Jiao Tong University. **Email:** yuyu@yuyu.hk

since the garbled circuit can be large, data transfer becomes a significant bottleneck during evaluation. Furthermore, selective schemes are widely mis-implemented based on the informal folklore description quoted by many practical applications: “after receiving a garbled circuit from the garbler, the evaluator can run an oblivious transfer protocol with the garbler to obtain its garbled input”. Given that the selective security only holds when the evaluator cannot base its input on the sent garbled circuit, such mis-implementations are not provably secure.

To reduce the evaluation latency of many systems and achieve provable security, one could consider the *adaptive security* of a garbling scheme, where the garbler can send the garbled circuit to the evaluator in an offline phase before the inputs are known. In adaptive security, the adversary is more powerful to adaptively choose input x according to the garbling f of f : it chooses f and receives $\hat{f}$ in the offline phase, and then chooses x and receives $\hat{x}$ in the online phase. Besides 2PC with low online communication [IK00, AIKW13, LR14, LR15, RR16, MR17], adaptive garbling can also reduce online communication of many other cryptographic constructions, e.g., one-time programs [GKR08], functional encryption [SS10, GVW12], and verifiable computation [GGP10, AIK10]. Unfortunately, most works [BHR12a, HJO⁺16, JW16, JKK⁺17, GS18, JO20] on adaptive garbling take a more theoretical aspect. They are mostly in the standard model but all require some compromise in either exponential security loss or undesirable online communication, unless Cryptomania assumptions are used. When real-world systems require a practical adaptive garbling scheme, they [RR16] either use a random oracle to garble a selective scheme, which has been proven to be adaptive [BHR12a, LR15], or they just use a selective scheme without modification and hope for no adaptive attack. Since SHA256 (with SHA-NI) is $60\times$ slower than AES-NI, such adaptive GC implementations in existing systems remain either inefficient or not provably secure.

1.1 Our Contribution

In this work, we fill the gap by proving that the two state-of-the-art selective schemes, half-gates [ZRE15] and three-halves [RR21], are adaptively secure *for free* in the non-programmable RPM (npRPM) [RS08, BHKR13] where their selective security is proved (with npRPM-based circular correlation robust (CCR) hash functions [ZRE15, GKWY20, RR21]). We also demonstrate that we can further send the decoding information of garbled outputs in the offline phase if the RPM is programmable and that the feasibility of offline decoding information implies a separation result. We have the following results in both practice and theory.

Practical side. An immediate implication of our results is that existing systems do not need any new implementation or new assumption to obtain adaptive security. This is the best that real systems could hope for: no extra computation/communication cost, no change of implementation, and the state-of-the-art efficiency of the selective-secure schemes. For each AND gate, our result leads to at least $20\times$ (resp., $10\times$) improvement in online evaluation (resp., offline garbling) since we only have the cost of the two selective schemes. Furthermore, since many practical implementations (e.g., [KLS⁺17, JVC18, KRS⁺19, PSSY21]) already shift the GC communication of the selective schemes to the offline phase *without an adaptive proof*, our results can provide a security patch to these implementations, without harming concrete efficiency. We detail our practical impacts in Section 2.

Theoretical side. We present a general proof blueprint for adaptive security. It bounds the advantage of an adaptive adversary via the statistical distance of its resulting transcript distribution. However, this statistical distance cannot be computed directly since we do not know the distribution of the messages from the adversary. We demonstrate how to bypass this problem by borrowing an observation behind the H-coefficient technique [Pat09, CS14, DLMS14] to rethink the statistical distance equivalently and more simply. This blueprint can be of independent interest, not only for adaptive garbling.

As our main result, we use the blueprint to prove the adaptive security of half-gates and three-halves in the npRPM. Since the two garbling schemes are fairly complex, analyzing the statistical distance requires non-trivial efforts. To simplify this analysis, we propose a technique called *transcript padding*, which explicitly fixes some randomness given to the adversary to save the quantification over its possible values. Moreover, this technique is important to well define the transcripts needed for

	Offline Cost	Online Cost	Security Loss	Assumption
[HJO ⁺ 16]	$C \cdot 4\lambda$	$m + n\lambda + O(d\lambda^2 \log C)$	$2^{O(d)}$	OWF
	$C \cdot 4\lambda$	$m + n\lambda + O(w\lambda^2 \log C)$	$\text{poly}(C, \lambda)$	OWF
[JW16, JKK ⁺ 17]	$C \cdot 4\lambda$	$m + n\lambda$	$2^{O(d)}$	OWF
[JO20]	$C \cdot 2\lambda + A \cdot \lambda$	$m + n\lambda$	$2^{O(d)}$	OWF
[GS18]	$C \cdot \text{poly}(\lambda)$	$m + n + \text{poly}(\lambda, \log C)$	$\text{poly}(C, \lambda)$	CDH/LWE
[LR15]	$C \cdot 4\lambda + m$	$n\lambda$	$\text{poly}(C, \lambda)$	pROM
[BHR12a]	$A \cdot \{1.5\lambda, 2\lambda\} + m$	$n\lambda$	$\text{poly}(C, \lambda)$	pROM
This work	$A \cdot \{1.5\lambda, 2\lambda\}$	$m + n\lambda$	$\text{poly}(C, \lambda)$	npRPM
	$A \cdot \{1.5\lambda, 2\lambda\} + m$	$n\lambda$	$\text{poly}(C, \lambda)$	pRPM

Table 1: **Comparison of adaptively secure garbling schemes.** The offline (resp., online) cost denotes the communication of a garbled circuit (resp., garbled input), where C (resp., A) is the total number of all gates (resp., only AND gates). Let λ, n, m, w, d be the security parameter, input size, output size, circuit width, and circuit depth, respectively. OWF denotes the one-way function.

the H-coefficient bound of statistical distance. Although the resulting construction still needs an ideal model, it is the first time that the adaptive security of concretely efficient garbling schemes is proven in a non-programmable model. Besides, a λ -bit random permutation is known more conservative than a λ -bit length-preserving random oracle in [BHR12a, LR15, BHKO23].

Since some implementations (e.g., ABY [DSZ15] and MP-SPDZ [Kel20]) of the two schemes send the decoding information in the offline phase to further reduce the online communication, we extend our proofs to cover their adaptive security. We show that these implementations can only be adaptively secure in the programmable RPM (pRPM), where the programmability is to ensure the decoding correctness of garbled outputs. We further prove that there is a separation between npRPM and pRPM, reminiscent of that of random oracle. See Table 1 for the theoretical comparison and Section 3.2 and Section 3.3 for a high-level discussion of the intuition of our proof.

Concurrent work. Recently, Barnum et al. [BHKO23] proved that half-gates [ZRE15] and three-halves [RR21] are adaptively secure in the non-programmable random oracle model (npROM). Their proofs require some hybrids manipulating the entries of the random oracle but arrive at a simulator without such a manipulation. Their methods still require modifying the protocol and the need of a non-programmable random oracle, which in practice would be instantiated using e.g., SHA256. In this sense, their work focuses on the theoretical side of weakening the assumption only, and their adaptive garbling scheme is as inefficient as the adaptive schemes based on the programmable random oracle (pROM).

1.2 Organization

Since our paper does not propose a new construction but rather shows that popularly implemented schemes have extra highly desirable properties, we organize the rest of this paper as follows. In Section 2, we provide a detailed discussion of the practical impacts of this paper, including the broad range of applications that can benefit from our results and the amount of computational efficiency they can get. In Section 3, we revisit the limitations of previous proofs and provide a high-level overview of the methodology of our proofs, followed by Section 4 for preliminaries. For the detailed proofs of adaptive security, we refer readers to Section 5 for the half-gates in the npRPM, Appendix C for the half-gates in the pRPM, and Appendix D for the three-halves in both RPMs. We prove the separation of two RPMs in Appendix E.

2 Practical Impacts

Half-gates and three-halves have a huge number of applications, many of them working in the popular offline/online setting where data-independent heavy computation is done in the offline phase and the online phase is cheap. In Table 2, we provide a comprehensive survey on them and show whether their implementations are provably secure or not. Some of them deliberately move the computation and transfer of GCs to the offline phase to minimize online cost, or unconsciously do this since they are implemented on 2PC frameworks with offline GC transfer (e.g., ABY [DSZ15]). Before our result, almost all applications take the adaptive security of the two schemes for granted, without a rigorous proof. The only provable approaches before our work would incur at least an order of magnitude slowdown and thus these applications would either incur such overhead or using an unprovable construction.

For example, consider the private set intersection protocol of [KRS⁺19], which already uses half-gates in the offline/online setting to garble AES circuits (but without an adaptive proof). In its experiment, there are 2^{10} circuits, each of which has 5,120 AND gates. With our result, the computation overhead can be about 0.1 second for garbling and 0.05 second for evaluation on a single 3 GHz core. In contrast, the previous best adaptive garbling takes 1.14 second and 1.09 second, respectively. If the GC of these gates are moved to the online phase, there will be 160 MB communication, significantly increasing the latency on its 230 Mbit/s bandwidth.

2.1 Analysis of Efficiency Improvements

Here, we provide a detailed analysis on how our results yield huge practical efficiency gains for adaptive garbling. To provide implementation independent performance, we show the number of CPU cycle needed for different constructions. In particular, we assume a CPU of type Intel(R) Xeon(R) Platinum 8124M. In this setup, a fixed-key AES-NI based CCR hash requires 14 CPU cycles while a SHA-NI based SHA256 evaluation takes around 594 CPU cycles.

- [LR15] uses a programmable random oracle (RO) to replace the hash calls in a garbling scheme. Since the garbling requires 4 calls to the hash functions and the evaluation requires 2 calls to the hash function, the cost is entirely dominated by the RO calls.
- [BHR12a] proposes a generic framework to use an RO to mask the garbled circuits. Thus, they require 4 calls to RPM to compute the selective garbling and one calls to RO to obtain the mask. Since SHA256 produces 256 bits of output in a call, one call to SHA256 can produce sufficient mask bits for half gates.
- [BHKO23] replaces the AES-based CCR hash functions in the selective scheme by a non-programmable random oracle, which has the same SHA256-based instantiation as in [LR15]. Thus, its performance is dominated by the RO calls.
- In our protocol, the computation and the assumption is exactly the same as prior work with 4 random permutation (RP) calls for garbling and 2 RP calls for evaluation.

In summary, we conclude that the only way to preserve the efficiency of modern garbling schemes is to avoid using ROs, which is exactly what we are able to achieve. Table 3 presents our results.

3 Technical Overview

3.1 Limitations of Previous Proof Techniques

The existing techniques cannot prove the adaptive security of the *unmodified* half-gates [ZRE15] and three-halves [RR21], which are the two state-of-the-art practical garbling schemes *only with selective security*. They use free-XOR optimization [KS08], where the XOR of two labels per wire equals the same global offset Δ . In these schemes, a garbled gate in the whole garbled circuit is an one-time pad (OTP) encryption with a key $H(X \oplus \Delta, k) \oplus b\Delta$ for some active label X , distinct tweak k , and truth bit b dependent on online input x . We summarize the limitations of the existing techniques as follows.

Application	Work	Security
Private set intersection	[PSSZ15]	Not provable
	[KLS ⁺ 17]	Not provable
	[KRS ⁺ 19]	Not provable
Neural network inference	[JVC18]	Not provable
	[MLS ⁺ 20]	Not provable
	[CGOS22]	Not provable
	[GGJ ⁺ 23]	Not provable
	[CJKH24]	Not provable
Transport Layer Security (TLS)	[XYWY24]	Not provable
	[LEFS25]	Not provable
Mixed-mode 2PC	[DSZ15]	Not provable
	[PSSY21]	Not provable
Generic 2PC	[RR16]	pROM
Crowd sensing	[TWYW18]	Not provable
Genome analysis	[CHW18]	Not provable
	[ST19]	Not provable
Localization	[JKS ⁺ 18]	Not provable
	[JLL ⁺ 19]	Not provable
	[vdBNS22]	Not provable
Clustering	[BCE ⁺ 21]	Not provable
Ridge regression	[NWI ⁺ 13]	Not provable
Matrix factorization	[NIW ⁺ 13]	Not provable
Remote diagnostics	[BPSW07]	Not provable
Face identification	[HW23]	Not provable
Federated learning	[XLZ ⁺ 22]	Not provable

Table 2: **Practical use of adaptive garbling.**

Scheme	Size (Bytes)	Garbling			Evaluation		
		# RP calls	# RO calls	Total # CPU Cycles	# RP calls	# RO calls	Total # CPU Cycles
[LR15]	64	0	4	2376	0	2	1188
[BHR12a]	32	4	1	650	2	1	622
[BHKO23]	32	0	4	2376	0	2	1188
This work	32	4	0	56	2	0	28

Table 3: **Comparison of the practical adaptively secure garbling scheme.** We measure the communication (size) and computation for each AND gate, where XOR gates are free except for [LR15]. We use SHA256 (594 CPU cycles) for the length-preserving ROM and fixed-key AES (14 CPU cycles) for the RPM. The numbers of CPU cycles are estimated from [GKWW20].

Proofs for selective security. The proofs use the CCR assumption to argue the pseudorandomness of OTP keys and a garbled circuit, but do not extend to the adaptive setting. In these proofs, the reduction algorithm is given the circuit f and the input x in one shot. Thus, it can compute all truth bits on wires during the evaluation of $f(x)$ and calls the CCR oracle to obtain OTP keys. If it is given a real CCR

oracle, these OTP keys follow the above form (by properly dealing with tweaks, see [GKWY20]) so that the reduction algorithm can construct a real garbled circuit. If it is given an ideal CCR oracle (i.e., random function), these OTP keys are independently random so that the constructed garbled circuit is uniformly random as in the simulation. This reduction to the CCR assumption applies to the selective security of the two schemes.

However, in adaptive security, the same reduction fails since it is no longer given f and x simultaneously. The technical challenge is that, by the time the reduction algorithm needs to call the CCR oracle to obtain OTP keys and the garbled circuit to be sent in the offline phase, it has not received x to compute all truth bits yet. In other words, the reduction algorithm cannot meaningfully bridge the two worlds using the given oracle.

Pebbling-based proofs in the standard model. For adaptive security, the known works in the standard model [HJO⁺16, JW16, JKK⁺17, GS18, JO20] use the proof idea [LP09] of Yao’s garbling scheme in the selective setting, which is modeled as a pebbling game. Such a pebbling game changes all real garbled gates (i.e., white pebbles) to simulated ones (i.e., black pebbles) using a careful hybrid argument, where each hybrid bridges an *input-consistent* garbled gate (i.e., a gray pebble) and a real or simulated one. The input-consistency here is similar to that existing in between the OTP masks (or rather, truth bits therein) and the input x . Thus, to adopt a pebbling game to prove the adaptive security of their schemes (*which are totally different from half-gates and three-halves*), these works also need to guarantee that gray pebbles are consistent with the input undefined until the online phase, i.e., facing essentially the same technical challenge in the selective security. They address this challenge by using somewhere equivocal encryption or piecewise guessing in the construction of gray pebbles, yet resulting in impractical performance or non-negligible security loss.

Another fatal technical challenge is that pebbling-based proofs are not compatible with the free-XOR optimization. To see this result, we recall two important facts in such proofs: (i) a pebbling hybrid can only change one pebble, and (ii) the indistinguishability between a gray pebble and a white/black one relies on a *black-box* reduction to the security of some low-level cryptographic primitive. As a corollary, two pebbling hybrids should respectively reduce their indistinguishability to two *independent* instances of the primitive in a black-box way. One can regard the low-level primitive as an encryption scheme for truth bits so that the corollary implies that all encryptions should be independent. However, with the free-XOR optimization, these encryptions are correlated under the global offset Δ . This correlation contradicts the independence and concludes that a pebble-by-pebble hybrid argument cannot prove the adaptive security of the garbling schemes using free-XOR, including half-gates and three-halves.

Proofs in the programmable ROM. Some proofs [BHR12a, LR14] resorts to the programmability of the random oracle model to bypass the input-consistency challenge and be compatible with free-XOR. As noted in [BHR12a], we have the following generic selective-to-adaptive transformation for any selectively secure garbling scheme: one additionally uses some RO output as an OTP key to encrypt the garbled circuit sent in the offline phase and decrypts it in the online phase. In the simulation for the adaptive security, the reduction algorithm simply sends a random message in the offline phase. Finally, in the online phase, it programs the random oracle to equivocate this message to be decrypted to a selectively simulated garbled circuit consistent with an online input x known at this time. Although this transformation applies to half-gates and three-halves, it leads to additional overhead due to the one length-expanding RO call. This overhead is significant since the SHA-based instantiation of random oracle is less efficient than the block-cipher-based instantiation of random permutation. Moreover, from a theoretical perspective, programmability is recognized as a strong property and less desirable. Meanwhile, [LR14] uses the programmability gate-wise but focuses on a variant of Yao’s garbling scheme, which is much slower than half-gates and three-halves. Removing programmability and extending its proof to the two schemes are also non-trivial.

3.2 Our Proof Blueprint for Adaptive Security

Unlike all recapped techniques, we have a new perspective of adaptive security and a general proof blueprint for it. *This blueprint can be of independent interest, not only for adaptive garbling.*

We point out that, in an adaptive experiment, the “adaptivity” means that a message sent by the adversary can depend on its received messages so far. This is the root cause of the challenge in proving the adaptive security of primitives (e.g., adaptive garbling). From an induction, the argument for this point is as follows. Consider the first message received by the adversary. On the one hand, if this message can be simulated *without* any subsequent message(s) sent by the adversary, its simulation is equivalent in both adaptive and selective security. This case is trivial and less interesting. On the other hand, if the simulation of the first messages depends on the subsequent message(s) sent by the adversary (e.g., the previous simulated garbled circuit depends on the online input x), it incurs a “chicken-or-egg” causality dilemma. This induction is applied to every message received by the adversary, until the dilemma happens or we finally arrive at that the adaptive security degenerates to the selective security trivially. To overcome this dilemma, the simulator seemingly requires the ability to equivocate its simulated messages to be consistent with the subsequent messages of the adaptive adversary. This requirement finally causes expensive primitives or unsatisfying techniques (e.g., guessing subsequent messages or programmability).

We observe that the above thinking is somewhat a stereotype from the view of complexity-theoretic reduction. Instead, we can avoid the seemingly inevitable equivocation if we rethink the problem from a statistical view. Without loss of generality, consider a computationally unbounded non-uniform *deterministic*¹ adaptive adversary $\mathcal{A}$ that interacts with either a real-world oracle $\mathcal{O}_0$ or an ideal-world oracle $\mathcal{O}_1$ and outputs its decision bit after the interaction. Both oracles have the same query interfaces for $\mathcal{A}$. The interaction between $\mathcal{A}$ and $\mathcal{O}_b$ defines a random variable Z_b of transcripts, which include ordered query-response pairs in the interaction. For any adaptive adversary $\mathcal{A}$ and its auxiliary input, the advantage of $\mathcal{A}$ is at most the statistical distance

$$\begin{aligned} \text{SD}(Z_0, Z_1) &= \frac{1}{2} \cdot \sum_{\tau} |\Pr[Z_0 = \tau] - \Pr[Z_1 = \tau]| \\ &= \sum_{\tau: \Pr[Z_1 = \tau] \geq \Pr[Z_0 = \tau]} \left(\Pr[Z_1 = \tau] - \Pr[Z_0 = \tau] \right). \end{aligned}$$

However, relying solely on this statistical view is not enough because it is unknown how to compute the statistical distance. More specifically, since the random variable Z_b is induced by the adaptive messages (i.e., queries) of $\mathcal{A}$ and the next-message function of $\mathcal{A}$ can be arbitrary, it is infeasible to compute the probability $\Pr[Z_b = \tau]$ directly, let alone the statistical distance. Therefore, we need insights to deal with the adaptivity in the definition of Z_b to adopt this statistical view to bound the advantage of the adaptive adversary.

The insights to “peel” adaptivity. Given any fixed adaptive adversary $\mathcal{A}$, there is a set $\mathcal{T}_{\mathcal{A}}$ of *attainable transcripts* such that for each transcript $\tau \in \mathcal{T}_{\mathcal{A}}$, it holds that $\Pr[Z_1 = \tau] > 0$ ². It is obvious that

$$\text{SD}(Z_0, Z_1) = \sum_{\tau \in \mathcal{T}_{\mathcal{A}}: \Pr[Z_1 = \tau] \geq \Pr[Z_0 = \tau]} \left(\Pr[Z_1 = \tau] - \Pr[Z_0 = \tau] \right).$$

In particular, the transcripts in $\mathcal{T}_{\mathcal{A}}$ must respect the syntax of the intersection, i.e., having the allowed number of query-response pairs with prescribed forms. Otherwise, the probability $\Pr[Z_1 = \tau] = 0$ trivially in the ideal world, contradicting the definition of $\mathcal{T}_{\mathcal{A}}$. In proofs, $\mathcal{T}_{\mathcal{A}}$ is conceptually defined in the sense that one only knows the transcript syntax therein but cannot identify all transcripts as the

¹Note that a non-uniform deterministic adversary is at least as powerful as a probabilistic adversary. See, e.g., [Can00, Footnote 3, p. 14].

²In fact, we have an equivalent definition for the definition of attainable transcripts, see Section 4.4 for the definition and Appendix B for the equivalence.

next-message function of $\mathcal{A}$ is unknown. However, we will show that this conceptual $\mathcal{T}_{\mathcal{A}}$ is enough to compute the probabilities in $\text{SD}(Z_0, Z_1)$.

Based on the definition of $\mathcal{T}_{\mathcal{A}}$, we can compute the probability $\Pr[Z_b = \tau]$ from a pure non-adaptive probability analysis, without considering the arbitrary next-message function of $\mathcal{A}$. Fix some $b \in \{0, 1\}$ and transcript $\tau \in \mathcal{T}_{\mathcal{A}}$ of ordered query-response pairs $((q_1, r_1), \dots, (q_n, r_n))$. *We stress that fixing such a τ merely requires the transcript syntax, using no literal value dependent on the next-message function of $\mathcal{A}$.* Furthermore, in the world b , we define the probability $\Pr_{O_b}[O_b \in \text{comp}_b(\tau)]$ that a randomly picked oracle O_b is *compatible with* $\tau = ((q_1, r_1), \dots, (q_n, r_n))$, i.e., that the oracle O_b can produce the responses $r_1, \dots, r_n$ in order when it is given the queries $q_1, \dots, q_n$ in order.

A key insight is that for a fixed $\tau \in \mathcal{T}_{\mathcal{A}}$, $\Pr[Z_b = \tau] = \Pr_{O_b}[O_b \in \text{comp}_b(\tau)]$. The reason is as follows. The definition of $\mathcal{T}_{\mathcal{A}}$ ensures $\Pr[Z_1 = \tau] > 0$. This probability implies that, in the ideal world, the deterministic $\mathcal{A}$ must produce the queries $q_1, \dots, q_n$ (where the first query is determined by its given auxiliary input) in order if the responses $r_1, \dots, r_n$ arrive in order, as $\mathcal{A}$ is deterministic so that this is a yes-or-no event. When it comes to the real world, the same deterministic $\mathcal{A}$ with the same auxiliary input will produce the same queries $q_1, \dots, q_n$ in order as in the ideal world when given the same ordered responses $r_1, \dots, r_n$. Therefore, in either world $b \in \{0, 1\}$, the event $Z_b = \tau$ happens if and only if the event that a randomly picked oracle O_b produces the responses $r_1, \dots, r_n$ in order when it is given the queries $q_1, \dots, q_n$ in order, where the randomness underlying the two events is due to the sampling of O_b since $\mathcal{A}$ is deterministic. The above argument concludes the key insight. *Here, computing the probability $\Pr_{O_b}[O_b \in \text{comp}_b(\tau)]$ is independent of $\mathcal{A}$ since the probability is taken over the randomness of O_b .* This insight is implicitly used in the analysis (e.g., [Pat09, CS14, DLMS14, HT16]) of symmetric-key primitives and, as far as we know, has not been explicitly connected to the adaptive security of cryptographic protocols.

Instead of computing the exact statistical distance $\text{SD}(Z_0, Z_1)$, we consider its upper bound for simplicity. A bound is given by the H-coefficient technique [Pat09, CS14, DLMS14]. It partitions $\mathcal{T}_{\mathcal{A}}$ into a set $\mathcal{T}_{\text{bad}}$ of *bad transcripts* and a set $\mathcal{T}_{\text{good}}$ of *good transcripts*. It says that $\text{SD}(Z_0, Z_1) \leq \varepsilon_1 + \varepsilon_2$ if $\sum_{\tau \in \mathcal{T}_{\text{bad}}} \Pr[Z_1 = \tau] \leq \varepsilon_1$ and, for each $\tau \in \mathcal{T}_{\text{good}}$, $1 - \Pr[Z_0 = \tau]/\Pr[Z_1 = \tau] \leq \varepsilon_2$. In proofs, ε_1 can be computed from the key insight and a union bound and ε_2 is computed from the key insight and the properties of good transcripts. *In the computation of ε_1 and ε_2 , τ is fixed when computing $\Pr[Z_b = \tau]$ and, as noted before, requires only the transcript syntax. Moreover, a fixed $\tau \in \mathcal{T}_{\mathcal{A}}$ can be conceptually categorized into $\mathcal{T}_{\text{bad}}$ or $\mathcal{T}_{\text{good}}$ as per some predicates (which only require the transcript syntax), and the union bound over $\mathcal{T}_{\text{bad}}$ can be relaxed to the union bound over $\mathcal{T}_{\mathcal{A}}$, whose size is further bounded by the number of the transcripts with the given syntax.*

In the given upper bound, the separation between bad and good transcripts can be arbitrary as long as it is a partition. Nevertheless, it is often more understandable to make the partition specific to the scheme to be proved. At a high level, bad transcripts record the interaction where the adaptive adversary successfully breaks the scheme while the complement good transcripts are with desired properties to ease the computation of ε_2 . In other words, we bound the advantage of breaking the scheme by a negligible ε_1 . Meanwhile, a negligible ε_2 implies that the distribution of good transcripts (i.e., corresponding to the interaction without valid attacks) is statistically indistinguishable. In practice, capturing all potential attacks in bad transcripts can be non-trivial and should be instantiated as per the proved scheme, making our blueprint non-black-box.

Putting all things together, we have the proof blueprint in Figure 1.

Why we can avoid programmability or the modifications to schemes. The philosophy behind our blueprint is that it implicitly quantifies over all possible interaction transcripts in both two worlds and then replaces an adaptive adversary by all sequences of ordered queries in these transcripts. Using this quantification, we no longer run an adaptive adversary to extract its adaptive queries and use the oracle of some low-level adaptively secure primitive to answer each query as in a complexity-theoretic reduction. As said, the programmability or the modifications are to implement such an adaptively secure primitive but not required anymore if we jump out of the complexity-theoretic thinking.

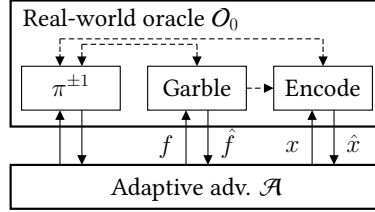
By considering all possible outcomes of adaptive queries and bounding their effect on the advan-

For any adaptive adversary $\mathcal{A}$, define a set of predicates to get two disjoint subsets $\mathcal{T}_{\text{bad}}$ and $\mathcal{T}_{\text{good}}$ partitioning $\mathcal{T}_{\mathcal{A}} = \{\tau : \Pr[Z_1 = \tau] > 0\}$, where the transcript syntax is specific to the scheme but not dependent of the next-message function of $\mathcal{A}$. If there exists $\varepsilon_1, \varepsilon_2 \geq 0$ such that

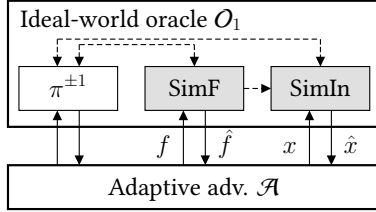
- $\sum_{\tau \in \mathcal{T}_{\text{bad}}} \Pr_{O_1} [O_1 \in \text{comp}_1(\tau)] \leq \varepsilon_1$, and
- $\forall \tau \in \mathcal{T}_{\text{good}} : 1 - \Pr_{O_0} [O_0 \in \text{comp}_0(\tau)] / \Pr_{O_1} [O_1 \in \text{comp}_1(\tau)] \leq \varepsilon_2$,

then the advantage of $\mathcal{A}$ is at most $\varepsilon_1 + \varepsilon_2$. The probabilities are only taken over the randomness of oracles and independent of $\mathcal{A}$.

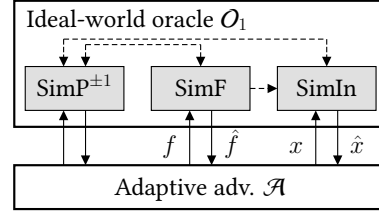
Figure 1: Our proof blueprint of adaptive security.



(a) Real world in the npRPM/pRPM.



(b) Ideal world in the npRPM.



(c) Ideal world in the pRPM.

Figure 2: Experiments of adaptive garbling in the npRPM and pRPM. (a) versus (b): The experiment in the npRPM, with simulator (SimF, SimIn). (a) versus (c): The experiment in the pRPM, with simulator (SimF, SimIn, SimP $^{\pm 1}$). In the two experiments, $\mathcal{A}$ can make permutation queries at any time.

tage of an adaptive adversary, our blueprint will not face the main challenge in the reduction-based proofs of adaptive security (e.g., in adaptive garbling): given the real execution where a response will be consistent with future adaptive queries as per the real oracle of some low-level primitive, and the ideal execution where the response is sampled at random as in the ideal oracle, how to bridge the two executions using a reduction to the primitive? However, we need to pay some price in terms of the modularity of proofs.

Remark 1 (On the proof modularity: computationally secure primitives are useless in the blueprint). *Our blueprint needs to compute two negligible bounds ε_1 and ε_2 . Nevertheless, we cannot obtain these statistical bounds from the adversarial advantage of some computationally secure primitives, thus sacrificing the modularity ensured by the primitive-by-primitive reduction. Instead, we should use explicit randomness (e.g., ideal models and uniform coins) to compute the probabilities for the two bounds directly, making a resulting proof monolithic.*

On the one hand, this blueprint can be of interest if a proved construction has a simple structure atop ideal models and random coins (with an easy probability analysis). On the other hand, if the construction is complex (e.g., garbling schemes), the probability analysis in the blueprint will be non-trivial.

3.3 Adaptive Garbling from Our Blueprint

We prove the adaptive security of half-gates and three-halves by instantiating the blueprint. In our proof, real-world oracle O_0 runs the garbling scheme while ideal-world oracle O_1 is defined from the

simulation. Since the two schemes are in RPM, both oracles provide adaptive adversary $\mathcal{A}$ with not only an offline interface for garbled circuit and an online interface for garbled input, but also two interfaces for the random permutation and its inverse. In Figure 2, we demonstrate the two experiments of adaptive garbling in this blueprint according to whether the RPM is programmable. In either RPM, $\mathcal{A}$ can make queries to the random permutation and its inverse at any time, while the online interface can only be queried by $\mathcal{A}$ after the offline one. In each world, the probability is taken over the random permutation and other random coins in garbling.

Recap of two garbling schemes: OTP-based security. In both half-gates and three-halves, a real garbled circuit contains OTP ciphertexts, each with a key of form $H(X \oplus \Delta, k) \oplus b\Delta$ for some active label X , distinct tweak k , truth bit b , and global key Δ , where only Δ is hidden from the adversary. In the ideal world, a garbled circuit is simulated uniformly. Recall that in selective security, this ideal garbled circuit is *computationally indistinguishable* from the real one by reducing the pseudorandomness of OTP keys to the CCR randomness Δ . However, in our blueprint of adaptive security, we cannot use the reduction to CCR but need the *statistical* analysis of two bounds $\varepsilon_1, \varepsilon_2$. Since we hope the garbled circuit has indistinguishable distribution in the two worlds (otherwise, breaking the adaptive security is easy), we can use ε_1 to bound the probability of bad transcripts which induce the collision attack between two OTP keys so that the indistinguishability cannot hold. Meanwhile, by using ε_2 for each good transcript where the induced OTP keys are pairwise distinct, we aim to show the transcript (including not only the garbled circuit but also other messages) has a *statistically indistinguishable* distribution in the two worlds.

To obtain these two bounds, we require a non-black-box investigation into the construction of hash function H in the RPM, which provides an important source of randomness. In the two schemes, the hash function is tweakable and can be implemented by $H(X, k) = \pi(X \oplus k) \oplus \sigma(X \oplus k)$, where π is a random permutation and σ is a linear function (see [GKWY20] or Section 4.1 for details). With this hash function, an OTP encryption is effectively masked by a permutation image of form $\pi(X \oplus \Delta \oplus k)$, where Δ is unknown to the adversary. Thus, we treat such permutation images as effective OTP keys in the analysis of $\varepsilon_1, \varepsilon_2$. To be effective OTP keys, these permutation images should never be trivially revealed in transcripts or have pairwise collision; otherwise, the transcripts from which the permutation images are induced constitute bad transcripts.

Challenge in bound analysis and transcript padding to rescue. A challenge is how to count the number of O_b 's compatible with some fixed τ to compute the probability $\Pr_{O_b} [O_b \in \text{comp}_b(\tau)]$. This challenge mainly comes from counting the number of $\pi^{\pm 1}$'s/SimP $^{\pm 1}$'s (each being a part of an O_b) that are consistent with the query-response pairs of the RPM and the permutation images in the OTP-based garbled circuit. Below, we focus on $\pi^{\pm 1}$ in the npRPM to explain the challenge; the same challenge will hold for the interface SimP $^{\pm 1}$ in the pRPM by replacing π with SimP. At a high level, each OTP encryption G in the garbled circuit can be abstractly written as:

$$G = \pi(X \oplus \Delta \oplus k) \oplus \pi(X \oplus k) \oplus \sigma(\Delta) \oplus L_{\alpha, \beta, \gamma}(X, X', \Delta), \quad (1)$$

where $L_{\alpha, \beta, \gamma}$ is a public linear function such that, for $\alpha, \beta, \gamma \in \{0, 1\}$,

$$L_{\alpha, \beta, \gamma}(X, X', \Delta) = \alpha X \oplus \beta X' \oplus \gamma \Delta,$$

X, X' are two active labels, k is a distinct tweak, and Δ is the global key for free-XOR. By collecting all equations of form (1), we can obtain a *mirror system* [CDN⁺23, DNS22] of the images of π including the equations of the following form:

$$\pi(X \oplus \Delta \oplus k) \oplus \pi(X \oplus k) = G \oplus \sigma(\Delta) \oplus L_{\alpha, \beta, \gamma}(X, X', \Delta). \quad (2)$$

We can apply a theorem from the mirror theory [CDN⁺23, DNS22] to count the number of $\pi^{\pm 1}$'s meeting both this mirror system (2) and the query-response pairs of the RPM. *However, even a fixed transcript τ may induce multiple mirror systems of the form (2) since the global key Δ and the internal*

active labels not in $\hat{x}$ are missing in τ . That is, counting the number of $\pi^{\pm 1}$ consistent with τ has to enumerate all possible missing values to enumerate all possible mirror systems to be solved, making the bound analysis quite complex.

We propose a technique called *transcript padding* to avoid this enumeration. The key idea is to have the oracle $\mathcal{O}_b$ additionally reveal the missing values to the adversary by padding them into transcripts at their specific points. More specifically, the oracle can send all truth bits and the missing active labels with $\hat{x}$, and send Δ after the adaptive adversary making all queries to the RPM (so that no query can depend on Δ). In this way, a fixed transcript τ can induce *only one* mirror system. Moreover, we can assume without loss of generality that the adversary always queries each $\pi(X \oplus k)$ after receiving the active labels (so that a fixed τ is additionally padded with these images). This helps us to fix the value of $\pi(X \oplus \Delta \oplus k)$ as per (2) without the mirror theory, finally fixing some entries of $\pi^{\pm 1}$ and making the counting easy. As the adversary can omit the padding values at will, a simple proof from the optimal-distinguisher-based definition of statistical distance can show that the statistical distance for this more informed adversary is at least the original one.

Another important benefit of transcript padding is that it enables a simple definition of bad transcripts, which capture the pairwise collision between the effective OTP keys of form $\pi(X \oplus \Delta \oplus k)$. As noted, their values are fixed by the transcript τ with transcript padding so that whether τ is bad or not can be identified by the values explicitly fixed by τ .

Bad and good transcripts in the two schemes. We provide more details about how to separate bad transcripts from good ones. In addition to the same capture of the pairwise collision of effective OTP keys of form $\pi(X \oplus \Delta \oplus k)$, the definition of bad transcripts has a slight difference according to whether the RPM is programmable or not. In our proofs, npRPM considers the existing implementations that send the decoding information in the online phase (e.g., TinyGarble [SHS⁺15], Obliv-C [ZE15], and OblivM [LWN⁺15]) while pRPM is for those with offline decoding information (e.g., ABY [DSZ15] and MP-SPDZ [Kel20]).

In the npRPM, a transcript is bad if and only if, computed from its induced mirror system (2), two permutation images of form $\pi(X \oplus \Delta \oplus k)$ have collision or either of them is trivially revealed in the transcript, making one permutation image not an effective OTP key. On the one hand, such a permutation image is trivially revealed if and only if there exists in the transcript a query-response pair of form $(X \oplus \Delta \oplus k, \pi(X \oplus \Delta \oplus k))$. On the other hand, two permutation images have collision if and only if at least one the following events occurs:

1. There is a collision between two permutation pre-images of form $X \oplus \Delta \oplus k$. Equivalently, there exists a collision between two permutation pre-images of form $X \oplus k$, or between two permutation images of form $\pi(X \oplus k)$. Such an equivalence comes from the bijective π . In the equivalent event, the values of pre-images and images are explicit in the transcript with transcript padding.
2. There exists a collision between the values of two permutation images, each of which is determined by subtracting other values from an OTP ciphertext in (1): $\pi(X \oplus \Delta \oplus k) = G \oplus \pi(X \oplus k) \oplus \sigma(\Delta) \oplus L_{\alpha, \beta, \gamma}(X, X', \Delta)$, where the right-hand values are also fixed via the transcript padding. In particular, if the value of $\pi(X \oplus k)$ is not fixed via the transcript padding, then the value of $\pi(X \oplus \Delta \oplus k)$ is not well-defined from the subtraction.

Following the interpretation of bad transcripts in our insights (see Section 3.2), the badness reflects the attacks against the effective OTP keys.

In the pRPM, bad transcripts reflect not only the attacks against OTP keys but also failed programming. Recall that this model allows for offline decoding information, which should be consistent with the LSBs of active output labels and the truth bits on output wires. Since these truth bits are not fixed until the online phase, we program the random permutation and its inverse to let them output the active labels consistent with the decoding information fixed in the offline phase. The programming manipulates all permutation entries of form $(X \oplus k, \pi(X \oplus k))$, which are queried to evaluate a garbled circuit. The values of these entries are already fixed in the transcript with transcript padding. A

programming is successful if and only if, in this transcript, these entries have not been occupied by the query-response pairs before the online phase. Finally, our definition of bad transcripts in the pRPM is that in the npRPM plus this predicate of failed programming.

Proof overview. At a high level, we explain our proof based on the abstract construction (1) of garbled circuit, which underlies half-gates and three-halves. Assume that the garbled circuit $\hat{f}$ contributes N OTP encryptions of form (1) and the adaptive adversary $\mathcal{A}$ makes q distinct queries to the random permutation and its inverse in addition to the N “bonus” queries of form $\pi(X \oplus k)$ in the transcript padding. Given any fixed adaptive adversary $\mathcal{A}$ with transcript padding, a transcript $\tau \in \mathcal{T}_{\mathcal{A}}$ consists of offline part $(f, \hat{f})$, online part $(x, \hat{x})$, query-response pairs of the random permutation and its inverse, and free-XOR offset Δ with $\text{lsb}(\Delta) = 1$. In particular, the query-response pairs occur before Δ at any time (so that they are independent of Δ) and correspond to the adaptive queries chosen by $\mathcal{A}$ and their responses. Moreover, $\hat{x}$ contains all active labels, not only those on the input wires. We prove the two negligible bounds $\varepsilon_1, \varepsilon_2$.

In the npRPM, the decoding information d is included in the online part $\hat{x}$ and trivially consistent with the active labels and the truth bits on the output wires since the online input x is known before sending d . As a result, as long as the oracle $\mathcal{O}_b$ is compatible with other parts, it will be further compatible with the decoding information d . The compatibility between the oracle and the garbled input (i.e., the active labels on the input wires) in $\hat{x}$ is also trivial as, in either world, this garbled input is essentially sampled at random due to the free-XOR encoding $X = W \oplus b\Delta$ for uniform zero label W and input bit b . The active labels on the internal wires are fixed in τ and consistent with the garbled input and “bonus” queries. Therefore, it remains to consider the compatibility between $\pi^{\pm 1}$ (a part of the oracle $\mathcal{O}_b$, see Figure 2) and the OTP encryptions in $\hat{f}$ as well as the query-response pairs in a fixed transcript τ .

We begin with the bound ε_2 for any good transcript. According to the pairwise distinctness of permutation entries kept in good transcripts, the N “bonus” queries, the N effective OTP keys solved from the system (1), and the other q distinct permutation queries jointly fix $q + 2N$ linkages between the pre-images and images of $\pi : \{0, 1\}^\lambda \rightarrow \{0, 1\}^\lambda$. Taking over the uniform W ’s for an n -bit input, the random permutation π with fixed $q + 2N$ linkages, and the uniform global key Δ with $\text{lsb}(\Delta) = 1$, it holds that

$$\Pr_{\mathcal{O}_0} [\mathcal{O}_0 \in \text{comp}_0(\tau)] = \frac{1}{(2^\lambda)^n} \cdot \frac{(2^\lambda - q - 2N)!}{(2^\lambda)!} \cdot \frac{1}{2^{\lambda-1}}.$$

In the ideal world, each OTP encryption G is sampled at random and may not satisfy (1) to fix a linkage of form $(X \oplus \Delta \oplus k, \pi(X \oplus \Delta \oplus k))$. Such a linkage is also not fixed by a good transcript. In addition, the active label X for each of the n input bits is random in the ideal world while those for the internal wires are also fixed and consistent with the garbled input and “bonus” queries. Thus, the random permutation π only has $q + N$ fixed linkages in the ideal world, and

$$\Pr_{\mathcal{O}_1} [\mathcal{O}_1 \in \text{comp}_1(\tau)] = \frac{1}{(2^\lambda)^n} \cdot \frac{(2^\lambda - q - N)!}{(2^\lambda)!} \cdot \frac{1}{(2^\lambda)^N} \cdot \frac{1}{2^{\lambda-1}}.$$

As a result, we have $\varepsilon_2 = 0$ since, for any $q, N \geq 0$,

$$\frac{\Pr_{\mathcal{O}_0} [\mathcal{O}_0 \in \text{comp}_0(\tau)]}{\Pr_{\mathcal{O}_1} [\mathcal{O}_1 \in \text{comp}_1(\tau)]} = \frac{(2^\lambda)_{q+N} \cdot (2^\lambda)^N}{(2^\lambda)_{q+2N}} = \frac{(2^\lambda)^N}{(2^\lambda - q - N)_N} \geq 1.$$

Then, we claim that a negligible ε_1 can bound the probability of bad transcripts in the ideal world. This claim resorts to a negligible probability of any pairwise collision or the existence of any Δ -dependent query-response pair. Intuitively, the former probability results from the almost uniform pre-images of form $X \oplus k$ or images of form $\pi(X \oplus k)$. The latter one is from the entropy of Δ since no query-response pair can depend on it.

In the pRPM, transcripts are of the same syntax as in the npRPM, except that the decoding information d is given in the offline garbled circuit $\hat{f}$ rather than the online garbled input $\hat{x}$. Once the

programming succeeds (additionally guaranteed by each good transcript in the pRPM), the decoding information d should be consistent with the active labels and the truth bits on the output wires, as in the npRPM. Given the same free-XOR encoding of garbled input, the analysis of ε_2 in the pRPM is similar to that in the npRPM. However, the bound ε_1 will increase as bad transcripts also consider failed programming in the pRPM. Note that any permutation entry of form $(X \oplus k, \pi(X \oplus k))$ must never be occupied by the query-response pairs *before the online phase* to fulfill programming. Therefore, it follows from the randomness of such pre-images and images that the increased ε_1 is still negligible.

4 Preliminaries

4.1 Notation

Let $\lambda \in \mathbb{N}^+$ denote the security parameter. We use $\text{poly}(\cdot)$ (resp., $\text{negl}(\cdot)$) for an unspecified polynomial (resp., negligible) function. For $a, b \in \mathbb{N}$ with $a \leq b$, we use $[a, b]$ to denote the set $\{a, \dots, b\}$ and $(b)_a$ to denote the falling factorial $b \cdot (b-1) \cdots (b-a+1)$. We use $x \leftarrow S$ to be the uniform sampling of x from a finite set S . We use $:=$ to denote assigning a value or an output of a deterministic algorithm to a left-hand variable. Let $\mathcal{S}_\ell$ denote the set of permutations on $\{0, 1\}^\ell$. Let $\text{lsb}(x)$ denote the least significant bit (LSB) of $x \in \{0, 1\}^n$. Let $\parallel$ denote the concatenation of bit-strings. Let $\ominus$ denote the symmetric difference of sets, i.e., for two sets A, B , $A \ominus B := (A \setminus B) \cup (B \setminus A)$.

Linear orthomorphism. A permutation $\sigma : \mathbb{G} \rightarrow \mathbb{G}$ over an additive Abelian group $\mathbb{G}$ is a linear orthomorphism if (i) $\sigma(x + y) = \sigma(x) + \sigma(y)$ for any $x, y \in \mathbb{G}$, (ii) $\sigma'(x) := \sigma(x) - x$ is also a permutation, and (iii) σ, σ' and their inverses are efficiently computable. There are two instantiations in [GKWY20]: (i) if $\mathbb{G}$ is a field, $\sigma(x) := c \cdot x$ for some $c \neq 0, 1 \in \mathbb{G}$, and (ii) if $\mathbb{G} = \{0, 1\}^n$, $\sigma(x) := (x_L \oplus x_R) \parallel x_L$ where x_L and x_R are the left and right halves of x .

Circuits. In a circuit f with fan-in two and fan-out one:

- $|f|$: The number of AND gates.
- $\mathcal{W}(f)$, $\mathcal{W}_{\text{in}}(f)$, $\mathcal{W}_{\text{out}}(f)$, $\mathcal{W}_{\text{and}}(f)$: The sets of wires, circuit input wires, circuit output wires, and output wires of AND gates, respectively.
- $\mathcal{G}(f)$, $\mathcal{G}_{\text{and}}(f)$: The sets of gates and AND gates, respectively.
- For a gate $g \in \mathcal{G}(f)$, let $(a, b) := (\text{in}_0(g), \text{in}_1(g))$ denote the two input wires and $c := \text{out}(g)$ denote the output wire.

In Appendix A, we present the notation used in the appendices.

4.2 Random Permutation Model

In the RPM [RS08, BHKR13], all parties have oracle access to random permutation π and its inverse $\pi^{-1} := \text{inv}(\pi)$. We refer to queries to π as *forward queries* and queries to π^{-1} as *backward queries*. In this work, we consider *non-programmable RPM* and *programmable RPM*. Like the separation [Nie02] regarding the random oracle model [BR93], we formalize the pRPM as a hybrid model, where an ideal functionality provides two $\pi^{\pm 1}$ interfaces. This model gives the simulator the power to “appropriately” choose the responses to the adversary’s queries to $\pi^{\pm 1}$. In contrast, all parties, the real-world adversary, and the simulator in the npRPM have access to a global $\pi^{\pm 1}$, whose responses cannot be chosen by the simulator.

4.3 Adaptive Security of Garbling Schemes

In Definition 1, we adapt the definition of adaptively secure garbling schemes in [HJO⁺16, JW16] for a q -query *computationally unbounded non-uniform* adversary in the npRPM and the pRPM. All algorithms and the adversary implicitly take the unary security parameter 1^λ as input.

Definition 1 (Adaptively secure garbling scheme). *For a polynomial ℓ , an $\ell(\lambda)$ -garbling scheme in the npRPM/pRPM includes three PPT algorithms (Garble, Encode, DecEval), each of which has access to a random permutation $\pi \in \mathcal{S}_{\ell(\lambda)}$ and its inverse $\pi^{-1} := \text{inv}(\pi)$.*

Given two polynomials q, s , a negligible function ε , and a side-information function Φ , this scheme is $(q(\lambda), s(\lambda), \varepsilon(\lambda), \Phi)$ -adaptively secure in the npRPM (resp., pRPM) if it complies with **correctness** and **adaptive security in the npRPM** (resp., **pRPM**). By default, $\Phi(f) = f$ is omitted in the definition.

- **Correctness.** For every polynomial-size circuit $f : \{0, 1\}^{\ell_{\text{in}}} \rightarrow \{0, 1\}^{\ell_{\text{out}}}$ and input $x \in \{0, 1\}^{\ell_{\text{in}}}$, it holds that

$$\Pr \left[\begin{array}{l} \pi \leftarrow \mathcal{S}_{\ell(\lambda)}, \pi^{-1} := \text{inv}(\pi), \\ (\hat{f}, k) \leftarrow \text{Garble}^{\pi^{\pm 1}(\cdot)}(f), : \text{DecEval}^{\pi^{\pm 1}(\cdot)}(\hat{f}, \hat{x}) = f(x) \\ \hat{x} := \text{Encode}^{\pi^{\pm 1}(\cdot)}(k, x) \end{array} \right] = 1.$$

- **Adaptive security in the npRPM.** There exists a PPT simulator $\text{Sim} = (\text{SimF}, \text{SimIn})$ with a state st_{sim} s.t., for every auxiliary input $z \in \{0, 1\}^*$ and computationally unbounded adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2, \mathcal{A}_3)$ making $q(\lambda)$ oracle queries and choosing a circuit with $s(\lambda)$ AND gates, it holds that

$$\left| \Pr \left[\begin{array}{l} \pi \leftarrow \mathcal{S}_{\ell(\lambda)}, \pi^{-1} := \text{inv}(\pi), \\ (f, \text{st}_1) \leftarrow \mathcal{A}_1^{\pi^{\pm 1}(\cdot)}(z), \\ (\hat{f}, k) \leftarrow \text{Garble}^{\pi^{\pm 1}(\cdot)}(f), : \mathcal{A}_3^{\pi^{\pm 1}(\cdot)}(\text{st}_2, \hat{f}, \hat{x}) = 1 \\ (x, \text{st}_2) \leftarrow \mathcal{A}_2^{\pi^{\pm 1}(\cdot)}(\text{st}_1, \hat{f}), \\ \hat{x} := \text{Encode}^{\pi^{\pm 1}(\cdot)}(k, x) \end{array} \right] - \Pr \left[\begin{array}{l} \pi \leftarrow \mathcal{S}_{\ell(\lambda)}, \pi^{-1} := \text{inv}(\pi), \\ (f, \text{st}_1) \leftarrow \mathcal{A}_1^{\pi^{\pm 1}(\cdot)}(z), \\ \hat{f} \leftarrow \text{SimF}^{\pi^{\pm 1}(\cdot)}(\Phi(f)), : \mathcal{A}_3^{\pi^{\pm 1}(\cdot)}(\text{st}_2, \hat{f}, \hat{x}) = 1 \\ (x, \text{st}_2) \leftarrow \mathcal{A}_2^{\pi^{\pm 1}(\cdot)}(\text{st}_1, \hat{f}), \\ \hat{x} \leftarrow \text{SimIn}^{\pi^{\pm 1}(\cdot)}(f(x)) \end{array} \right] \right| \leq \varepsilon(\lambda).$$

- **Adaptive security in the pRPM.** There exists a PPT simulator $\text{Sim} = (\text{SimF}, \text{SimIn}, \text{SimP}^{\pm 1})$ with a state st_{sim} s.t., for every auxiliary input $z \in \{0, 1\}^*$ and computationally unbounded adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2, \mathcal{A}_3)$ making $q(\lambda)$ oracle queries and choosing a circuit with $s(\lambda)$ AND gates, it holds that

$$\left| \Pr \left[\begin{array}{l} \pi \leftarrow \mathcal{S}_{\ell(\lambda)}, \pi^{-1} := \text{inv}(\pi), \\ (f, \text{st}_1) \leftarrow \mathcal{A}_1^{\pi^{\pm 1}(\cdot)}(z), \\ (\hat{f}, k) \leftarrow \text{Garble}^{\pi^{\pm 1}(\cdot)}(f), : \mathcal{A}_3^{\pi^{\pm 1}(\cdot)}(\text{st}_2, \hat{f}, \hat{x}) = 1 \\ (x, \text{st}_2) \leftarrow \mathcal{A}_2^{\pi^{\pm 1}(\cdot)}(\text{st}_1, \hat{f}), \\ \hat{x} := \text{Encode}^{\pi^{\pm 1}(\cdot)}(k, x) \end{array} \right] - \Pr \left[\begin{array}{l} (f, \text{st}_1) \leftarrow \mathcal{A}_1^{\text{SimP}^{\pm 1}(\cdot)}(z), \\ \hat{f} \leftarrow \text{SimF}(\Phi(f)), : \mathcal{A}_3^{\text{SimP}^{\pm 1}(\cdot)}(\text{st}_2, \hat{f}, \hat{x}) = 1 \\ (x, \text{st}_2) \leftarrow \mathcal{A}_2^{\text{SimP}^{\pm 1}(\cdot)}(\text{st}_1, \hat{f}), \\ \hat{x} \leftarrow \text{SimIn}(f(x)) \end{array} \right] \right| \leq \varepsilon(\lambda).$$

4.4 Blueprint of Adaptive Security and H-coefficient Technique

In our blueprint, we consider a computationally unbounded non-uniform adversary $\mathcal{A}$, which takes an auxiliary input and outputs a decision bit after making a bounded number of adaptive queries to either a real-world or ideal-world oracle. Each oracle is *stateful*: for a query, a response is a *deterministic* function of the random tape, the query, and previous query-response pairs. The two oracles have multiple interfaces, each of which has the same syntax in the two worlds. For simplicity, whenever we refer to a non-uniform adversary (e.g., $\mathcal{A}$ or $\mathcal{A}'$), we assume that it has some auxiliary input fixed in its context unless this auxiliary input is given explicitly.

Without loss of generality, we consider a *deterministic* $\mathcal{A}$. The interaction between $\mathcal{A}$ and the oracle produces a transcript, which gives an *ordered* list of query-response pairs from the view of $\mathcal{A}$. For some fixed $\mathcal{A}$, the distribution of transcripts in either world comes from the random sampling of a *deterministic* oracle. The decision bit of $\mathcal{A}$ is a deterministic function of its auxiliary input and a transcript produced in the interaction, and its advantage is at most the statistical distance of transcripts in the two worlds.

Let Ω_{real} (resp., Ω_{ideal}) be the sample space where a *deterministic* real-world (resp., ideal-world) oracle is sampled. For any fixed $\mathcal{A}$, let $\mathcal{T}_{\mathcal{A}}$ be the set of *attainable transcripts* such that a transcript $\tau \in \mathcal{T}_{\mathcal{A}}$ if and only if there exists a *deterministic* oracle $\omega' \in \Omega_{\text{ideal}}$ such that the interaction between $\mathcal{A}$ and ω' produces τ . Let $X : \Omega_{\text{real}} \rightarrow \mathcal{T}_{\mathcal{A}}$ (resp., $Y : \Omega_{\text{ideal}} \rightarrow \mathcal{T}_{\mathcal{A}}$) be the random variable for the transcripts produced in the interaction between $\mathcal{A}$ and a real-world (resp., ideal-world) oracle $\omega \leftarrow \Omega_{\text{real}}$ (resp., $\omega \leftarrow \Omega_{\text{ideal}}$). For any fixed τ , let $\text{comp}_{\text{real}}(\tau) \subseteq \Omega_{\text{real}}$ (resp., $\text{comp}_{\text{ideal}}(\tau) \subseteq \Omega_{\text{ideal}}$) be the set of *compatible* real-world (resp., ideal-world) oracles such that an oracle $\omega \in \text{comp}_{\text{real}}(\tau)$ (resp., $\omega \in \text{comp}_{\text{ideal}}(\tau)$) if and only if there exists a *deterministic* non-uniform adversary $\mathcal{A}'$ such that the interaction between $\mathcal{A}'$ and ω produces τ .

Our main insight to peel the adaptivity in X, Y is formalized in Lemma 1. Its proof was sketched in [DLMS14, Appendix D], and we also present a formal proof in Appendix B for completeness.

Lemma 1 ([DLMS14]). *We use the notation defined in Section 4.4. For every auxiliary input $z \in \{0, 1\}^*$, every computationally unbounded adversary $\mathcal{A}$, and every attainable transcript $\tau \in \mathcal{T}_{\mathcal{A}(z)}$, we have*

$$\begin{aligned} \Pr_{\omega \leftarrow \Omega_{\text{real}}} [X(\omega) = \tau] &= \Pr_{\omega \leftarrow \Omega_{\text{real}}} [\omega \in \text{comp}_{\text{real}}(\tau)], \\ \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [Y(\omega) = \tau] &= \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\omega \in \text{comp}_{\text{ideal}}(\tau)] > 0. \end{aligned}$$

For every fixed non-uniform $\mathcal{A}$, the H-coefficient technique [Pat09, CS14, DLMS14] gives a bound on the statistical distance of transcripts. It divides $\mathcal{T}_{\mathcal{A}}$ into two disjoint subsets $\mathcal{T}_{\text{bad}} \subseteq \mathcal{T}_{\mathcal{A}}$ and $\mathcal{T}_{\text{good}} := \mathcal{T}_{\mathcal{A}} \setminus \mathcal{T}_{\text{bad}}$ and proves an upper bound $\varepsilon_1 + \varepsilon_2$ of this statistical distance if $\Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [Y(\omega) \in \mathcal{T}_{\text{bad}}] \leq \varepsilon_1$ and

$$\forall \tau \in \mathcal{T}_{\text{good}} : \frac{\Pr_{\omega \leftarrow \Omega_{\text{real}}} [X(\omega) = \tau]}{\Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [Y(\omega) = \tau]} \geq 1 - \varepsilon_2.$$

Putting this bound with Lemma 1, we obtain the blueprint in Figure 1.

5 Adaptive Security of Half-Gates in npRPM

We prove that half-gates is adaptively secure in the npRPM. This scheme is implemented as in Figure 3, which includes decoding information d in garbled input $\hat{x}$ rather than garbled circuit $\hat{f}$. We will see in Appendix E that this inclusion is to respect the online-complexity lower bound in the npRPM.

Theorem 1. *Let $H(X, k) = \pi(X \oplus k) \oplus \sigma(X \oplus k)$ be a tweakable hash function where $X, k \in \{0, 1\}^\lambda$, $\pi \in \mathcal{S}_\lambda$ is random permutation, and $\sigma : \{0, 1\}^\lambda \rightarrow \{0, 1\}^\lambda$ is a linear orthomorphism. Then, half-gates (Figure 3) is a λ -garbling scheme with (q, s, ε) -adaptive security in the npRPM, where $\varepsilon = (16qs + 38s^2)/2^\lambda$.*

Proof. The correctness is given by the proof [ZRE15] as postponing decoding information d does not affect correctness. We only need to consider the simulation. Our simulator $\text{Sim} = (\text{SimF}, \text{SimIn})$ is presented in Figure 4 and is obviously PPT. Then, we prove this theorem using the following three hybrids:

- Hybrid₀. This is the adaptive experiment with simulator Sim .
- Hybrid₁. This is identical to the previous hybrid, except that we replace $\pi^{\pm 1}$ (which can be emulated on-the-fly as in Figure 5) by an approximation $\tilde{\pi}^{\pm 1}$ (Figure 6). This approximation is the same as random permutation except that, for a new query of the simulator, it returns a fresh random string as response and records this query-response pair. This hybrid is used to simplify probability analysis.

HG.Garble$^{\pi^{\pm 1}(\cdot)}(f)$: 1: $\Delta \leftarrow \{0, 1\}^{\lambda-1} \parallel 1$ 2: for $i \in \mathcal{W}_{\text{in}}(f)$ do 3: $W_i \leftarrow \{0, 1\}^\lambda$ 4: for $g \in \mathcal{G}(f)$ in order do 5: $(a, b, c) := (\text{in}_0(g), \text{in}_1(g), \text{out}(g))$ 6: if $\text{type}(g) = \text{XOR}$ do $W_c := W_a \oplus W_b$ 7: else if $\text{type}(g) = \text{AND}$ do 8: $k_0^g := 2 \cdot g - 1, k_1^g := 2 \cdot g$ 9: $p_a := \text{lsb}(W_a), p_b := \text{lsb}(W_b)$ 10: $G_0^g := H(W_a, k_0^g) \oplus H(W_a \oplus \Delta, k_0^g) \oplus p_b \Delta$ 11: $G_1^g := H(W_b, k_1^g) \oplus H(W_b \oplus \Delta, k_1^g) \oplus W_a$ 12: $W_c := H(W_a \oplus p_a \Delta, k_0^g) \oplus H(W_b \oplus p_b \Delta, k_1^g) \oplus p_a p_b \Delta$ 13: for $i \in \mathcal{W}_{\text{out}}(f)$ do $d_i := \text{lsb}(W_i)$ 14: return $\hat{f} := (f' := f, F := \{(G_0^g, G_1^g)\}_{g \in \mathcal{G}_{\text{and}}(f)}), k := (f, d, \Delta, W)$ HG.DecEval$^{\pi^{\pm 1}(\cdot)}(\hat{f}, \hat{x})$: 1: Parse $\hat{f} = (f, \{(G_0^g, G_1^g)\}_{g \in \mathcal{G}_{\text{and}}(f)}), \hat{x} = (\{X_i\}_{i \in \mathcal{W}_{\text{in}}(f)}, d)$ 2: for $g \in \mathcal{G}(f)$ in order do 3: $(a, b, c) := (\text{in}_0(g), \text{in}_1(g), \text{out}(g))$ 4: if $\text{type}(g) = \text{XOR}$ do $X_c := X_a \oplus X_b$ 5: else if $\text{type}(g) = \text{AND}$ do 6: $k_0^g := 2 \cdot g - 1, k_1^g := 2 \cdot g$ 7: $s_a := \text{lsb}(X_a), s_b := \text{lsb}(X_b)$ 8: $X_c := H(X_a, k_0^g) \oplus H(X_b, k_1^g) \oplus s_a G_0^g \oplus s_b (G_1^g \oplus X_a)$ 9: for $i \in \mathcal{W}_{\text{out}}(f)$ do $y_i := d_i \oplus \text{lsb}(X_i)$ 10: return y	HG.Encode$^{\pi^{\pm 1}(\cdot)}(k, x)$: 1: Parse $k = (f, d, \Delta, W)$ 2: for $i \in \mathcal{W}_{\text{in}}(f)$ do 3: $X_i := W_i \oplus x_i \Delta$ 4: return $\hat{x} := (\{X_i\}_{i \in \mathcal{W}_{\text{in}}(f)}, d)$
--	---

Figure 3: Half-gates garbling scheme [ZRE15].

SimF$^{\pi^{\pm 1}(\cdot)}(f)$: 1: $F := \{(G_0^g, G_1^g)\}_{g \in \mathcal{G}_{\text{and}}(f)} \leftarrow (\{0, 1\}^{2\lambda})^{ f }$ 2: $\{X_i\}_{i \in \mathcal{W}_{\text{in}}(f)} \leftarrow (\{0, 1\}^\lambda)^{ \mathcal{W}_{\text{in}}(f) }$ 3: for $g \in \mathcal{G}(f)$ in order do 4: $(a, b, c) := (\text{in}_0(g), \text{in}_1(g), \text{out}(g))$ 5: if $\text{type}(g) = \text{XOR}$ do $X_c := X_a \oplus X_b$ 6: else if $\text{type}(g) = \text{AND}$ do 7: $k_0^g := 2 \cdot g - 1, k_1^g := 2 \cdot g$ 8: $s_a := \text{lsb}(X_a), s_b := \text{lsb}(X_b)$ 9: $U_0^g := \pi(X_a \oplus k_0^g) \oplus \sigma(X_a \oplus k_0^g)$ 10: $U_1^g := \pi(X_b \oplus k_1^g) \oplus \sigma(X_b \oplus k_1^g)$ 11: $X_c := U_0^g \oplus U_1^g \oplus s_a G_0^g \oplus s_b (G_1^g \oplus X_a)$ 12: return $\hat{f} := (f, F), \text{st}_{\text{sim}} := (f, \tilde{X} := \{X_i\}_{i \in \mathcal{W}(f)}, \tilde{U} := \{U_0^g, U_1^g\}_{g \in \mathcal{G}_{\text{and}}(f)})$ SimIn$^{\pi^{\pm 1}(\cdot)}(f(x))$: 1: Parse $\text{st}_{\text{sim}} = (f, \tilde{X} = \{X_i\}_{i \in \mathcal{W}(f)}, \tilde{U})$ 2: for $i \in \mathcal{W}_{\text{out}}(f)$ do $d_i := f(x)_i \oplus \text{lsb}(X_i)$ 3: return $\hat{x} := (\{X_i\}_{i \in \mathcal{W}_{\text{in}}(f)}, d), \tilde{X}, \tilde{U}$.

Figure 4: Our simulator for half-gates in the nPRPM.

- Hybrid₂. This is the adaptive experiment with half-gates.

By using Lemma 3 for the indistinguishability between Hybrid₀ and Hybrid₁ and Lemma 4 for that between Hybrid₁ and Hybrid₂, we prove this theorem. $\square$

First, to bound the advantage of any q -query adaptive adversary in distinguishing between Hybrid_0 and Hybrid_1 , we introduce the following lemma.

Lemma 2. Let $\tilde{\mathcal{P}}_{\ell(\lambda)}$ denote the distribution of $\tilde{\pi}^{\pm 1}$ in Figure 6 for $n(\lambda) \in \mathbb{N}^+$ queries. For every PPT simulator $\text{Sim} = (\text{SimF}, \text{SimIn})$ making $n_{\text{sim}}(\lambda) \leq n(\lambda)$ queries, every auxiliary input $z \in \{0, 1\}^*$, and every computationally unbounded adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2, \mathcal{A}_3)$ making $n(\lambda) - n_{\text{sim}}(\lambda)$ queries, it holds that

$$\left| \Pr \left[\begin{array}{l} \pi \leftarrow \mathcal{S}_{\ell(\lambda)}, \pi^{-1} := \text{inv}(\pi), \\ (f, \text{st}_1) \leftarrow \mathcal{A}_1^{\pi^{\pm 1}(\cdot)}(z), \\ \hat{f} \leftarrow \text{SimF}^{\pi^{\pm 1}(\cdot)}(\Phi(f)), : \mathcal{A}_3^{\pi^{\pm 1}(\cdot)}(\text{st}_2, \hat{f}, \hat{x}) = 1 \\ (x, \text{st}_2) \leftarrow \mathcal{A}_2^{\pi^{\pm 1}(\cdot)}(\text{st}_1, \hat{f}), \\ \hat{x} \leftarrow \text{SimIn}^{\pi^{\pm 1}(\cdot)}(f(x)) \end{array} \right] - \Pr \left[\begin{array}{l} \tilde{\pi}^{\pm 1} \leftarrow \tilde{\mathcal{P}}_{\ell(\lambda)}, \\ (f, \text{st}_1) \leftarrow \mathcal{A}_1^{\tilde{\pi}^{\pm 1}(\cdot)}(z), \\ \hat{f} \leftarrow \text{SimF}^{\tilde{\pi}^{\pm 1}(\cdot)}(\Phi(f)), : \mathcal{A}_3^{\tilde{\pi}^{\pm 1}(\cdot)}(\text{st}_2, \hat{f}, \hat{x}) = 1 \\ (x, \text{st}_2) \leftarrow \mathcal{A}_2^{\tilde{\pi}^{\pm 1}(\cdot)}(\text{st}_1, \hat{f}), \\ \hat{x} \leftarrow \text{SimIn}^{\tilde{\pi}^{\pm 1}(\cdot)}(f(x)) \end{array} \right] \right| \leq \frac{n(\lambda) \cdot n_{\text{sim}}(\lambda)}{2^{\ell(\lambda)-1}}.$$

Proof. Let $\mathcal{N} \subseteq [1, n(\lambda)]$ be the index set of the queries made by Sim such that $|\mathcal{N}| = n_{\text{sim}}(\lambda)$. Let $\text{true}(\pi)$ denote the event that $\mathcal{A}_3^{\pi^{\pm 1}(\cdot)}(\text{st}_2, \hat{f}, \hat{x}) = 1$ in the first experiment, $\text{true}(\tilde{\pi})$ denote the counterpart in the second one, and bad denote the event that $\forall i \in \mathcal{N} ((\dots, c_i) \in \mathcal{Q}_{i-1} \vee (c_i, \dots) \in \mathcal{Q}_{i-1})$. We study the two experiments under the same probability space, i.e., they use the same literal values of (r_π, r_π^*) and the random tapes of Sim and $\mathcal{A}$. Note that $\text{true}(\pi) \wedge \neg \text{bad}$ occurs if and only if $\text{true}(\tilde{\pi}) \wedge \neg \text{bad}$ occurs, i.e., the two experiments are identical unless bad occurs. From the Difference Lemma [Sho04],

$$\left| \Pr[\text{true}(\pi)] - \Pr[\text{true}(\tilde{\pi})] \right| \leq \Pr[\text{bad}] \leq \sum_{i \in \mathcal{N}} \frac{2|\mathcal{Q}_{i-1}|}{2^{\ell(\lambda)}} \leq \frac{n(\lambda) \cdot n_{\text{sim}}(\lambda)}{2^{\ell(\lambda)-1}},$$

which completes this proof. $\square$

Lemma 3. Let Sim be defined as in Figure 4. Then, for every auxiliary input $z \in \{0, 1\}^*$ and every computationally unbounded adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2, \mathcal{A}_3)$ making q oracle queries and choosing a circuit with s AND gates, $\mathcal{A}(z)$ can distinguish Hybrid_0 and Hybrid_1 with advantage at most $(2qs + 4s^2)/2^{\lambda-1}$.

Proof. This lemma follows from Lemma 2 by using $\ell(\lambda) = \lambda$, $n_{\text{sim}}(\lambda) = 2|f|$, and $n(\lambda) = q + n_{\text{sim}}(\lambda)$, given q queries of $\mathcal{A}$ and $2|f| = 2s$ queries of Sim . $\square$

Second, we combine our transcript padding with the H-coefficient technique to bound the advantage of distinguishing Hybrid_1 and Hybrid_2 .

Lemma 4. Let Sim be defined as in Figure 4. Then, for every auxiliary input $z \in \{0, 1\}^*$ and every computationally unbounded adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2, \mathcal{A}_3)$ making q oracle queries and choosing a circuit with s AND gates, $\mathcal{A}(z)$ can distinguish Hybrid_1 and Hybrid_2 with advantage at most $(12qs + 30s^2)/2^\lambda$.

Proof. Fix z and $\mathcal{A}$. We view Hybrid_1 (resp., Hybrid_2) as the ideal (resp., real) world in the H-coefficient technique, where the computationally unbounded non-uniform adversary $\mathcal{A} = \mathcal{A}(z)$ and $\varepsilon_1, \varepsilon_2$ are computed as follows.

Transcript padding. In either world, $\mathcal{A}$ will interact with an integrated oracle that acts as the two-round challenger in the adaptive experiment and provides interfaces $\pi^{*\pm 1} \in \{\pi^{\pm 1}, \tilde{\pi}^{\pm 1}\}$ for forward/backward permutation queries. Here, $\mathcal{A}$ can learn $\pi^*(\alpha) = \beta$ if and only if it sent forward query α to π^* and received response β , or sent backward query β to π^{*-1} and received response α .

To compute $\varepsilon_1, \varepsilon_2$ more easily, we ask the oracle to send more messages to $\mathcal{A}$ and $\mathcal{A}$ to make extra queries (in addition to the supposed q queries) in both two worlds. More specifically,

```

1: Initialize a list  $\mathcal{Q}_0 = \emptyset$ .
2:  $c_1, \dots, c_{n(\lambda)} \leftarrow \{0, 1\}^{\ell(\lambda)}$  a.
3: for  $i \in [1, n(\lambda)]$  do
4:   if  $\pi$  is queried with  $\alpha_i \in \{0, 1\}^{\ell(\lambda)}$  do
5:     if  $\exists (\alpha_i, \gamma_i) \in \mathcal{Q}_{i-1}$  do Return  $\gamma_i$ .
6:     if  $(\dots, c_i) \notin \mathcal{Q}_{i-1}$  do  $\gamma_i := c_i$ .
7:     else  $\gamma_i \leftarrow \{s_i \in \{0, 1\}^{\ell(\lambda)} \mid (\dots, s_i) \notin \mathcal{Q}_{i-1}\}$  b.
8:     Return  $\gamma_i$  and define  $\mathcal{Q}_i := \mathcal{Q}_{i-1} \cup \{(\alpha_i, \gamma_i)\}$ .
9:   else if  $\pi^{-1}$  is queried with  $\beta_i \in \{0, 1\}^{\ell(\lambda)}$  do
10:    if  $\exists (\gamma_i, \beta_i) \in \mathcal{Q}_{i-1}$  do Return  $\gamma_i$ .
11:    if  $(c_i, \dots) \notin \mathcal{Q}_{i-1}$  do  $\gamma_i := c_i$ .
12:    else  $\gamma_i \leftarrow \{s_i \in \{0, 1\}^{\ell(\lambda)} \mid (s_i, \dots) \notin \mathcal{Q}_{i-1}\}$  b.
13:    Return  $\gamma_i$  and define  $\mathcal{Q}_i := \mathcal{Q}_{i-1} \cup \{(\gamma_i, \beta_i)\}$ .

```

^aA uniform random tape r_π is used.

^bA uniform random tape r_π^* is used to run rejection sampling.

Figure 5: The workflow of oracle $\pi^{\pm 1}(\cdot)$ up to $n(\lambda)$ queries.

```

1: Initialize a list  $\mathcal{Q}_0 = \emptyset$ .
2:  $c_1, \dots, c_{n(\lambda)} \leftarrow \{0, 1\}^{\ell(\lambda)}$ .
3: for  $i \in [1, n(\lambda)]$  do
4:   if  $\tilde{\pi}$  is queried with  $\alpha_i \in \{0, 1\}^{\ell(\lambda)}$  by Sim do
5:     if  $\exists (\alpha_i, \gamma_i) \in \mathcal{Q}_{i-1}$  do Return  $\gamma_i$ .
6:     Return  $\gamma_i := c_i$  and define  $\mathcal{Q}_i := \mathcal{Q}_{i-1} \cup \{(\alpha_i, \gamma_i)\}$ .
7:   else if  $\tilde{\pi}^{-1}$  is queried with  $\beta_i \in \{0, 1\}^{\ell(\lambda)}$  by Sim do
8:     if  $\exists (\gamma_i, \beta_i) \in \mathcal{Q}_{i-1}$  do Return  $\gamma_i$ .
9:     Return  $\gamma_i := c_i$  and define  $\mathcal{Q}_i := \mathcal{Q}_{i-1} \cup \{(\gamma_i, \beta_i)\}$ .
10:  else if  $\tilde{\pi}$  is queried with  $\alpha_i \in \{0, 1\}^{\ell(\lambda)}$  by  $\mathcal{A}$  do
11:    Same as step 5 to 8 in  $\pi^{\pm 1}(\cdot)$  (Figure 5).
12:  else if  $\tilde{\pi}^{-1}$  is queried with  $\beta_i \in \{0, 1\}^{\ell(\lambda)}$  by  $\mathcal{A}$  do
13:    Same as step 10 to 13 in  $\pi^{\pm 1}(\cdot)$  (Figure 5).

```

Figure 6: The workflow of approximate oracle $\tilde{\pi}^{\pm 1}(\cdot)$ up to $n(\lambda)$ queries, where the differences are highlighted in box.

- Upon receiving x from $\mathcal{A}$, the oracle sends $(\tilde{X} := \{X_i\}_{i \in \mathcal{W}(f)}, d)$ rather than $\hat{x} := (\{X_i\}_{i \in \mathcal{W}_{\text{in}}(f)}, d)$ to $\mathcal{A}$. In addition to the active input labels given by $\hat{x}$, the former also gives the active internal and output labels. In the real world, the oracle can run $\text{HG.DecEval}^{\pi^{\pm 1}(\cdot)}$, which determines other active labels in $\tilde{X}$. In the ideal world, this $\tilde{X}$ can be directly output by SimIn .
- Along with $(\tilde{X}, d)$, the oracle sends $\tilde{x} := \{x_i\}_{i \in \mathcal{W}(f)}$ to $\mathcal{A}$, which are the wire truth values in the evaluation of $f(x)$. Both two oracles “echo” these values, which are self-evident to $\mathcal{A}$, to explicitly include them in transcripts. In the experiment, the real-world oracle uses $x = \{x_i\}_{i \in \mathcal{W}_{\text{in}}(f)}$ in $\text{HG.Encode}^{\pi^{\pm 1}(\cdot)}$, but the ideal-world oracle can only use $f(x) = \{x_i\}_{i \in \mathcal{W}_{\text{out}}(f)}$ in SimIn .
- Along with $(\tilde{X}, d)$, the oracle sends $\tilde{U} := \{U_0^g, U_1^g\}_{g \in \mathcal{G}_{\text{and}}(f)}$ to $\mathcal{A}$. In the real world, the oracle computes $U_0^g := \text{H}(X_a, k_0^g)$ and $U_1^g := \text{H}(X_b, k_1^g)$ per $g \in \mathcal{G}_{\text{and}}(f)$ with $(a, b) := (\text{in}_0(g), \text{in}_1(g))$. In the ideal world, this $\tilde{U}$ is output by SimIn and is essentially the same hash outputs.
- (Extra queries) Upon receiving $(\tilde{X}, d, \tilde{x}, \tilde{U})$ from the oracle, $\mathcal{A}$ also makes a forward permutation

query $X_a \oplus k_0^g$ (resp., $X_b \oplus k_1^g$) for each $g \in \mathcal{G}_{\text{and}}(f)$ with $(a, b) := (\text{in}_0(g), \text{in}_1(g))$, if it has never learned $\pi^*(X_a \oplus k_0^g) = Y$ (resp., $\pi^*(X_b \oplus k_1^g) = Y$) for some Y in its interaction with $\pi^{*\pm 1} \in \{\pi^{\pm 1}, \tilde{\pi}^{\pm 1}\}$.

- At the end of the experiment (i.e., once all other transcripts are settled), the oracle sends Δ to $\mathcal{A}$. In the real world, the oracle gets this Δ from the output of $\text{HG.Garble}^{\pi^{\pm 1}(\cdot)}$. In the ideal world, Δ is dummy and sampled by the oracle at this time (note that Sim does not use Δ).

According to the oracle constructions, real-world sample space

$$\Omega_{\text{real}} = \{0, 1\}^{\lambda-1} \times \{0, 1\}^{|\mathcal{W}_{\text{in}}(f)|\lambda} \times \mathcal{S}_\lambda,$$

and ideal-world sample space

$$\Omega_{\text{ideal}} = (\{0, 1\}^{2\lambda})^{|f|} \times (\{0, 1\}^\lambda)^{|\mathcal{W}_{\text{in}}(f)|} \times \underbrace{\{0, 1\}^*}_{\text{for } \tilde{\pi}^{\pm 1}(\cdot)} \times \underbrace{\{0, 1\}^{\lambda-1}}_{\text{dummy } \Delta}$$

With the padding, a transcript is of the form:

$$\tau = (\mathcal{K}_1, (f, \hat{f}), \mathcal{K}_2, (x, (\tilde{X}, d, \tilde{x}, \tilde{U})), \mathcal{K}_3, \Delta),$$

where $\mathcal{K}_1$, $\mathcal{K}_2$, and $\mathcal{K}_3$ are the ordered lists of query-response pairs seen in the interleaved interaction with permutation oracles. We do not explicitly consider query direction in these pairs. Given $\pi^{*\pm 1} \in \{\pi^{\pm 1}, \tilde{\pi}^{\pm 1}\}$, $\mathcal{A}$ learns $\pi^*(\alpha) = \beta$ if and only if there exists $(\alpha, \beta) \in \cup_{\ell=1}^3 \mathcal{K}_\ell$. Let $q_\ell := |\mathcal{K}_\ell|$ for every $\ell \in \{1, 2, 3\}$ and $q_\Sigma := \sum_{\ell=1}^3 q_\ell$. It follows from the extra queries that $q_\Sigma \leq q + 2|f|$. Without loss of generality, we assume that $\mathcal{A}$ only makes *non-repeating* queries, i.e., it never makes forward query α to π^* or backward query β to π^{*-1} for any learned permutation entry (α, β) .

Bad transcripts. Recall that the intuition behind our definition of bad transcripts are given in Section 3.3. Formally, a transcript $\tau \in \mathcal{T}_{\text{bad}}$ if and only if it incurs at least one of the following events³:

- bad_1 . There exist distinct $(g, u), (g', u') \in \mathcal{G}_{\text{and}}(f) \times \{0, 1\}$ s.t.

$$X_w \oplus k_u^g = X_{w'} \oplus k_{u'}^{g'} \quad \vee \quad (3)$$

$$U_u^g \oplus \sigma(X_w \oplus k_u^g) = U_{u'}^{g'} \oplus \sigma(X_{w'} \oplus k_{u'}^{g'}) \quad (4)$$

where $w := \text{in}_u(g)$ and $w' := \text{in}_{u'}(g')$, or $g \in \mathcal{G}_{\text{and}}(f)$ s.t.

$$(s_b \oplus x_b)\Delta \oplus G_0^g \oplus U_0^g \oplus \sigma(X_a \oplus k_0^g) = x_a\Delta \oplus X_a \oplus G_1^g \oplus U_1^g \oplus \sigma(X_b \oplus k_1^g) \quad (5)$$

where $(a, b) := (\text{in}_0(g), \text{in}_1(g))$, or distinct $g, g' \in \mathcal{G}_{\text{and}}(f)$ s.t.

$$(s_b \oplus x_b)\Delta \oplus G_0^g \oplus U_0^g \oplus \sigma(X_a \oplus k_0^g) = (s_{b'} \oplus x_{b'})\Delta \oplus G_0^{g'} \oplus U_0^{g'} \oplus \sigma(X_{a'} \oplus k_0^{g'}) \quad \vee \quad (6)$$

$$x_a\Delta \oplus X_a \oplus G_1^g \oplus U_1^g \oplus \sigma(X_b \oplus k_1^g) = x_{a'}\Delta \oplus X_{a'} \oplus G_1^{g'} \oplus U_1^{g'} \oplus \sigma(X_{b'} \oplus k_1^{g'}) \quad \vee \quad (7)$$

$$(s_b \oplus x_b)\Delta \oplus G_0^g \oplus U_0^g \oplus \sigma(X_a \oplus k_0^g) = x_{a'}\Delta \oplus X_{a'} \oplus G_1^{g'} \oplus U_1^{g'} \oplus \sigma(X_{b'} \oplus k_1^{g'}) \quad \vee \quad (8)$$

$$x_a\Delta \oplus X_a \oplus G_1^g \oplus U_1^g \oplus \sigma(X_b \oplus k_1^g) = (s_{b'} \oplus x_{b'})\Delta \oplus G_0^{g'} \oplus U_0^{g'} \oplus \sigma(X_{a'} \oplus k_0^{g'}) \quad (9)$$

where $(a, b) := (\text{in}_0(g), \text{in}_1(g))$ and $(a', b') := (\text{in}_0(g'), \text{in}_1(g'))$.

In this case, $\mathcal{A}$ can check the consistency between $G_u^g \oplus G_{u'}^{g'}$ and Δ at the end of experiment *without* further sending required queries, which are computed from Δ , to a random permutation or its inverse. In the real world, the consistency certainly holds. However, the ideal-world garbled rows and Δ are independently sampled, leading to the consistency only with negligible probability. As a result, $\mathcal{A}$ has non-negligible advantage to distinguish the two worlds and the statistical distance, as an upper bound, also blows up.

³Strictly speaking, such bad_i 's are the disjunctive predicates of $\tau \in \mathcal{T}_{\mathcal{A}}$ to define, in set-builder notation, a set $\mathcal{T}_{\text{bad}} \subseteq \mathcal{T}_{\mathcal{A}}$ of bad transcripts. We treat them as “events” to avoid cumbersome notation. The formal events (i.e., the specific sets of oracles) w.r.t. bad_i 's are well-defined from $\mathcal{T}_{\text{bad}}$ and the two random variables in Section 4.4.

- bad_2 . There exists $((\alpha, \beta), g) \in \cup_{\ell=1}^3 \mathcal{K}_\ell \times \mathcal{G}_{\text{and}}(f)$ s.t.

$$\alpha = X_a \oplus \Delta \oplus k_0^g \quad \vee \quad (10)$$

$$\alpha = X_b \oplus \Delta \oplus k_1^g \quad \vee \quad (11)$$

$$\beta = \sigma(\Delta) \oplus (s_b \oplus x_b)\Delta \oplus G_0^g \oplus U_0^g \oplus \sigma(X_a \oplus k_0^g) \quad \vee \quad (12)$$

$$\beta = \sigma(\Delta) \oplus x_a\Delta \oplus X_a \oplus G_1^g \oplus U_1^g \oplus \sigma(X_b \oplus k_1^g) \quad (13)$$

where $(a, b) := (\text{in}_0(g), \text{in}_1(g))$.

In this case, $\mathcal{A}$ essentially makes it to guess Δ before receiving this value. It allows $\mathcal{A}$ to distinguish the real world, where every G_i^g is consistent with Δ , and the ideal world with a dummy Δ . So, the statistical distance blows up.

- bad_3 . There exists $((\alpha, \beta), g) \in \cup_{\ell=1}^2 \mathcal{K}_\ell \times \mathcal{G}_{\text{and}}(f)$ s.t.

$$\alpha = X_a \oplus k_0^g \quad \vee \quad (14)$$

$$\alpha = X_b \oplus k_1^g \quad \vee \quad (15)$$

$$\beta = U_0^g \oplus \sigma(X_a \oplus k_0^g) \quad \vee \quad (16)$$

$$\beta = U_1^g \oplus \sigma(X_b \oplus k_1^g) \quad (17)$$

where $(a, b) := (\text{in}_0(g), \text{in}_1(g))$.

In this case, $\mathcal{A}$ can make forward/backward queries w.r.t. some active labels before receiving active input labels and computing other active ones. We use this case to explicitly ensure that these query-response pairs are fixed by the queries to $\pi^{\pm 1}(\cdot)$ in the step 10 to 12 of $\text{HG.Garble}^{\pi^{\pm 1}(\cdot)}$ (when τ is produced in the real world) or the queries to $\tilde{\pi}^{\pm 1}(\cdot)$ in the step 9 of $\text{SimF}^{\tilde{\pi}^{\pm 1}(\cdot)}$ (when τ is produced in the ideal world), instead of the extra queries of $\mathcal{A}$. This case is used to simplify probability analysis.

Bounding $1 - \varepsilon_2$. Without loss of generality, we can consider some fixed good transcript τ such that $\Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [Y(\omega) = \tau] > 0$. Using Lemma 1, we turn to analyze the sampled oracle's compatibility (Section 4.4) with such a transcript, instead of the interaction between $\mathcal{A}$ and the sampled oracle.

Note that there is a computationally unbounded non-uniform adversary $\mathcal{A}'$ such that, for every oracle ω , it sends the queries in τ in order in its interaction with ω (e.g., $\mathcal{A}'$ has auxiliary input τ and sends its ordered queries). Fix $\mathcal{A}'$ in the following compatibility analysis so that any real-world or ideal-world oracle will receive the queries in τ in order. For a response c recorded in a fixed τ , let $\omega \vdash c$ denote the event that, fixing the ordered queries as per τ , oracle ω produces c given the corresponding query. Let $\mathcal{K}^R$ denote the order-preserving list of the responses in an ordered list $\mathcal{K}$ of permutation query-response pairs.

In the real world, an oracle $\omega = (\Delta, \{W_i\}_{i \in \mathcal{W}_{\text{in}}(f)}, \pi) \in \Omega_{\text{real}}$ according to half-gates and

$$\Pr_{\omega \leftarrow \Omega_{\text{real}}} [\omega \in \text{comp}_{\text{real}}(\tau)] = \Pr_{\omega \leftarrow \Omega_{\text{real}}} [\omega \vdash (\mathcal{K}_1^R, \hat{f}, \mathcal{K}_2^R, \tilde{X}, d, \tilde{x}, \tilde{U}, \mathcal{K}_3^R, \Delta)].$$

To begin with, every real-world ω certainly produces f' in $\hat{f}$ and $\tilde{x}$ in τ , being consistent with $\Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [Y(\omega) = \tau] > 0$. This non-zero probability implies that $(f', \tilde{x})$ in τ is honestly and deterministically computed from the fixed queries (f, x) . Otherwise, no ideal-world oracle, which computes $(f', \tilde{x})$ from the same deterministic procedure, can produce this transcript, contradicting the non-zero probability. As every real-world ω will follow the same deterministic procedure, it certainly produces the two values.

Meanwhile, a real-world oracle ω should have the same literal value of Δ as in τ . Conditioned on

the compatibility so far,

$$\begin{aligned}
& \Pr_{\omega \leftarrow \Omega_{\text{real}}} [\omega \in \text{comp}_{\text{real}}(\tau)] \\
&= \Pr_{\omega \leftarrow \Omega_{\text{real}}} [\omega \vdash (\mathcal{K}_1^R, F, \mathcal{K}_2^R, \tilde{X}, d, \tilde{U}, \mathcal{K}_3^R) \mid \omega \vdash (f', \tilde{x}) \wedge \omega \vdash \Delta] \\
&\quad \cdot \Pr_{\omega \leftarrow \Omega_{\text{real}}} [\omega \vdash (f', \tilde{x}) \wedge \omega \vdash \Delta] \\
&= \Pr_{\omega \leftarrow \Omega_{\text{real}}} [\omega \vdash (\mathcal{K}_1^R, F, \mathcal{K}_2^R, \tilde{X}, d, \tilde{U}, \mathcal{K}_3^R) \mid \omega \vdash (f', \tilde{x}) \wedge \omega \vdash \Delta] \\
&\quad \cdot \Pr_{\omega \leftarrow \Omega_{\text{real}}} [\omega \vdash (f', \tilde{x})] \cdot \Pr_{\omega \leftarrow \Omega_{\text{real}}} [\omega \vdash \Delta] \\
&= \Pr_{\omega \leftarrow \Omega_{\text{real}}} [\omega \vdash (\mathcal{K}_1^R, F, \mathcal{K}_2^R, \tilde{X}, d, \tilde{U}, \mathcal{K}_3^R) \mid \omega \vdash (f', \tilde{x}) \wedge \omega \vdash \Delta] \cdot \frac{1}{2^{\lambda-1}}.
\end{aligned}$$

Conditioned on the compatibility with $(f', \tilde{x}, \Delta)$, a real-world ω should also be compatible with $(\cup_{\ell=1}^3 \mathcal{K}_\ell^R, F, \tilde{U})$ and some active labels in $\tilde{X}$ such that

- (i) $\pi^{\pm 1}$ respects the query-response pairs fixed in $\cup_{\ell=1}^3 \mathcal{K}_\ell^R$.
- (ii) For each $i \in \mathcal{W}_{\text{in}}(f)$, it holds that $X_i = W_i \oplus x_i \Delta$.
- (iii) For each $g \in \mathcal{G}_{\text{and}}(f)$ with $(a, b) := (\text{in}_0(g), \text{in}_1(g))$,

$$\begin{aligned}
H(X_a, k_0^g) &:= \pi(X_a \oplus k_0^g) \oplus \sigma(X_a \oplus k_0^g) = U_0^g, \\
H(X_b, k_1^g) &:= \pi(X_b \oplus k_1^g) \oplus \sigma(X_b \oplus k_1^g) = U_1^g.
\end{aligned} \tag{18}$$

- (iv) For each $g \in \mathcal{G}_{\text{and}}(f)$ with $(a, b, c) := (\text{in}_0(g), \text{in}_1(g), \text{out}(g))$,

$$X_c = H(X_a, k_0^g) \oplus H(X_b, k_1^g) \oplus s_a G_0^g \oplus s_b (G_1^g \oplus X_a), \tag{19}$$

$$\begin{aligned}
G_0^g &= H(X_a, k_0^g) \oplus H(X_a \oplus \Delta, k_0^g) \oplus (s_b \oplus x_b) \Delta \\
G_1^g &= H(X_b, k_1^g) \oplus H(X_b \oplus \Delta, k_1^g) \oplus x_a \Delta \oplus X_a,
\end{aligned} \tag{20}$$

where $x_a, x_b, s_a = \text{lsb}(X_a), s_b = \text{lsb}(X_b)$ are given in τ .

Conditioned on the compatibility so far, every real-world oracle ω is always compatible with the leftover values in τ , i.e., decoding table d and other active labels in $\tilde{X}$, which are deterministically computed from XOR combination. The reason is that, for τ ensuring $\Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [Y(\omega) = \tau] > 0$, these values should be honestly determined by the conditioned values as in the real world. Otherwise, this probability will be zero for an ideal-world oracle, which obtains them from a consistent deterministic computation as per the conditioned values. As every real-world oracle ω honestly follows the real-world computation, this “leftover” compatibility must hold conditioned on the previous compatibility.

It remains to compute the conditional probabilities for (i) to (iv). Consider (iii) and (iv). We note that every good τ satisfying $\Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [Y(\omega) = \tau] > 0$ already implies (18) and (19). Otherwise, no ideal-world oracle, which computes these values according to the step 9 and 10 in SimF (using the given tweakable hash function), can produce τ , contradicting the non-zero probability.

Then, we rewrite (20), the leftover part of (iii) and (iv), as:

$$\mathcal{V} := \left\{ \begin{aligned} & g \in \mathcal{G}_{\text{and}}(f), (a, b) := (\text{in}_0(g), \text{in}_1(g)) : \\ & \underbrace{\pi(X_a \oplus k_0^g)}_{P_{g,0}} \oplus \underbrace{\pi(X_a \oplus \Delta \oplus k_0^g)}_{P_{g,2}} = \sigma(\Delta) \oplus (s_b \oplus x_b) \Delta \oplus G_0^g, \\ & \underbrace{\pi(X_b \oplus k_1^g)}_{P_{g,1}} \oplus \underbrace{\pi(X_b \oplus \Delta \oplus k_1^g)}_{P_{g,3}} = \sigma(\Delta) \oplus x_a \Delta \oplus X_a \oplus G_1^g \end{aligned} \right\}$$

As τ is good transcript, there are exactly $4|f|$ pairwise distinct permutation pre-images on the left hand (otherwise, there will be a pair of permutation pre-images leading to (3) in bad_1 or a permutation pre-image leading to (10) $\vee$ (11) in bad_2 given the extra queries). $\mathcal{V}$ has exact $4|f|$ syntactically different variables $\mathcal{P} := \{P_{g,0}, P_{g,1}, P_{g,2}, P_{g,3}\}_{g \in \mathcal{G}_{\text{and}}(f)}$. They fix the same number of the entries of permutation π in a real-world ω if and only if their literal values fixed by τ are also pairwise distinct. Every good transcript τ indeed fixes *exact one* such assignment of these values since:

- (18) already holds for τ as, for each $g \in \mathcal{G}_{\text{and}}(f)$ with $(a, b) := (\text{in}_0(g), \text{in}_1(g))$, the literal values of

$$\begin{aligned} P_{g,0} &:= \pi(X_a \oplus k_0^g) = U_0^g \oplus \sigma(X_a \oplus k_0^g), \\ P_{g,1} &:= \pi(X_b \oplus k_1^g) = U_1^g \oplus \sigma(X_b \oplus k_1^g) \end{aligned} \quad (21)$$

are fixed by the responses in $\mathcal{K}_3^R$ given the extra queries and will be pairwise distinct due to the impossible (4) from $\neg \text{bad}_1$.

- For each $g \in \mathcal{G}_{\text{and}}(f)$ with $(a, b) := (\text{in}_0(g), \text{in}_1(g))$, $\{P_{g,2}, P_{g,3}\}_{g \in \mathcal{G}_{\text{and}}(f)}$ are literally assigned the following values fixed by τ according to $\mathcal{V}$ and (21):

$$\begin{aligned} P_{g,2} &:= \pi(X_a \oplus \Delta \oplus k_0^g) = \sigma(\Delta) \oplus (s_b \oplus x_b)\Delta \oplus G_0^g \oplus U_0^g \oplus \sigma(X_a \oplus k_0^g), \\ P_{g,3} &:= \pi(X_b \oplus \Delta \oplus k_1^g) = \sigma(\Delta) \oplus x_a\Delta \oplus X_a \oplus G_1^g \oplus U_1^g \oplus \sigma(X_b \oplus k_1^g). \end{aligned} \quad (22)$$

The literal values of $\{P_{g,2}, P_{g,3}\}_{g \in \mathcal{G}_{\text{and}}(f)}$ are pairwise distinct due to the impossible (5) $\vee$ (6) $\vee$ (7) $\vee$ (8) $\vee$ (9) from $\neg \text{bad}_1$.

- The goodness of τ also ensures that there do not exist

$$P' \in \{P_{g,0}, P_{g,1}\}_{g \in \mathcal{G}_{\text{and}}(f)}, P'_\Delta \in \{P_{g,2}, P_{g,3}\}_{g \in \mathcal{G}_{\text{and}}(f)}$$

such that $P' = P'_\Delta$. Otherwise, this equality and (22) ensure that there exist $(g, u) \in \mathcal{G}_{\text{and}}(f) \times \{0, 1\}$ and $g' \in \mathcal{G}_{\text{and}}(f)$ such that

$$\begin{aligned} \pi(X_w \oplus k_u^g) &= \pi(X_{a'} \oplus \Delta \oplus k_0^{g'}) = \sigma(\Delta) \oplus (s_{b'} \oplus x_{b'})\Delta \oplus G_0^{g'} \oplus U_0^{g'} \oplus \sigma(X_{a'} \oplus k_0^{g'}) \quad \vee \\ \pi(X_w \oplus k_u^g) &= \pi(X_{b'} \oplus \Delta \oplus k_1^{g'}) = \sigma(\Delta) \oplus x_{a'}\Delta \oplus X_{a'} \oplus G_1^{g'} \oplus U_1^{g'} \oplus \sigma(X_{b'} \oplus k_1^{g'}) \end{aligned} \quad (23)$$

where $w := \text{in}_u(g)$ and $(a', b') := (\text{in}_0(g'), \text{in}_1(g'))$. Recall that $\neg \text{bad}_3$ and the extra queries implies $(X_w \oplus k_u^g, \pi(X_w \oplus k_u^g)) \in \mathcal{K}_3$. As a result, (23) leads to contradiction with the impossible (12) $\vee$ (13) from $\neg \text{bad}_2$.

Putting these cases together, τ results in a value assignment of $\mathcal{P}$, which fixes exact $4|f|$ entries of real-world permutation π .

Conditioned on $\neg \text{bad}_1 \wedge \neg \text{bad}_3$ of good transcript τ , $2|f|$ extra queries are non-repeating and the number of non-repeating queries is $q + 2|f| = q_\Sigma$. So, $2|f|$ responses in $\cup_{\ell=1}^3 \mathcal{K}_\ell^R$ for the non-repeating extra queries are fixed by the values in $\mathcal{P}$ while the other $q_\Sigma - 2|f| = q$ responses are fixed by real-world π (conditioned on the values in $\mathcal{P}$). Using (i), (ii), (iii), and (iv) together with the “leftover” compatibility, we have in the real world that

$$\begin{aligned} &\Pr_{\omega \leftarrow \Omega_{\text{real}}} [\omega \vdash (\mathcal{K}_1^R, F, \mathcal{K}_2^R, \tilde{X}, d, \tilde{U}, \mathcal{K}_3^R) \mid \omega \vdash (f', \tilde{x}) \wedge \omega \vdash \Delta] \\ &= \frac{1}{(2^\lambda)^{|\mathcal{W}_{\text{in}}(f)|}} \cdot \frac{(2^\lambda - 4|f| - (q_\Sigma - 2|f|))!}{(2^\lambda)!}, \\ &\Rightarrow \Pr_{\omega \leftarrow \Omega_{\text{real}}} [\omega \in \text{comp}_{\text{real}}(\tau)] = \frac{1}{(2^\lambda)^{|\mathcal{W}_{\text{in}}(f)|}} \cdot \frac{1}{(2^\lambda)_{q+4|f|}} \cdot \frac{1}{2^{\lambda-1}}. \end{aligned}$$

In the ideal world, condition $\neg \text{bad}_3$ ensures that the responses for the $2|f|$ extra queries are fixed by the queries in the step 9 of $\text{SimF}^{\tilde{\pi}^{\pm 1}(\cdot)}$. So, each U_u^g is independently uniform according to the

randomness of $\tilde{\pi}^{\pm 1}$ and the pairwise distinct pre-images by $\neg \text{bad}_1$. From the garbled rows, the active input labels, and these U_u^g , the active internal and output labels (as well as decoding table d) are fixed in topology order. So, we have

$$\begin{aligned} & \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\omega \in \text{comp}_{\text{ideal}}(\tau)] \\ &= \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\omega \vdash (\mathcal{K}_1^R, \mathcal{K}_2^R, \mathcal{K}_3^R) \mid \omega \vdash (\hat{f}, \tilde{X}, d, \tilde{x}, \tilde{U}, \Delta)] \cdot \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\omega \vdash (\hat{f}, \tilde{X}, d, \tilde{x}, \tilde{U}, \Delta)] \\ &= \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\omega \vdash (\mathcal{K}_1^R, \mathcal{K}_2^R, \mathcal{K}_3^R) \mid \omega \vdash (\hat{f}, \tilde{X}, d, \tilde{x}, \tilde{U}, \Delta)] \cdot \frac{1}{2^{|\mathcal{W}_{\text{in}}(f)|\lambda + 2\lambda|f| + (\lambda-1) + 2\lambda|f|}}. \end{aligned}$$

From the condition $\neg \text{bad}_1 \wedge \neg \text{bad}_3$ and the $2|f|$ extra queries, there are exact $q + 2|f| = q_\Sigma$ non-repeating queries. Moreover, $2|f|$ responses in $\cup_{\ell=1}^3 \mathcal{K}_\ell^R$ for the non-repeating extra queries are fixed by the conditioned values but the other responses are fixed by $\tilde{\pi}^{\pm 1}$ for other $q_\Sigma - 2|f| = q$ non-repeating queries (which are responded with the rejection sampling in Figure 6). Let $\mathcal{N} \subseteq [1, q_\Sigma]$ denote the index set of these q queries in q_Σ non-repeating queries to $\tilde{\pi}^{\pm 1}(\cdot)$ such that $|\mathcal{N}| = q$. The rejection sampling in $\tilde{\pi}^{\pm 1}(\cdot)$ gives

$$\begin{aligned} & \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\omega \vdash (\mathcal{K}_1^R, \mathcal{K}_2^R, \mathcal{K}_3^R) \mid \omega \vdash (\hat{f}, \tilde{X}, d, \tilde{x}, \tilde{U}, \Delta)] \\ &= \prod_{i \in \mathcal{N}} \frac{1}{2^\lambda - |\mathcal{Q}_{i-1}|} = \prod_{i \in \mathcal{N}} \frac{1}{2^\lambda - (i-1)} \\ &\leq \frac{1}{2^\lambda - 2|f|} \times \frac{1}{2^\lambda - (2|f| + 1)} \times \cdots \times \frac{1}{2^\lambda - (q_\Sigma - 1)} \leq \frac{1}{(2^\lambda - 2|f|)_q}, \\ &\Rightarrow \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\omega \in \text{comp}_{\text{ideal}}(\tau)] \leq \frac{1}{(2^\lambda)^{|\mathcal{W}_{\text{in}}(f)|}} \cdot \frac{1}{(2^\lambda - 2|f|)_q \cdot (2^\lambda)^{4|f|}} \cdot \frac{1}{2^{\lambda-1}}. \end{aligned}$$

So, we have $\varepsilon_2 = 0$ since, for every $|f| \geq 0$ and $q \geq 0$,

$$\begin{aligned} \frac{\Pr_{\omega \leftarrow \Omega_{\text{real}}} [\omega \in \text{comp}_{\text{real}}(\tau)]}{\Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\omega \in \text{comp}_{\text{ideal}}(\tau)]} &\geq \frac{(2^\lambda - 2|f|)_q \cdot (2^\lambda)^{4|f|}}{(2^\lambda)_{q+4|f|}} \\ &\geq \frac{(2^\lambda - 2|f|)_q \cdot (2^\lambda)_{2|f|} \cdot (2^\lambda)^{2|f|}}{(2^\lambda)_{q+4|f|}} \\ &= \frac{(2^\lambda)_{q+2|f|} \cdot (2^\lambda)^{2|f|}}{(2^\lambda)_{q+4|f|}} = \frac{(2^\lambda)^{2|f|}}{(2^\lambda - (q + 2|f|))_{2|f|}} \geq 1. \end{aligned}$$

Bounding ε_1 . First, consider $\text{bad}_1 \vee \text{bad}_3$. For each $i \in [2, 2|f|]$, let coll_i denote the event that there exist distinct $(g, u), (g', u') \in \text{"the first } i \text{ pairs of } \mathcal{G}_{\text{and}}(f) \times \{0, 1\}"$ such that $X_w \oplus k_u^g = X_{w'} \oplus k_{u'}^{g'}$, where $w := \text{in}_u(g)$ and $w' := \text{in}_{u'}(g')$, query_i denote the event that there exists $((\alpha, \beta), (g, u)) \in \cup_{\ell=1}^2 \mathcal{K}_\ell \times \text{"the first } i \text{ pairs of } \mathcal{G}_{\text{and}}(f) \times \{0, 1\}"$ such that $\alpha = X_w \oplus k_u^g$, where $w := \text{in}_u(g)$, and bFwd_i denote $\text{coll}_i \vee \text{query}_i$. Then, $\Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\text{bFwd}_{2|f|}] = \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [(\text{3}) \vee (\text{14}) \vee (\text{15})]$. We will prove the following bound using an induction:

$$\Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\text{bFwd}_i] \leq \frac{i(i-1) + 2i \cdot (q_1 + q_2)}{2^{\lambda+1}}.$$

In the base case ($i = 2$), X_w and $X_{w'}$ are two active circuit input labels so that coll_2 occurs with probability at most $1/2^\lambda$ due to the sampling in the step 2 of SimF (note that the probability is zero if $w = w'$). For query_2 , each of the two labels matches a fixed $\alpha \oplus k_u^g$ also with probability $1/2^\lambda$. Following from a union bound, the target bound holds for $i = 2$.

Assume that the probability bound holds for $i \in [2, 2|f| - 1]$ and consider the $i + 1$ case. From the law of total probability,

$$\begin{aligned}
& \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\text{bFwd}_{i+1}] \\
&= \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\text{bFwd}_{i+1} \mid \text{bFwd}_i] \cdot \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\text{bFwd}_i] + \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\text{bFwd}_{i+1} \mid \neg \text{bFwd}_i] \cdot \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\neg \text{bFwd}_i] \\
&= \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\text{bFwd}_i] + \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\text{bFwd}_{i+1} \mid \neg \text{bFwd}_i] \cdot \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\neg \text{bFwd}_i] \\
&\leq \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\text{bFwd}_i] + \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\text{bFwd}_{i+1} \mid \neg \text{bFwd}_i] \\
&\leq \frac{i(i-1) + 2i \cdot (q_1 + q_2)}{2^{\lambda+1}} + \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\text{bFwd}_{i+1} \mid \neg \text{bFwd}_i].
\end{aligned}$$

Let (g^*, u^*) denote the $(i + 1)$ -th pair of $\mathcal{G}_{\text{and}}(f) \times \{0, 1\}$ and $w^* := \text{in}_{u^*}(g^*)$. To incur coll_{i+1} conditioned on $\neg \text{bFwd}_i = \neg \text{coll}_i \wedge \neg \text{query}_i$, we have $X_{w^*} \oplus k_{u^*}^{g^*} = X_w \oplus k_u^g$, where $(g, u) \in$ “the first i pairs of $\mathcal{G}_{\text{and}}(f) \times \{0, 1\}$ ” and $w := \text{in}_u(g)$. Recall that each active label X_i is the XOR from (i) some active circuit input labels, and/or (ii) some active output labels of the *precedent* AND gates, i.e.,

$$X_i = \left(\bigoplus_{w \in \mathcal{I}_i \subseteq \mathcal{W}_{\text{in}}(f)} X_w \right) \oplus \left(\bigoplus_{w \in \mathcal{J}_i \subseteq \mathcal{W}_{\text{and}}(f)} X_w \right) = \bigoplus_{w \in \mathcal{I}_i \oplus \mathcal{J}_i} X_w \in \{0, 1\}^\lambda. \quad (24)$$

for $\mathcal{I}_i \oplus \mathcal{J}_i \neq \emptyset$. We use (24) to rewrite the equality to incur coll_{i+1} as

$$\bigoplus_{i \in (\mathcal{I}_w \oplus \mathcal{I}_{w^*}) \oplus (\mathcal{J}_w \oplus \mathcal{J}_{w^*})} X_i = k_u^g \oplus k_{u^*}^{g^*} \in \{0, 1\}^\lambda.$$

Here, the active circuit input labels in $\mathcal{I}_w \oplus \mathcal{I}_{w^*}$ are sampled at random in Sim. For the active labels in $\mathcal{J}_w \oplus \mathcal{J}_{w^*}$, they are masked by the responses sent from $\tilde{\pi}^{\pm 1}(\cdot)$ to Sim. Conditioned on $\neg \text{bFwd}_i$, these responses are taken from uniform $c_1, \dots, c_{n(\lambda)}$ in $\tilde{\pi}^{\pm 1}(\cdot)$ and pairwise independent since the queries (i.e., $X_w \oplus k_u^g$ for $(g, u) \in$ “the first i pairs of $\mathcal{G}_{\text{and}}(f) \times \{0, 1\}$ ” and $w := \text{in}_u(g)$) are pairwise distinct under this condition. The active labels in $\mathcal{J}_w \oplus \mathcal{J}_{w^*}$ are uniform, and the XOR on the left hand is uniform unless $\mathcal{I}_w = \mathcal{I}_{w^*}$ and $\mathcal{J}_w = \mathcal{J}_{w^*}$. That is, the equality to incur coll_{i+1} holds with probability at most $1/2^\lambda$ for some fixed (g, u) . The same argument and probability hold for the equality $\alpha = X_{w^*} \oplus k_{u^*}^{g^*}$ to incur query_{i+1} for some fixed (α, β) under $\neg \text{bFwd}_i$. Using a union bound,

$$\Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\text{bFwd}_{i+1} \mid \neg \text{bFwd}_i] \leq \frac{i + (q_1 + q_2)}{2^\lambda},$$

which concludes the induction for the $i + 1$ case. We have the following bound by setting $i = 2|f|$:

$$\Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\text{bFwd}_{2|f|}] \leq \frac{2|f|(2|f| - 1) + 4|f| \cdot (q_1 + q_2)}{2^{\lambda+1}}. \quad (25)$$

Consider $\text{bBwd} := (4) \vee (5) \vee (6) \vee (7) \vee (8) \vee (9) \vee (16) \vee (17)$ conditioned on $\text{bFwd}_{2|f|}$. In each of them, $U_u^g \oplus \sigma(X_w \oplus k_u^g)$ is the response for query $X_w \oplus k_u^g$ to $\tilde{\pi}^{\pm 1}(\cdot)$. Conditioned on $\neg \text{bFwd}_i$, these responses are taken from uniform $c_1, \dots, c_{n(\lambda)}$ in $\tilde{\pi}^{\pm 1}(\cdot)$ and pairwise independent as the queries are pairwise distinct under this condition. Therefore, each of them occurs with probability $1/2^\lambda$ for some fixed quantifier. From a union bound,

$$\Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\text{bBwd} \mid \neg \text{bFwd}_{2|f|}] \leq \frac{2|f|(2|f| - 1) + 2|f| \cdot (q_1 + q_2)}{2^\lambda}. \quad (26)$$

Using (25) and (26), we have

$$\begin{aligned}
& \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\text{bad}_1 \vee \text{bad}_3] \\
&= \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\text{bFwd}_{2|f|} \vee \text{bBwd}] \\
&= \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\text{bFwd}_{2|f|}] + \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\text{bBwd} \mid \neg \text{bFwd}_{2|f|}] \\
&\leq \frac{3|f|(2|f| - 1) + 4|f| \cdot (q_1 + q_2)}{2^\lambda}.
\end{aligned} \quad (27)$$

Then, consider bad_2 . From (10) (resp., (11)), we see $\Delta = \alpha \oplus X_a \oplus k_0^g$ (resp., $\Delta = \alpha \oplus X_b \oplus k_1^g$), occurring with probability $2^{-(\lambda-1)}$ due to the randomness of Δ . In (12), if $s_b \oplus x_b = 0$, linear orthomorphism σ ensures that $\Delta = \sigma^{-1}(\beta \oplus G_0^g \oplus U_0^g \oplus \sigma(X_a \oplus k_0^g))$, which occurs with probability $2^{-(\lambda-1)}$; if $s_b \oplus x_b = 1$, according to permutation $\sigma'(x) := \sigma(x) \oplus x$ well-defined from σ , it holds with the same probability that $\Delta = \sigma'^{-1}(\beta \oplus G_0^g \oplus U_0^g \oplus \sigma(X_a \oplus k_0^g))$. Similar result holds for (13). Taking a union bound over all pairs, we have

$$\Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\text{bad}_2] \leq \frac{4|f| \cdot (q_1 + q_2 + q_3)}{2^{\lambda-1}}. \quad (28)$$

We have a bound ε_1 from (27) and (28):

$$\begin{aligned} & \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [Y(\omega) \in \mathcal{T}_{\text{bad}}] \\ &= \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\text{bad}_1 \vee \text{bad}_2 \vee \text{bad}_3] \\ &\leq \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\text{bad}_1 \vee \text{bad}_3] + \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\text{bad}_2] \\ &\leq \frac{12qs + 30s^2 - 3s}{2^\lambda} = \varepsilon_1. \end{aligned}$$

This lemma follows from the H-coefficient bound and $(\varepsilon_1, \varepsilon_2)$. $\square$

References

- [AIK10] Benny Applebaum, Yuval Ishai, and Eyal Kushilevitz. From secrecy to soundness: Efficient verification via secure computation. In Samson Abramsky, Cyril Gavoille, Claude Kirchner, Friedhelm Meyer auf der Heide, and Paul G. Spirakis, editors, *ICALP 2010, Part I*, volume 6198 of *LNCS*, pages 152–163. Springer, Heidelberg, July 2010.
- [AIKW13] Benny Applebaum, Yuval Ishai, Eyal Kushilevitz, and Brent Waters. Encoding functions with constant online rate or how to compress garbled circuits keys. In Ran Canetti and Juan A. Garay, editors, *CRYPTO 2013, Part II*, volume 8043 of *LNCS*, pages 166–184, August 2013.
- [BCE⁺21] Beyza Bozdemir, Sébastien Canard, Orhan Ermiş, Helen Möllering, Melek Önen, and Thomas Schneider. Privacy-preserving density-based clustering. In Jiannong Cao, Man Ho Au, Zhiqiang Lin, and Moti Yung, editors, *ASIACCS 21*, pages 658–671. ACM Press, June 2021.
- [BHKO23] Cruz Barnum, David Heath, Vladimir Kolesnikov, and Rafail Ostrovsky. Adaptive garbled circuits and garbled RAM from non-programmable random oracles. *Cryptology ePrint Archive*, Report 2023/1527, 2023.
- [BHKR13] Mihir Bellare, Viet Tung Hoang, Sriram Keelveedhi, and Phillip Rogaway. Efficient garbling from a fixed-key blockcipher. In *2013 IEEE Symposium on Security and Privacy*, pages 478–492. IEEE Computer Society Press, May 2013.
- [BHR12a] Mihir Bellare, Viet Tung Hoang, and Phillip Rogaway. Adaptively secure garbling with applications to one-time programs and secure outsourcing. In Xiaoyun Wang and Kazuo Sako, editors, *ASIACRYPT 2012*, volume 7658 of *LNCS*, pages 134–153, December 2012.
- [BHR12b] Mihir Bellare, Viet Tung Hoang, and Phillip Rogaway. Foundations of garbled circuits. In Ting Yu, George Danezis, and Virgil D. Gligor, editors, *ACM CCS 2012*, pages 784–796. ACM Press, October 2012.
- [BMR90] Donald Beaver, Silvio Micali, and Phillip Rogaway. The round complexity of secure protocols (extended abstract). In *22nd ACM STOC*, pages 503–513. ACM Press, May 1990.

- [BPSW07] Justin Brickell, Donald E. Porter, Vitaly Shmatikov, and Emmett Witchel. Privacy-preserving remote diagnostics. In Peng Ning, Sabrina De Capitani di Vimercati, and Paul F. Syverson, editors, *ACM CCS 2007*, pages 498–507. ACM Press, October 2007.
- [BR93] Mihir Bellare and Phillip Rogaway. Random oracles are practical: A paradigm for designing efficient protocols. In Dorothy E. Denning, Raymond Pyle, Ravi Ganesan, Ravi S. Sandhu, and Victoria Ashby, editors, *ACM CCS 93*, pages 62–73. ACM Press, November 1993.
- [BSW03] Boaz Barak, Ronen Shaltiel, and Avi Wigderson. Computational analogues of entropy. In Sanjeev Arora, Klaus Jansen, José D. P. Rolim, and Amit Sahai, editors, *APPROX-RANDOM 2003*, volume 2764 of *LNCS*, pages 200–215. Springer, 2003.
- [Can00] Ran Canetti. Security and composition of multiparty cryptographic protocols. *Journal of Cryptology*, 13(1):143–202, January 2000.
- [CDN⁺23] Benoît Cogliati, Avijit Dutta, Mridul Nandi, Jacques Patarin, and Abishanka Saha. Proof of mirror theory for a wide range of $\xi_{\max}$. *LNCS*, pages 470–501, June 2023.
- [CGOS22] Nishanth Chandran, Divya Gupta, Sai Lakshmi Bhavana Obbattu, and Akash Shah. SIMC: ML inference secure against malicious clients at semi-honest cost. In Kevin R. B. Butler and Kurt Thomas, editors, *USENIX Security 2022*, pages 1361–1378. USENIX Association, August 2022.
- [CHW18] Ke Cheng, Yantian Hou, and Liangmin Wang. Secure similar sequence query on outsourced genomic data. In Jong Kim, Gail-Joon Ahn, Seungjoo Kim, Yongdae Kim, Javier López, and Taesoo Kim, editors, *ASIACCS 18*, pages 237–251. ACM Press, April 2018.
- [CJHK24] Hyunjun Cho, Jaeho Jeon, Jaehoon Heo, and Joo-Young Kim. APINT: A full-stack framework for acceleration of privacy-preserving inference of transformers based on garbled circuits. In Jinjun Xiong and Robert Wille, editors, *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design, ICCAD 2024, Newark Liberty International Airport Marriott, NJ, USA, October 27-31, 2024*, pages 209:1–209:9. ACM, 2024.
- [COT24] COTI. Garbled circuits on the blockchain for the very first time! <https://coti.substack.com/p/garbled-circuits-on-the-blockchain>, 2024.
- [CS14] Shan Chen and John P. Steinberger. Tight security bounds for key-alternating ciphers. In Phong Q. Nguyen and Elisabeth Oswald, editors, *EUROCRYPT 2014*, volume 8441 of *LNCS*, pages 327–350, May 2014.
- [DLMS14] Yuanxi Dai, Jooyoung Lee, Bart Mennink, and John P. Steinberger. The security of multiple encryption in the ideal cipher model. In Juan A. Garay and Rosario Gennaro, editors, *CRYPTO 2014, Part I*, volume 8616 of *LNCS*, pages 20–38, August 2014.
- [DNS22] Avijit Dutta, Mridul Nandi, and Abishanka Saha. Proof of mirror theory for $\xi_{\max} = 2$. *IEEE Trans. Inf. Theory*, 68(9):6218–6232, 2022.
- [DSZ15] Daniel Demmler, Thomas Schneider, and Michael Zohner. ABY - A framework for efficient mixed-protocol secure two-party computation. In *NDSS 2015*. The Internet Society, February 2015.
- [GGJ⁺23] Karthik Garimella, Zahra Ghodsi, Nandan Kumar Jha, Siddharth Garg, and Brandon Reagen. Characterizing and optimizing end-to-end systems for private inference. In Tor M. Aamodt, Natalie D. Enright Jerger, and Michael M. Swift, editors, *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and*

Operating Systems, Volume 3, ASPLOS 2023, Vancouver, BC, Canada, March 25-29, 2023, pages 89–104. ACM, 2023.

- [GGP10] Rosario Gennaro, Craig Gentry, and Bryan Parno. Non-interactive verifiable computing: Outsourcing computation to untrusted workers. In Tal Rabin, editor, *CRYPTO 2010*, volume 6223 of *LNCS*, pages 465–482, August 2010.
- [GKR08] Shafi Goldwasser, Yael Tauman Kalai, and Guy N. Rothblum. One-time programs. In David Wagner, editor, *CRYPTO 2008*, volume 5157 of *LNCS*, pages 39–56, August 2008.
- [GKW⁺20] Chun Guo, Jonathan Katz, Xiao Wang, Chenkai Weng, and Yu Yu. Better concrete security for half-gates garbling (in the multi-instance setting). In Daniele Micciancio and Thomas Ristenpart, editors, *CRYPTO 2020, Part II*, volume 12171 of *LNCS*, pages 793–822, August 2020.
- [GKWW20] Chun Guo, Jonathan Katz, Xiao Wang, and Yu Yu. Efficient and secure multiparty computation from fixed-key block ciphers. In *2020 IEEE Symposium on Security and Privacy*, pages 825–841. IEEE Computer Society Press, May 2020.
- [GLNP15] Shay Gueron, Yehuda Lindell, Ariel Nof, and Benny Pinkas. Fast garbling of circuits under standard assumptions. In Indrajit Ray, Ninghui Li, and Christopher Kruegel, editors, *ACM CCS 2015*, pages 567–578. ACM Press, October 2015.
- [GS18] Sanjam Garg and Akshayaram Srinivasan. Adaptively secure garbling with near optimal online complexity. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part II*, volume 10821 of *LNCS*, pages 535–565, April / May 2018.
- [GVW12] Sergey Gorbunov, Vinod Vaikuntanathan, and Hoeteck Wee. Functional encryption with bounded collusions via multi-party computation. In Reihaneh Safavi-Naini and Ran Canetti, editors, *CRYPTO 2012*, volume 7417 of *LNCS*, pages 162–179, August 2012.
- [HJO⁺16] Brett Hemenway, Zahra Jafargholi, Rafail Ostrovsky, Alessandra Scafuro, and Daniel Wichs. Adaptively secure garbled circuits from one-way functions. In Matthew Robshaw and Jonathan Katz, editors, *CRYPTO 2016, Part III*, volume 9816 of *LNCS*, pages 149–178, August 2016.
- [HLR07] Chun-Yuan Hsiao, Chi-Jen Lu, and Leonid Reyzin. Conditional computational entropy, or toward separating pseudoentropy from compressibility. In Moni Naor, editor, *EUROCRYPT 2007*, volume 4515 of *LNCS*, pages 169–186, May 2007.
- [HT16] Viet Tung Hoang and Stefano Tessaro. Key-alternating ciphers and key-length extension: Exact bounds and multi-user security. In Matthew Robshaw and Jonathan Katz, editors, *CRYPTO 2016, Part I*, volume 9814 of *LNCS*, pages 3–32, August 2016.
- [HW15] Pavel Hubacek and Daniel Wichs. On the communication complexity of secure function evaluation with long output. In Tim Roughgarden, editor, *ITCS 2015*, pages 163–172. ACM, January 2015.
- [HW23] Hai Huang and Luyao Wang. Efficient privacy-preserving face identification protocol. *IEEE Trans. Serv. Comput.*, 16(4):2632–2641, 2023.
- [IK00] Yuval Ishai and Eyal Kushilevitz. Randomizing polynomials: A new representation with applications to round-efficient secure computation. In *41st FOCS*, pages 294–304. IEEE Computer Society Press, November 2000.

- [JKK⁺17] Zahra Jafargholi, Chethan Kamath, Karen Klein, Ilan Komargodski, Krzysztof Pietrzak, and Daniel Wichs. Be adaptive, avoid overcommitting. In Jonathan Katz and Hovav Shacham, editors, *CRYPTO 2017, Part I*, volume 10401 of *LNCS*, pages 133–163, August 2017.
- [JKS⁺18] Kimmo Järvinen, Ágnes Kiss, Thomas Schneider, Oleksandr Tkachenko, and Zheng Yang. Faster privacy-preserving location proximity schemes. In Jan Camenisch and Panos Papadimitratos, editors, *CANS 18*, volume 11124 of *LNCS*, pages 3–22, September / October 2018.
- [JLL⁺19] Kimmo Järvinen, Helena Leppäkoski, Elena Simona Lohan, Philipp Richter, Thomas Schneider, Oleksandr Tkachenko, and Zheng Yang. PILOT: Practical privacy-preserving indoor localization using Outsourcing. pages 448–463, 2019.
- [JO20] Zahra Jafargholi and Sabine Oechsner. Adaptive security of practical garbling schemes. In Karthikeyan Bhargavan, Elisabeth Oswald, and Manoj Prabhakaran, editors, *INDOCRYPT 2020*, volume 12578 of *LNCS*, pages 741–762, December 2020.
- [JVC18] Chiraag Juvekar, Vinod Vaikuntanathan, and Anantha Chandrakasan. GAZELLE: A low latency framework for secure neural network inference. In William Enck and Adrienne Porter Felt, editors, *USENIX Security 2018*, pages 1651–1669. USENIX Association, August 2018.
- [JW16] Zahra Jafargholi and Daniel Wichs. Adaptive security of Yao’s garbled circuits. In Martin Hirt and Adam D. Smith, editors, *TCC 2016-B, Part I*, volume 9985 of *LNCS*, pages 433–458, October / November 2016.
- [Kel20] Marcel Keller. MP-SPDZ: A versatile framework for multi-party computation. In Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna, editors, *ACM CCS 2020*, pages 1575–1590. ACM Press, November 2020.
- [KLS⁺17] Ágnes Kiss, Jian Liu, Thomas Schneider, N. Asokan, and Benny Pinkas. Private set intersection for unequal set sizes with mobile applications. *PoPETs*, 2017(4):177–197, October 2017.
- [KMR14] Vladimir Kolesnikov, Payman Mohassel, and Mike Rosulek. FleXOR: Flexible garbling for XOR gates that beats free-XOR. In Juan A. Garay and Rosario Gennaro, editors, *CRYPTO 2014, Part II*, volume 8617 of *LNCS*, pages 440–457, August 2014.
- [KRS⁺19] Daniel Kales, Christian Rechberger, Thomas Schneider, Matthias Senker, and Christian Weinert. Mobile private contact discovery at scale. In Nadia Heninger and Patrick Traynor, editors, *USENIX Security 2019*, pages 1447–1464. USENIX Association, August 2019.
- [KS08] Vladimir Kolesnikov and Thomas Schneider. Improved garbled circuit: Free XOR gates and applications. In Luca Aceto, Ivan Damgård, Leslie Ann Goldberg, Magnús M. Halldórsson, Anna Ingólfssdóttir, and Igor Walukiewicz, editors, *ICALP 2008, Part II*, volume 5126 of *LNCS*, pages 486–498. Springer, Heidelberg, July 2008.
- [LEFS25] Jan Lauinger, Jens Ernstberger, Andreas Finkenzeller, and Sebastian Steinhorst. Janus: Fast privacy-preserving data provenance for TLS. *Proc. Priv. Enhancing Technol.*, 2025(1):511–530, 2025.
- [LP09] Yehuda Lindell and Benny Pinkas. A proof of security of Yao’s protocol for two-party computation. *Journal of Cryptology*, 22(2):161–188, April 2009.
- [LR14] Yehuda Lindell and Ben Riva. Cut-and-choose Yao-based secure computation in the online/offline and batch settings. In Juan A. Garay and Rosario Gennaro, editors, *CRYPTO 2014, Part II*, volume 8617 of *LNCS*, pages 476–494, August 2014.

- [LR15] Yehuda Lindell and Ben Riva. Blazing fast 2PC in the offline/online setting with security for malicious adversaries. In Indrajit Ray, Ninghui Li, and Christopher Kruegel, editors, *ACM CCS 2015*, pages 579–590. ACM Press, October 2015.
- [LWN⁺15] Chang Liu, Xiao Shaun Wang, Kartik Nayak, Yan Huang, and Elaine Shi. OblivM: A programming framework for secure computation. In *2015 IEEE Symposium on Security and Privacy*, pages 359–376. IEEE Computer Society Press, May 2015.
- [MLS⁺20] Pratyush Mishra, Ryan Lehmkuhl, Akshayaram Srinivasan, Wenting Zheng, and Raluca Ada Popa. Delphi: A cryptographic inference service for neural networks. In Srdjan Capkun and Franziska Roesner, editors, *USENIX Security 2020*, pages 2505–2522. USENIX Association, August 2020.
- [MR17] Payman Mohassel and Mike Rosulek. Non-interactive secure 2PC in the offline/online and batch settings. In Jean-Sébastien Coron and Jesper Buus Nielsen, editors, *EUROCRYPT 2017, Part III*, volume 10212 of *LNCS*, pages 425–455, April / May 2017.
- [Nie02] Jesper Buus Nielsen. Separating random oracle proofs from complexity theoretic proofs: The non-committing encryption case. In Moti Yung, editor, *CRYPTO 2002*, volume 2442 of *LNCS*, pages 111–126, August 2002.
- [NIW⁺13] Valeria Nikolaenko, Stratis Ioannidis, Udi Weinsberg, Marc Joye, Nina Taft, and Dan Boneh. Privacy-preserving matrix factorization. In Ahmad-Reza Sadeghi, Virgil D. Gligor, and Moti Yung, editors, *ACM CCS 2013*, pages 801–812. ACM Press, November 2013.
- [NPS99] Moni Naor, Benny Pinkas, and Reuban Sumner. Privacy preserving auctions and mechanism design. In Stuart I. Feldman and Michael P. Wellman, editors, *Proceedings of the First ACM Conference on Electronic Commerce (EC’99)*, pages 129–139. ACM, 1999.
- [NWT⁺13] Valeria Nikolaenko, Udi Weinsberg, Stratis Ioannidis, Marc Joye, Dan Boneh, and Nina Taft. Privacy-preserving ridge regression on hundreds of millions of records. In *2013 IEEE Symposium on Security and Privacy*, pages 334–348. IEEE Computer Society Press, May 2013.
- [Pat09] Jacques Patarin. The “coefficients H” technique (invited talk). In Roberto Maria Avanzi, Liam Keliher, and Francesco Sica, editors, *SAC 2008*, volume 5381 of *LNCS*, pages 328–345, August 2009.
- [PSSW09] Benny Pinkas, Thomas Schneider, Nigel P. Smart, and Stephen C. Williams. Secure two-party computation is practical. In Mitsuru Matsui, editor, *ASIACRYPT 2009*, volume 5912 of *LNCS*, pages 250–267, December 2009.
- [PSSY21] Arpita Patra, Thomas Schneider, Ajith Suresh, and Hossein Yalame. ABY2.0: Improved mixed-protocol secure two-party computation. In Michael Bailey and Rachel Greenstadt, editors, *USENIX Security 2021*, pages 2165–2182. USENIX Association, August 2021.
- [PSSZ15] Benny Pinkas, Thomas Schneider, Gil Segev, and Michael Zohner. Phasing: Private set intersection using permutation-based hashing. In Jaeyeon Jung and Thorsten Holz, editors, *USENIX Security 2015*, pages 515–530. USENIX Association, August 2015.
- [RR16] Peter Rindal and Mike Rosulek. Faster malicious 2-party secure computation with on-line/offline dual execution. In Thorsten Holz and Stefan Savage, editors, *USENIX Security 2016*, pages 297–314. USENIX Association, August 2016.
- [RR21] Mike Rosulek and Lawrence Roy. Three halves make a whole? Beating the half-gates lower bound for garbled circuits. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part I*, volume 12825 of *LNCS*, pages 94–124, Virtual Event, August 2021.

- [RS08] Phillip Rogaway and John P. Steinberger. Constructing cryptographic hash functions from fixed-key blockciphers. In David Wagner, editor, *CRYPTO 2008*, volume 5157 of *LNCS*, pages 433–450, August 2008.
- [RSC⁺19] M. Sadegh Riazi, Mohammad Samragh, Hao Chen, Kim Laine, Kristin E. Lauter, and Farinaz Koushanfar. XONN: XNOR-based oblivious deep neural network inference. In Nadia Heninger and Patrick Traynor, editors, *USENIX Security 2019*, pages 1501–1518. USENIX Association, August 2019.
- [Sho04] Victor Shoup. Sequences of games: a tool for taming complexity in security proofs. Cryptology ePrint Archive, Report 2004/332, 2004.
- [SHS⁺15] Ebrahim M. Songhori, Siam U. Hussain, Ahmad-Reza Sadeghi, Thomas Schneider, and Farinaz Koushanfar. TinyGarble: Highly compressed and scalable sequential garbled circuits. In *2015 IEEE Symposium on Security and Privacy*, pages 411–428. IEEE Computer Society Press, May 2015.
- [SS10] Amit Sahai and Hakan Seyalioglu. Worry-free encryption: functional encryption with public keys. In Ehab Al-Shaer, Angelos D. Keromytis, and Vitaly Shmatikov, editors, *ACM CCS 2010*, pages 463–472. ACM Press, October 2010.
- [ST19] Thomas Schneider and Oleksandr Tkachenko. EPISODE: Efficient privacy-PreservIng similar sequence queries on outsourced genomic DatabasEs. In Steven D. Galbraith, Giovanni Russello, Willy Susilo, Dieter Gollmann, Engin Kirda, and Zhenkai Liang, editors, *ASIACCS 19*, pages 315–327. ACM Press, July 2019.
- [TLS] TLSNotary. tln-docs. <https://docs.tlsnotary.org/mpc/encryption.html>.
- [TWYW18] Xiaoting Tang, Cong Wang, Xingliang Yuan, and Qian Wang. Non-interactive privacy-preserving truth discovery in crowd sensing applications. In *2018 IEEE Conference on Computer Communications, INFOCOM 2018, Honolulu, HI, USA, April 16-19, 2018*, pages 1988–1996. IEEE, 2018.
- [vdBNS22] Christopher van der Beets, Raine Nieminen, and Thomas Schneider. FAPRIL: towards faster privacy-preserving fingerprint-based localization. In Sabrina De Capitani di Vimercati and Pierangela Samarati, editors, *Proceedings of the 19th International Conference on Security and Cryptography, SECURITY 2022, Lisbon, Portugal, July 11-13, 2022*, pages 108–120. SCITEPRESS, 2022.
- [WMK16] Xiao Wang, Alex J. Malozemoff, and Jonathan Katz. EMP-toolkit: Efficient MultiParty computation toolkit. <https://github.com/emp-toolkit>, 2016.
- [XLZ⁺22] Guowen Xu, Hongwei Li, Yun Zhang, Shengmin Xu, Jianting Ning, and Robert H. Deng. Privacy-preserving federated deep learning with irregular users. *IEEE Trans. Dependable Secur. Comput.*, 19(2):1364–1381, 2022.
- [XYWY24] Xiang Xie, Kang Yang, Xiao Wang, and Yu Yu. Lightweight authentication of web data via garble-then-prove. In *USENIX Security 2024*. USENIX Association, August 2024.
- [Yao82] Andrew Chi-Chih Yao. Theory and applications of trapdoor functions (extended abstract). In *23rd FOCS*, pages 80–91. IEEE Computer Society Press, November 1982.
- [Yao86] Andrew Chi-Chih Yao. How to generate and exchange secrets (extended abstract). In *27th FOCS*, pages 162–167. IEEE Computer Society Press, October 1986.
- [ZE15] Samee Zahur and David Evans. Obliv-C: A language for extensible data-oblivious computation. Cryptology ePrint Archive, Report 2015/1153, 2015.

- [ZRE15] Samee Zahur, Mike Rosulek, and David Evans. Two halves make a whole - reducing data transfer in garbled circuits using half gates. In Elisabeth Oswald and Marc Fischlin, editors, *EUROCRYPT 2015, Part II*, volume 9057 of *LNCS*, pages 220–250, April 2015.

A More Preliminaries

Let bold lowercase letters (e.g., $\mathbf{a}$) denote column vectors and bold uppercase letters (e.g., $\mathbf{A}$) denote matrices. Let $\mathbf{I}_n$ denote the n -by- n identity matrix. Let $\text{Half}_0(\mathbf{a}) \in \mathbb{F}_{2^n}$ (resp., $\text{Half}_1(\mathbf{a}) \in \mathbb{F}_{2^n}$) denote the lower (resp., upper) half of vector $\mathbf{a} \in \mathbb{F}_{2^{2n}}$. We can also define $\text{lsb}(\mathbf{a}) := \text{lsb}(\text{Half}_0(\mathbf{a}))$ for vector $\mathbf{a} \in \mathbb{F}_{2^{2n}}$. Let $\otimes$ denote the Kronecker product of matrices. Let $\mathbf{A}^+$ be the left inverse of matrix $\mathbf{A}$. We will use $\mathbb{F}_{2^n}$, $\mathbb{F}_2^n$, and $\{0, 1\}^n$ interchangeably. A generalized definition of linear orthomorphism is as follows:

Definition 2 (Linear orthomorphism). *A permutation $\sigma : \mathbb{G} \rightarrow \mathbb{G}$ over an additive Abelian group $\mathbb{G}$ is a linear orthomorphism for a function family $\mathcal{L}$ of some linear functions from $\mathbb{G}$ to $\mathbb{G}$, if (i) $\sigma(x + y) = \sigma(x) + \sigma(y)$ for any $x, y \in \mathbb{G}$, (ii) $\sigma'(x) := \sigma(x) - L(x)$ is also a permutation for every $L \in \mathcal{L}$, and (iii) σ, σ' and their inverses are efficiently computable. We will simply call σ a linear orthomorphism if $\mathcal{L}$ contains only the identity function.*

For half-gates [ZRE15], we have the two instantiations of σ in Section 4.1. For three-halves [RR21], we require a linear orthomorphism $\sigma : \mathbb{F}_{2^{\lambda/2+1}}^2 \rightarrow \mathbb{F}_{2^{\lambda/2+1}}^2$ for the function family

$$\mathcal{L} = \{L_{\xi_1, \xi_2, \xi_3, \xi_4} : \mathbb{F}_{2^{\lambda/2+1}}^2 \rightarrow \mathbb{F}_{2^{\lambda/2+1}}^2\}_{\xi_1, \xi_2, \xi_3, \xi_4 \in \mathbb{F}_2}, \quad L_{\xi_1, \xi_2, \xi_3, \xi_4} \left(\begin{bmatrix} x_L \\ x_R \end{bmatrix} \right) = \begin{bmatrix} \xi_1 x_L \oplus \xi_2 x_R \\ \xi_3 x_L \oplus \xi_4 x_R \end{bmatrix}.$$

The following linear orthomorphism σ is used by three-halves:

$$\sigma \left(\begin{bmatrix} x_L \\ x_R \end{bmatrix} \right) = \begin{bmatrix} c \cdot x_L \\ c \cdot x_R \end{bmatrix}$$

where $c \in \mathbb{F}_{2^{\lambda/2+1}} \setminus \mathbb{F}_{2^2}$ is fixed and the multiplication is in $\mathbb{F}_{2^{\lambda/2+1}}$.

Auxiliary circuit notation for the proofs in appendices. For a circuit f with fan-in two and fan-out one, we also define:

- $\mathcal{Y}_1(f), \dots, \mathcal{Y}_m(f) \subseteq \mathcal{W}_{\text{out}}(f)$: $m \geq 1$ maximal non-empty partitions of circuit output wires such that, for every $i \in [1, m]$ and distinct $w, w' \in \mathcal{Y}_i(f)$, wire w and w' are from the same XOR combination of (i) the circuit input wires, and/or (ii) the output wires of *last-level* AND gates (i.e., the AND gates without any subsequent AND gate). We have $\sum_{i=1}^m |\mathcal{Y}_i(f)| = |\mathcal{W}_{\text{out}}(f)| \geq m$.
- $\mathcal{Z}(f) \subseteq \mathcal{W}_{\text{and}}(f)$: The set of the output wires of the last-level AND gates.
- $\mathcal{X}(f) := \mathcal{W}_{\text{in}}(f) \cup \mathcal{Z}(f)$.
- For each $i \in [1, m]$, let $\mathcal{X}_i(f) \subseteq \mathcal{X}(f)$ be the set collecting the wires used to XOR-combine the circuit output wires in $\mathcal{Y}_i(f)$, and $y_i(f)$ be the first (i.e., representative) wire in $\mathcal{Y}_i(f)$.

For three-halves, we also define:

- For an AND gate $g \in \mathcal{G}_{\text{and}}(f)$, let $\text{in}_2(g)$ be the *global* alias of all wires that syntactically use a pair of labels $(W_a \oplus W_b, W_a \oplus W_b \oplus \Delta)$ with $a := \text{in}_0(g)$ and $b := \text{in}_1(g)$ in free-XOR. If no such a wire exists, we define it artificially. See [RR21, Section 6.2].
- $\mathcal{W}_{\cup}(f) := \cup_{g \in \mathcal{G}_{\text{and}}(f)} \{\text{in}_0(g), \text{in}_1(g), \text{in}_2(g)\}$.
- Let $n_i(f) \geq 0$ denote the number of times the wire i is used for the input to the AND gates in f .
- $N := \sum_{i \in \mathcal{W}_{\cup}(f)} \lceil n_i(f)/2 \rceil$.
- $M := |\{i \in \mathcal{W}_{\cup}(f) \mid n_i(f) \text{ is odd}\}|$.

B Proof of Lemma 1

Lemma 1 ([DLMS14]). *We use the notation defined in Section 4.4. For every auxiliary input $z \in \{0, 1\}^*$, every computationally unbounded adversary $\mathcal{A}$, and every attainable transcript $\tau \in \mathcal{T}_{\mathcal{A}(z)}$, we have*

$$\begin{aligned} \Pr_{\omega \leftarrow \Omega_{\text{real}}} [X(\omega) = \tau] &= \Pr_{\omega \leftarrow \Omega_{\text{real}}} [\omega \in \text{comp}_{\text{real}}(\tau)], \\ \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [Y(\omega) = \tau] &= \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\omega \in \text{comp}_{\text{ideal}}(\tau)] > 0. \end{aligned}$$

Proof. Let $\mathcal{M}^O \rightarrow x$ be the predicate that is true if and only if the interaction between $\mathcal{M}$ and oracle O produces a transcript with literal value x . For every computationally unbounded non-uniform adversary $\mathcal{A} = \mathcal{A}(z)$ and every attainable transcript $\tau \in \mathcal{T}_{\mathcal{A}}$, we have by definition

$$\Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [Y(\omega) = \tau] = \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\omega \in \{\omega \in \Omega_{\text{ideal}} \mid \mathcal{A}^\omega \rightarrow \tau\}] > 0$$

since $\exists \omega' \in \Omega_{\text{ideal}} : \mathcal{A}^{\omega'} \rightarrow \tau$ is true if and only if τ is attainable. We also point out that the reverse implication also holds as the non-zero probability implies a non-empty set $\{\omega \in \Omega_{\text{ideal}} \mid \mathcal{A}^\omega \rightarrow \tau\}$, leading to the existence of $\omega' \in \Omega_{\text{ideal}}$.

Moreover, we have by definition

$$\begin{aligned} \Pr_{\omega \leftarrow \Omega_{\text{real}}} [X(\omega) = \tau] &= \Pr_{\omega \leftarrow \Omega_{\text{real}}} [\omega \in \{\omega \in \Omega_{\text{real}} \mid \mathcal{A}^\omega \rightarrow \tau\}], \\ \Pr_{\omega \leftarrow \Omega_{\text{real}}} [\omega \in \text{comp}_{\text{real}}(\tau)] &= \Pr_{\omega \leftarrow \Omega_{\text{real}}} [\omega \in \{\omega \in \Omega_{\text{real}} \mid \exists \mathcal{A}' : \mathcal{A}'^\omega \rightarrow \tau\}] \\ &= \Pr_{\omega \leftarrow \Omega_{\text{real}}} [\omega \in \{\omega \in \Omega_{\text{real}} \mid (\exists \mathcal{A}' : \mathcal{A}'^\omega \rightarrow \tau) \wedge \tau \in \mathcal{T}_{\mathcal{A}}\}] \\ &= \Pr_{\omega \leftarrow \Omega_{\text{real}}} [\omega \in \{\omega \in \Omega_{\text{real}} \mid (\exists \mathcal{A}' : \mathcal{A}'^\omega \rightarrow \tau) \wedge (\exists \omega' \in \Omega_{\text{ideal}} : \mathcal{A}^{\omega'} \rightarrow \tau)\}] \end{aligned}$$

where $\mathcal{A}$, $\mathcal{A}'$, ω , and ω' are deterministic (c.f. Section 4.4).

For every computationally unbounded non-uniform $\mathcal{A}$, every attainable $\tau \in \mathcal{T}_{\mathcal{A}}$, and every $\omega \in \Omega_{\text{real}}$, the predicate equivalence $\mathcal{A}^\omega \rightarrow \tau \Leftrightarrow (\exists \mathcal{A}' : \mathcal{A}'^\omega \rightarrow \tau) \wedge (\exists \omega' \in \Omega_{\text{ideal}} : \mathcal{A}^{\omega'} \rightarrow \tau)$ holds. The forward implication is obvious by using $\mathcal{A}' := \mathcal{A}$, where $\exists \omega' \in \Omega_{\text{ideal}} : \mathcal{A}^{\omega'} \rightarrow \tau$ must hold for the attainable $\tau \in \mathcal{T}_{\mathcal{A}}$. For the backward implication, we can use in the following induction that τ is an *ordered* list of query-response pairs $\{(q_1, r_1), \dots\}$.

In the base case, $\mathcal{A}$ (resp., $\mathcal{A}'$) will send query q_1 to and receive response r_1 from ω' (resp., ω) according to the same fixed τ . Since q_1 (resp., r_1) is the first query (resp., response) of $\mathcal{A}$ (resp., ω) so that there is no previous list on which it can depend, $\mathcal{A}$ will also send q_1 and receive r_1 if it is given oracle ω . We assume that the interaction between $\mathcal{A}$ and ω has produced the same list of query-response pairs $\tau_{i-1} = \{(q_1, r_1), \dots, (q_{i-1}, r_{i-1})\}$, which is a prefix of τ . It is known from the same fixed τ that, conditioned on the same previous list τ_{i-1} , $\mathcal{A}$ (resp., $\mathcal{A}'$) will send query q_i to and receive response r_i from ω' (resp., ω). As a result, if $\mathcal{A}$ accesses ω for the i -th query conditioned on τ_{i-1} , $\mathcal{A}$ will also send q_i as query and receive r_i , which only depends on τ_{i-1} in oracle ω . The above induction concludes that the two predicates are equivalent for every non-uniform $\mathcal{A}$, attainable $\tau \in \mathcal{T}_{\mathcal{A}}$, and $\omega \in \Omega_{\text{real}}$. This equivalence leads to the following two equal probabilities:

$$\Pr_{\omega \leftarrow \Omega_{\text{real}}} [X(\omega) = \tau] = \Pr_{\omega \leftarrow \Omega_{\text{real}}} [\omega \in \text{comp}_{\text{real}}(\tau)].$$

We can prove the ideal-world result using the same induction. □

C Adaptive Security of Half-Gates in pRPM

We prove that the original implementation of half-gates [ZRE15], which is identically defined as Figure 3 but sends decoding information d as a part of garbled circuit f , is adaptively secure in the pRPM and does not suffer from the lower bound in the npRPM. We refer to Appendix A for the additional notation.

SimF(f):

- 1: $F \leftarrow (\{0, 1\}^{2\lambda})^{|f|}$, $d \leftarrow \mathbb{F}_2^{|\mathcal{W}_{\text{out}}(f)|}$ such that
 - (i) For each $i \in [1, m]$ and each wire $j \in \mathcal{Y}_i(f)$, $d_j = d_{y_i(f)}$.
 - (ii) For each $(\mu_1, \dots, \mu_m) \in \mathbb{F}_2^m$ s.t. $\bigoplus_{i \in [1, m], \mu_i = 1} \mathcal{X}_i(f) = \emptyset$, $\bigoplus_{i \in [1, m]} \mu_i \cdot d_{y_i(f)} = 0$.
- 2: **return** $\hat{f} := (f' := f, F, d)$, $\text{st}_{\text{sim}} := (\text{st}_{\text{sim}}, f, F, d)$

SimIn($f(x)$):

- 1: $\{X_i\}_{i \in \mathcal{X}(f)} \leftarrow (\{0, 1\}^\lambda)^{|\mathcal{X}(f)|}$ such that, for each $j \in [1, m]$, $\text{lsb}(\bigoplus_{i \in \mathcal{X}_j(f)} X_i) = d_{y_j(f)} \oplus f(x)_{y_j(f)}$.
- 2: **for** $g \in \mathcal{G}(f)$ **in order do**
- 3: $(a, b, c) := (\text{in}_0(g), \text{in}_1(g), \text{out}(g))$
- 4: **if** $\text{type}(g) = \text{XOR}$ **do** $X_c := X_a \oplus X_b$
- 5: **else if** $\text{type}(g) = \text{AND}$ **do**
- 6: $k_0^g := 2 \cdot g - 1$, $k_1^g := 2 \cdot g$
- 7: $s_a := \text{lsb}(X_a)$, $s_b := \text{lsb}(X_b)$
- 8: **if** $c \in \mathcal{Z}(f)$ **do**
- 9: $U_1^g \leftarrow \{0, 1\}^\lambda$
- 10: $U_0^g := X_c \oplus U_1^g \oplus s_a G_0^g \oplus s_b (G_1^g \oplus X_a)$
- 11: **else if** $c \notin \mathcal{Z}(f)$ **do**
- 12: $U_0^g, U_1^g \leftarrow \{0, 1\}^\lambda$
- 13: $X_c := U_0^g \oplus U_1^g \oplus s_a G_0^g \oplus s_b (G_1^g \oplus X_a)$
- 14: (Programming) Add two pairs $(X_a \oplus k_0^g, U_0^g \oplus \sigma(X_a \oplus k_0^g))$ and $(X_b \oplus k_1^g, U_1^g \oplus \sigma(X_b \oplus k_1^g))$ to the list $\mathcal{Q}$ kept in st_{sim} , if they cause no pre-image collision or image collision in $\mathcal{Q}$, to ensure

$$\underbrace{\pi(X_a \oplus k_0^g) \oplus \sigma(X_a \oplus k_0^g)}_{H(X_a, k_0^g)} = U_0^g, \quad \underbrace{\pi(X_b \oplus k_1^g) \oplus \sigma(X_b \oplus k_1^g)}_{H(X_b, k_1^g)} = U_1^g.$$

- 15: **return** $\hat{x} := \{X_i\}_{i \in \mathcal{W}_{\text{in}}(f)}$, $\text{st}_{\text{sim}} := (\text{st}_{\text{sim}}, \tilde{X}, \tilde{U})$ where st_{sim} on the right hand has an updated list $\mathcal{Q}$, $\tilde{X} := \{X_i\}_{i \in \mathcal{W}(f)}$, $\tilde{U} := \{U_0^g, U_1^g\}_{g \in \mathcal{G}_{\text{and}}(f)}$.

Figure 7: Our simulator for half-gates in the pRPM.

Theorem 2. Let $H(X, k) = \pi(X \oplus k) \oplus \sigma(X \oplus k)$ be a tweakable hash function where $X, k \in \{0, 1\}^\lambda$, $\pi \in \mathcal{S}_\lambda$ is random permutation, and $\sigma : \{0, 1\}^\lambda \rightarrow \{0, 1\}^\lambda$ is a linear orthomorphism. Then, half-gates ([ZRE15]) is a λ -garbling scheme with (q, s, ε) -adaptive security in the pRPM, where $\varepsilon = (8qs + 21s^2)/2^{\lambda-1}$.

Proof. The correctness has been proved in the original work [ZRE15]. We only need to consider the simulation.

Our simulator Sim consists of (SimF, SimIn) in Figure 7 and $\text{SimP}^{\pm 1}$, which emulates the random permutation and its inverse on-the-fly. More specifically, there is a list $\mathcal{Q}$ of query-response pairs in internal state st_{sim} . Upon receiving forward query α (resp., backward query β) from $\mathcal{A}_i$ to SimP (resp., SimP^{-1}), it reads $\mathcal{Q}$ from st_{sim} and checks whether $\exists(\alpha, \gamma) \in \mathcal{Q}$ (resp., $\exists(\gamma, \beta) \in \mathcal{Q}$). If true, it returns γ as response; otherwise it samples $\gamma \leftarrow \{s \in \{0, 1\}^\lambda \mid (\dots, s) \notin \mathcal{Q}\}$ (resp., $\gamma \leftarrow \{s \in \{0, 1\}^\lambda \mid (s, \dots) \notin \mathcal{Q}\}$), adds (α, γ) (resp., (γ, β)) to $\mathcal{Q}$, and returns γ as response. The programming is the step 14 of SimIn, where (α, β) is added to $\mathcal{Q}$ if and only if there is no pre-image collision with $(\alpha, \dots) \in \mathcal{Q}$ or image collision with $(\dots, \beta) \in \mathcal{Q}$.

To see that Sim is PPT, we note that the circuit-dependent notation can be efficiently computed by traversing the polynomial-size circuit f . Then, the crux is to show that the step 1 in both SimF and SimIn can be polynomial-time.

The runtime of the step 1 of SimF is dominated by the runtime of iterating through all qualified $(\mu_1, \dots, \mu_m) \in \mathbb{F}_2^m$. To find the qualified vectors, one can interpret each $\mathcal{X}_i(f)$ as a one-hot *non-zero* column vector in the space $\mathbb{F}_2^{|\mathcal{W}(f)|}$ and derive a $|\mathcal{W}(f)|$ -by- m matrix $\mathbf{E}$ from these column vectors. One can check that all qualified vectors fall in the kernel of $\mathbf{E}$. This kernel can be efficiently computed

from the Gaussian elimination on $\mathbf{E}$ and is a subspace spanned by $m - \text{rank}(\mathbf{E})$ basis vectors. So, the step 1 of SimF only needs to iterate through these basis vectors, and the other qualified vectors must satisfy the constraints as they are in the subspace. As a result, the step 1 of SimF runs in polynomial time due to the Gaussian elimination plus a linear-time pass to assign random or constrained values to $d_{y_i(f)}$'s according to the $m - \text{rank}(\mathbf{E})$ basis vectors.

The step 1 of SimIn only requires one linear-time pass to assign constrained or random values to the active labels so it runs in polynomial time. The linear constraint on the LSBs of these active labels has rank $\text{rank}(\mathbf{E})$ and is satisfiable for the $d_{y_i(f)}$'s assigned in SimF.

Then, we fix z and $\mathcal{A}$. We will use the H-coefficient technique (Section 4.4) with transcript padding to bound the advantage of distinguishing between the real world (i.e., the adaptive experiment that uses the half-gates scheme) and the ideal world (i.e., the adaptive experiment that uses simulator Sim). In this technique, we consider the computationally unbounded non-uniform adversary $\mathcal{A} = \mathcal{A}(z)$ and compute $\varepsilon_1, \varepsilon_2$ as follows.

Transcript padding. In either world, $\mathcal{A}$ will interact with an integrated oracle that acts as the two-round challenger in the adaptive experiment and provides interfaces $\pi^{*\pm 1} \in \{\pi^{\pm 1}, \text{SimP}^{\pm 1}\}$ for forward/backward permutation queries. $\mathcal{A}$ can learn $\pi^*(\alpha) = \beta$ if and only if it sent forward query α to π^* and received response β , or sent backward query β to π^{*-1} and received response α .

To compute $\varepsilon_1, \varepsilon_2$ more easily, we ask the oracle to send more messages to $\mathcal{A}$ and $\mathcal{A}$ to make extra queries (in addition to the supposed q queries) in both two worlds. More specifically,

- Upon receiving x from $\mathcal{A}$, the oracle sends $\tilde{X} := \{X_i\}_{i \in \mathcal{W}(f)}$ instead of $\hat{x} := \{X_i\}_{i \in \mathcal{W}_{\text{in}}(f)}$ to $\mathcal{A}$. In addition to the active input labels in $\hat{x}$, the former also gives the active internal and output labels. In the real world, the oracle can run $\text{HG.DecEval}^{\pi^{\pm 1}(\cdot)}$, which determines other active labels in $\tilde{X}$. In the ideal world, this $\tilde{X}$ can be directly output by SimIn.
- Along with $\tilde{X}$, the oracle sends $\tilde{x} := \{x_i\}_{i \in \mathcal{W}(f)}$ to $\mathcal{A}$, which denote the wire truth values in the evaluation of $f(x)$. Both two oracles “echo” these values, which are self-evident to $\mathcal{A}$, to explicitly include them in transcripts. In the experiment, the real-world oracle uses $x = \{x_i\}_{i \in \mathcal{W}_{\text{in}}(f)}$ in $\text{HG.Encode}^{\pi^{\pm 1}(\cdot)}$, but the ideal-world oracle can only use $f(x) = \{x_i\}_{i \in \mathcal{W}_{\text{out}}(f)}$ in SimIn.
- Along with $\tilde{X}$, the oracle sends $\tilde{U} := \{U_0^g, U_1^g\}_{g \in \mathcal{G}_{\text{and}}(f)}$ to $\mathcal{A}$. The real-world oracle computes $U_0^g := H(X_a, k_0^g)$ and $U_1^g := H(X_b, k_1^g)$ for each $g \in \mathcal{G}_{\text{and}}(f)$ with $(a, b) := (\text{in}_0(g), \text{in}_1(g))$. In the ideal world, this $\tilde{U}$ is output by SimIn and contains the “hash outputs” fixed by the random tape, which is specified by the oracle to run the programming therein.
- (Extra queries) Upon receiving $(\tilde{X}, \tilde{x}, \tilde{U})$ from the oracle, $\mathcal{A}$ will also make a forward permutation query $X_a \oplus k_0^g$ (resp., $X_b \oplus k_1^g$) for each $g \in \mathcal{G}_{\text{and}}(f)$ with $(a, b) := (\text{in}_0(g), \text{in}_1(g))$, if it has never learned $\pi^*(X_a \oplus k_0^g) = Y$ (resp., $\pi^*(X_b \oplus k_1^g) = Y$) for some Y in its interaction with $\pi^{*\pm 1} \in \{\pi^{\pm 1}, \text{SimP}^{\pm 1}\}$.
- At the end of the experiment (i.e., once all other transcripts are settled), the oracle sends Δ to $\mathcal{A}$. In the real world, the oracle gets this Δ from the output of $\text{HG.Garble}^{\pi^{\pm 1}(\cdot)}$. In the ideal world, Δ is dummy and sampled by the oracle at this time (note that Sim does not use Δ).

According to the two oracle constructions, real-world sample space

$$\Omega_{\text{real}} = \{0, 1\}^{\lambda-1} \times \{0, 1\}^{|\mathcal{W}_{\text{in}}(f)|\lambda} \times \mathcal{S}_\lambda,$$

and ideal-world sample space

$$\begin{aligned}\Omega_{\text{ideal}} = & (\{0, 1\}^{2\lambda})^{|f|} \times \{0, 1\}^{\text{rank}(\mathbf{E})} \\ & \times \{0, 1\}^{(|\mathcal{W}_{\text{in}}(f)| + |\mathcal{Z}(f)|)\lambda - \text{rank}(\mathbf{E})} \\ & \times (\{0, 1\}^\lambda)^{|\mathcal{Z}(f)|} \times (\{0, 1\}^{2\lambda})^{|f| - |\mathcal{Z}(f)|} \\ & \times \underbrace{\{0, 1\}^*}_{\text{random tape for}} \times \underbrace{\{0, 1\}^{\lambda-1}}_{\text{dummy } \Delta} \\ & \text{the sampling in } \text{SimP}^{\pm 1}(\cdot)\end{aligned}$$

Given the oracle constructions, a transcript in the original adaptive experiment will be padded with more literal values. Note that transcript padding will not lower the advantage of $\mathcal{A}$ since $\mathcal{A}$ can discard the padding values at will. With the padding, a transcript is of the form:

$$\tau = (\mathcal{K}_1, (f, \hat{f}), \mathcal{K}_2, (x, (\tilde{X}, \tilde{x}, \tilde{U})), \mathcal{K}_3, \Delta),$$

where $\mathcal{K}_1$, $\mathcal{K}_2$, and $\mathcal{K}_3$ are the ordered lists of query-response pairs seen in the interleaved interaction with permutation oracles. We do not explicitly consider query direction in these pairs. Given $\pi^{*\pm 1} \in \{\pi^{\pm 1}, \text{SimP}^{\pm 1}\}$, $\mathcal{A}$ is able to learn $\pi^*(\alpha) = \beta$ if and only if there exists $(\alpha, \beta) \in \cup_{\ell=1}^3 \mathcal{K}_\ell$.

Let $q_\ell := |\mathcal{K}_\ell|$ for every $\ell \in \{1, 2, 3\}$ and $q_\Sigma := \sum_{\ell=1}^3 q_\ell$. It follows from the extra queries that $q_\Sigma \leq q + 2|f|$. Without loss of generality, we assume that $\mathcal{A}$ only makes *non-repeating* queries, i.e., it never makes forward query α to π^* or backward query β to π^{*-1} for any learned permutation entry (α, β) .

Bad transcripts. A transcript $\tau \in \mathcal{T}_{\text{bad}}$ if and only if it incurs at least one of the following events:

- bad_1 . There exist distinct $(g, u), (g', u') \in \mathcal{G}_{\text{and}}(f) \times \{0, 1\}$ s.t.

$$X_w \oplus k_u^g = X_{w'} \oplus k_{u'}^{g'} \quad \vee \quad (3)$$

$$U_u^g \oplus \sigma(X_w \oplus k_u^g) = U_{u'}^{g'} \oplus \sigma(X_{w'} \oplus k_{u'}^{g'}) \quad (4)$$

where $w := \text{in}_u(g)$ and $w' := \text{in}_{u'}(g')$, or $g \in \mathcal{G}_{\text{and}}(f)$ s.t.

$$(s_b \oplus x_b)\Delta \oplus G_0^g \oplus U_0^g \oplus \sigma(X_a \oplus k_0^g) = x_a\Delta \oplus X_a \oplus G_1^g \oplus U_1^g \oplus \sigma(X_b \oplus k_1^g) \quad (5)$$

where $(a, b) := (\text{in}_0(g), \text{in}_1(g))$, or distinct $g, g' \in \mathcal{G}_{\text{and}}(f)$ s.t.

$$(s_b \oplus x_b)\Delta \oplus G_0^g \oplus U_0^g \oplus \sigma(X_a \oplus k_0^g) = (s_{b'} \oplus x_{b'})\Delta \oplus G_0^{g'} \oplus U_0^{g'} \oplus \sigma(X_{a'} \oplus k_0^{g'}) \quad \vee \quad (6)$$

$$x_a\Delta \oplus X_a \oplus G_1^g \oplus U_1^g \oplus \sigma(X_b \oplus k_1^g) = x_{a'}\Delta \oplus X_{a'} \oplus G_1^{g'} \oplus U_1^{g'} \oplus \sigma(X_{b'} \oplus k_1^{g'}) \quad \vee \quad (7)$$

$$(s_b \oplus x_b)\Delta \oplus G_0^g \oplus U_0^g \oplus \sigma(X_a \oplus k_0^g) = x_{a'}\Delta \oplus X_{a'} \oplus G_1^{g'} \oplus U_1^{g'} \oplus \sigma(X_{b'} \oplus k_1^{g'}) \quad \vee \quad (8)$$

$$x_a\Delta \oplus X_a \oplus G_1^g \oplus U_1^g \oplus \sigma(X_b \oplus k_1^g) = (s_{b'} \oplus x_{b'})\Delta \oplus G_0^{g'} \oplus U_0^{g'} \oplus \sigma(X_{a'} \oplus k_0^{g'}) \quad (9)$$

where $(a, b) := (\text{in}_0(g), \text{in}_1(g))$ and $(a', b') := (\text{in}_0(g'), \text{in}_1(g'))$.

In this case, $\mathcal{A}$ can check the consistency between the value of $G_u^g \oplus G_{u'}^{g'}$ and that of Δ at the end of experiment *without* further sending required queries, which are computed from Δ , to a random permutation or its inverse. In the real world, the consistency certainly holds. However, the ideal-world garbled rows and Δ are independently sampled, leading to the consistency only with negligible probability. So, $\mathcal{A}$ has non-negligible advantage to distinguish the two worlds and the statistical distance, as an upper bound, also blows up.

More specifically, the real world is as follows in this case. The pre-image collision (3) leads to the syntactically same XOR of two hash masks in $G_u^g, G_{u'}^{g'}$, which can be XORed to cancel all hash masks to check the consistency with Δ without further queries. Moreover, the image collision (4) also implies the pre-image collision (3) since π is permutation. The other equalities imply the image collision $\pi(X_w \oplus \Delta \oplus k_u^g) = \pi(X_{w'} \oplus \Delta \oplus k_{u'}^{g'})$ for some distinct tuple $(g, u), (g', u') \in \mathcal{G}_{\text{and}}(f) \times \{0, 1\}$, $w := \text{in}_u(g)$, and $w' := \text{in}_{u'}(g')$. Given permutation π , this collision implies the pre-image collision (3), which can be used to see the consistency. However, the above cancelling of hash masks will not give this consistency except with negligible probability in the ideal world.

- bad_2 . There exists $((\alpha, \beta), g) \in \cup_{\ell=1}^3 \mathcal{K}_\ell \times \mathcal{G}_{\text{and}}(f)$ such that

$$\alpha = X_a \oplus \Delta \oplus k_0^g \quad \vee \quad (10)$$

$$\alpha = X_b \oplus \Delta \oplus k_1^g \quad \vee \quad (11)$$

$$\beta = \sigma(\Delta) \oplus (s_b \oplus x_b)\Delta \oplus G_0^g \oplus U_0^g \oplus \sigma(X_a \oplus k_0^g) \quad \vee \quad (12)$$

$$\beta = \sigma(\Delta) \oplus x_a\Delta \oplus X_a \oplus G_1^g \oplus U_1^g \oplus \sigma(X_b \oplus k_1^g) \quad (13)$$

where $(a, b) := (\text{in}_0(g), \text{in}_1(g))$.

In this case, $\mathcal{A}$ essentially makes it to guess Δ before receiving this value. It allows $\mathcal{A}$ to distinguish the real world, where every G_i^g is consistent with Δ , and the ideal world with a dummy Δ . So, the statistical distance blows up.

- bad_3 . There exists $((\alpha, \beta), g) \in \cup_{\ell=1}^2 \mathcal{K}_\ell \times \mathcal{G}_{\text{and}}(f)$ such that

$$\alpha = X_a \oplus k_0^g \quad \vee \quad (14)$$

$$\alpha = X_b \oplus k_1^g \quad \vee \quad (15)$$

$$\beta = U_0^g \oplus \sigma(X_a \oplus k_0^g) \quad \vee \quad (16)$$

$$\beta = U_1^g \oplus \sigma(X_b \oplus k_1^g) \quad (17)$$

where $(a, b) := (\text{in}_0(g), \text{in}_1(g))$.

In this case, $\mathcal{A}$ can make forward/backward queries w.r.t. some active labels before receiving active input labels and computing other active ones.

This case is necessary to ensure successful programming in the ideal world as the values on the right hand should not be queried before the programming (otherwise it fails due to pre-image or image collision). If the programming fails in the ideal world, the two worlds can be distinguishable as the decoding consistency in the ideal world does not always hold as in the real world.

Bounding $1 - \varepsilon_2$. Without loss of generality, we can consider some fixed good transcript τ such that $\Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [Y(\omega) = \tau] > 0$. Using Lemma 1, we turn to analyze the sampled oracle's compatibility (Section 4.4) with such a transcript, instead of the interaction between $\mathcal{A}$ and the sampled oracle.

Note that there is a computationally unbounded non-uniform adversary $\mathcal{A}'$ such that, for every oracle ω , it sends the queries in τ in order in its interaction with ω (e.g., $\mathcal{A}'$ has auxiliary input τ and sends its ordered queries). Fix $\mathcal{A}'$ in the following compatibility analysis so that any real-world or ideal-world oracle will receive the queries in τ in order. For a response c recorded in a fixed τ , let $\omega \vdash c$ denote the event that, fixing the ordered queries as per τ , oracle ω produces c given the corresponding query. Let $\mathcal{K}^R$ denote the order-preserving list of the responses in an ordered list $\mathcal{K}$ of permutation query-response pairs.

In the real world, an oracle $\omega = (\Delta, \{W_i\}_{i \in \mathcal{W}_{\text{in}}(f)}, \pi) \in \Omega_{\text{real}}$ according to half-gates and

$$\Pr_{\omega \leftarrow \Omega_{\text{real}}} [\omega \in \text{comp}_{\text{real}}(\tau)] = \Pr_{\omega \leftarrow \Omega_{\text{real}}} [\omega \vdash (\mathcal{K}_1^R, \hat{f}, \mathcal{K}_2^R, \tilde{X}, \tilde{x}, \tilde{U}, \mathcal{K}_3^R, \Delta)].$$

To begin with, every real-world ω certainly produces f' in $\hat{f}$ and $\tilde{x}$ in τ , being consistent with $\Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [Y(\omega) = \tau] > 0$. This non-zero probability implies that $(f', \tilde{x})$ in τ is honestly and deterministically computed from the fixed queries (f, x) . Otherwise, no ideal-world oracle, which computes $(f', \tilde{x})$ from the same deterministic procedure, can produce this transcript, contradicting the non-zero probability. As every real-world ω will follow the same deterministic procedure, it certainly produces the two values.

Meanwhile, a real-world oracle ω should have the same literal value of Δ as its counterpart in τ . Conditioned on the compatibility so far, the probability

$$\begin{aligned}
& \Pr_{\omega \leftarrow \Omega_{\text{real}}} [\omega \in \text{comp}_{\text{real}}(\tau)] \\
&= \Pr_{\omega \leftarrow \Omega_{\text{real}}} [\omega \vdash (\mathcal{K}_1^R, F, d, \mathcal{K}_2^R, \tilde{X}, \tilde{U}, \mathcal{K}_3^R) \mid \omega \vdash (f', \tilde{x}) \wedge \omega \vdash \Delta] \\
&\quad \cdot \Pr_{\omega \leftarrow \Omega_{\text{real}}} [\omega \vdash (f', \tilde{x}) \wedge \omega \vdash \Delta] \\
&= \Pr_{\omega \leftarrow \Omega_{\text{real}}} [\omega \vdash (\mathcal{K}_1^R, F, d, \mathcal{K}_2^R, \tilde{X}, \tilde{U}, \mathcal{K}_3^R) \mid \omega \vdash (f', \tilde{x}) \wedge \omega \vdash \Delta] \\
&\quad \cdot \Pr_{\omega \leftarrow \Omega_{\text{real}}} [\omega \vdash (f', \tilde{x})] \cdot \Pr_{\omega \leftarrow \Omega_{\text{real}}} [\omega \vdash \Delta] \\
&= \Pr_{\omega \leftarrow \Omega_{\text{real}}} [\omega \vdash (\mathcal{K}_1^R, F, d, \mathcal{K}_2^R, \tilde{X}, \tilde{U}, \mathcal{K}_3^R) \mid \omega \vdash (f', \tilde{x}) \wedge \omega \vdash \Delta] \cdot \frac{1}{2^{\lambda-1}}.
\end{aligned}$$

Conditioned on the compatibility with $(f', \tilde{x}, \Delta)$, a real-world ω should also be compatible with $(\cup_{\ell=1}^3 \mathcal{K}_\ell^R, F, \tilde{U})$ and some active labels in $\tilde{X}$ such that

- (i) $\pi^{\pm 1}$ respects the query-response pairs fixed in $\cup_{\ell=1}^3 \mathcal{K}_\ell^R$.
- (ii) For each $i \in \mathcal{W}_{\text{in}}(f)$, it holds that $X_i = W_i \oplus x_i \Delta$.
- (iii) For each $g \in \mathcal{G}_{\text{and}}(f)$ with $(a, b) := (\text{in}_0(g), \text{in}_1(g))$,

$$\begin{aligned}
H(X_a, k_0^g) &:= \pi(X_a \oplus k_0^g) \oplus \sigma(X_a \oplus k_0^g) = U_0^g, \\
H(X_b, k_1^g) &:= \pi(X_b \oplus k_1^g) \oplus \sigma(X_b \oplus k_1^g) = U_1^g.
\end{aligned} \tag{18}$$

- (iv) For each $g \in \mathcal{G}_{\text{and}}(f)$ with $(a, b, c) := (\text{in}_0(g), \text{in}_1(g), \text{out}(g))$,

$$X_c = H(X_a, k_0^g) \oplus H(X_b, k_1^g) \oplus s_a G_0^g \oplus s_b (G_1^g \oplus X_a), \tag{19}$$

$$\begin{aligned}
G_0^g &= H(X_a, k_0^g) \oplus H(X_a \oplus \Delta, k_0^g) \oplus (s_b \oplus x_b) \Delta \\
G_1^g &= H(X_b, k_1^g) \oplus H(X_b \oplus \Delta, k_1^g) \oplus x_a \Delta \oplus X_a,
\end{aligned} \tag{20}$$

where $x_a, x_b, s_a = \text{lsb}(X_a), s_b = \text{lsb}(X_b)$ are given in τ .

Conditioned on the compatibility so far, every real-world oracle ω is always compatible with the leftover values in τ , i.e., decoding information d and other active labels in $\tilde{X}$, which are deterministically computed from XOR combination. The reason is that, for τ ensuring $\Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [Y(\omega) = \tau] > 0$, these values should be honestly determined by the conditioned values as in the real world. Otherwise, this probability will be zero for an ideal-world oracle, which obtains them from a consistent deterministic computation as per the conditioned values. As every real-world oracle ω honestly follows the real-world computation, this “leftover” compatibility must hold conditioned on the previous compatibility.

It remains to compute the conditional probabilities for (i) to (iv). Consider (iii) and (iv). We note that every good τ satisfying $\Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [Y(\omega) = \tau] > 0$ already implies (18) and (19). To see this, one can check that condition $\neg \text{bad}_3$ for good transcripts and the extra queries ensure that $\mathcal{K}_3$ fixes the pairs of permutation pre-images and images for hash values

$$\{H(X_a, k_0^g), H(X_b, k_1^g)\}_{g \in \mathcal{G}_{\text{and}}(f), (a,b) := (\text{in}_0(g), \text{in}_1(g))}.$$

These values are consistent with $\tilde{U}$ fixed in τ as per (18). Otherwise, $\Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [Y(\omega) = \tau] > 0$ cannot be satisfied by τ since $\neg \text{bad}_1 \wedge \neg \text{bad}_3$ for every good transcript implies successful programming in the ideal world so that $H(X_a, k_0^g) = U_0^g$ and $H(X_b, k_1^g) = U_1^g$. As a corollary, (19) holds for every good transcript according to this consistency and the step 10 and 13 of SimIn.

Then, we rewrite (20), the leftover part of (iii) and (iv), as:

$$\mathcal{V} := \left\{ \begin{array}{l} g \in \mathcal{G}_{\text{and}}(f), (a, b) := (\text{in}_0(g), \text{in}_1(g)) : \\ \underbrace{\pi(X_a \oplus k_0^g)}_{P_{g,0}} \oplus \underbrace{\pi(X_a \oplus \Delta \oplus k_0^g)}_{P_{g,2}} = \sigma(\Delta) \oplus (s_b \oplus x_b)\Delta \oplus G_0^g, \\ \underbrace{\pi(X_b \oplus k_1^g)}_{P_{g,1}} \oplus \underbrace{\pi(X_b \oplus \Delta \oplus k_1^g)}_{P_{g,3}} = \sigma(\Delta) \oplus x_a\Delta \oplus X_a \oplus G_1^g \end{array} \right\}$$

As τ is a good transcript, there are exactly $4|f|$ pairwise distinct permutation pre-images on the left hand (otherwise, there will be a pair of permutation pre-images leading to (3) in bad_1 or a permutation pre-image leading to (10) $\vee$ (11) in bad_2 given the extra queries). $\mathcal{V}$ has exact $4|f|$ syntactically different variables $\mathcal{P} := \{P_{g,0}, P_{g,1}, P_{g,2}, P_{g,3}\}_{g \in \mathcal{G}_{\text{and}}(f)}$. They fix the same number of the entries of permutation π in a real-world ω *if and only if their literal values fixed by τ are also pairwise distinct*. Note that every good transcript τ indeed fixes *exact one* such assignment of these values for the following reasons:

- (18) already holds for τ , i.e., for $g \in \mathcal{G}_{\text{and}}(f)$ with $(a, b) := (\text{in}_0(g), \text{in}_1(g))$,

$$\begin{aligned} P_{g,0} &:= \pi(X_a \oplus k_0^g) = U_0^g \oplus \sigma(X_a \oplus k_0^g), \\ P_{g,1} &:= \pi(X_b \oplus k_1^g) = U_1^g \oplus \sigma(X_b \oplus k_1^g). \end{aligned} \quad (21)$$

The literal values of $\{P_{g,0}, P_{g,1}\}_{g \in \mathcal{G}_{\text{and}}(f)}$ can be fixed by the responses in $\mathcal{K}_3^R$ given the extra queries and will be pairwise distinct according to the impossible (4) from $\neg \text{bad}_1$.

- For each $g \in \mathcal{G}_{\text{and}}(f)$ with $(a, b) := (\text{in}_0(g), \text{in}_1(g))$, $\{P_{g,2}, P_{g,3}\}_{g \in \mathcal{G}_{\text{and}}(f)}$ are literally assigned the following values fixed by τ according to $\mathcal{V}$ and (21):

$$\begin{aligned} P_{g,2} &:= \pi(X_a \oplus \Delta \oplus k_0^g) = \sigma(\Delta) \oplus (s_b \oplus x_b)\Delta \oplus G_0^g \oplus U_0^g \oplus \sigma(X_a \oplus k_0^g), \\ P_{g,3} &:= \pi(X_b \oplus \Delta \oplus k_1^g) = \sigma(\Delta) \oplus x_a\Delta \oplus X_a \oplus G_1^g \oplus U_1^g \oplus \sigma(X_b \oplus k_1^g). \end{aligned} \quad (22)$$

The literal values of $\{P_{g,2}, P_{g,3}\}_{g \in \mathcal{G}_{\text{and}}(f)}$ are pairwise distinct due to the impossible (5) $\vee$ (6) $\vee$ (7) $\vee$ (8) $\vee$ (9) from $\neg \text{bad}_1$.

- The goodness of τ also ensures that there do not exist

$$P' \in \{P_{g,0}, P_{g,1}\}_{g \in \mathcal{G}_{\text{and}}(f)}, P'_\Delta \in \{P_{g,2}, P_{g,3}\}_{g \in \mathcal{G}_{\text{and}}(f)}$$

such that $P' = P'_\Delta$. Otherwise, this equality and (22) ensure that there exist $(g, u) \in \mathcal{G}_{\text{and}}(f) \times \{0, 1\}$ and $g' \in \mathcal{G}_{\text{and}}(f)$ such that

$$\begin{aligned} \pi(X_w \oplus k_u^g) &= \pi(X_{a'} \oplus \Delta \oplus k_0^{g'}) = \sigma(\Delta) \oplus (s_{b'} \oplus x_{b'})\Delta \oplus G_0^{g'} \oplus U_0^{g'} \oplus \sigma(X_{a'} \oplus k_0^{g'}) \quad \vee \\ \pi(X_w \oplus k_u^g) &= \pi(X_{b'} \oplus \Delta \oplus k_1^{g'}) = \sigma(\Delta) \oplus x_{a'}\Delta \oplus X_{a'} \oplus G_1^{g'} \oplus U_1^{g'} \oplus \sigma(X_{b'} \oplus k_1^{g'}) \end{aligned} \quad (23)$$

where $w := \text{in}_u(g)$ and $(a', b') := (\text{in}_0(g'), \text{in}_1(g'))$. Recall that $\neg \text{bad}_3$ and the extra queries implies $(X_w \oplus k_u^g, \pi(X_w \oplus k_u^g)) \in \mathcal{K}_3$. As a result, (23) leads to contradiction with the impossible (12) $\vee$ (13) from $\neg \text{bad}_2$.

Putting these cases together, τ results in a value assignment of $\mathcal{P}$, which fixes exact $4|f|$ entries of real-world permutation π .

Conditioned on $\neg \text{bad}_1 \wedge \neg \text{bad}_3$ of good transcript τ , $2|f|$ extra queries are non-repeating and the number of non-repeating queries is $q + 2|f| = q_\Sigma$. So, $2|f|$ responses in $\cup_{\ell=1}^3 \mathcal{K}_\ell^R$ for the non-repeating extra queries are fixed by the values in $\mathcal{P}$ while the other $q_\Sigma - 2|f| = q$ responses are fixed by real-world

π (conditioned on the values in $\mathcal{P}$). Using (i), (ii), (iii), and (iv) together with the “leftover” compatibility, we have in the real world that

$$\begin{aligned}
& \Pr_{\omega \leftarrow \Omega_{\text{real}}} [\omega \vdash (\mathcal{K}_1^R, F, d, \mathcal{K}_2^R, \tilde{X}, \tilde{U}, \mathcal{K}_3^R) \mid \omega \vdash (f', \tilde{x}) \wedge \omega \vdash \Delta] \\
&= \frac{1}{(2^\lambda)^{|\mathcal{W}_{\text{in}}(f)|}} \cdot \frac{(2^\lambda - 4|f| - (q_\Sigma - 2|f|))!}{(2^\lambda)!} \\
&= \frac{1}{(2^\lambda)^{|\mathcal{W}_{\text{in}}(f)|}} \cdot \frac{1}{(2^\lambda)_{q+4|f|}}, \\
&\Rightarrow \Pr_{\omega \leftarrow \Omega_{\text{real}}} [\omega \in \text{comp}_{\text{real}}(\tau)] = \frac{1}{(2^\lambda)^{|\mathcal{W}_{\text{in}}(f)|}} \cdot \frac{1}{(2^\lambda)_{q+4|f|}} \cdot \frac{1}{2^{\lambda-1}}.
\end{aligned}$$

In the ideal world, we can use a similar argument to show

$$\begin{aligned}
& \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\omega \in \text{comp}_{\text{ideal}}(\tau)] \\
&= \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\omega \vdash (\mathcal{K}_1^R, \mathcal{K}_2^R, \mathcal{K}_3^R) \mid \omega \vdash (\hat{f}, \tilde{X}, \tilde{x}, \tilde{U}, \Delta)] \cdot \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\omega \vdash (\hat{f}, \tilde{X}, \tilde{x}, \tilde{U}, \Delta)] \\
&= \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\omega \vdash (\mathcal{K}_1^R, \mathcal{K}_2^R, \mathcal{K}_3^R) \mid \omega \vdash (\hat{f}, \tilde{X}, \tilde{x}, \tilde{U}, \Delta)] \cdot \frac{1}{2^{|\mathcal{W}_{\text{in}}(f)|\lambda + 2\lambda|f| + (\lambda-1) + 2\lambda|f|}}.
\end{aligned}$$

Due to the condition $\neg \text{bad}_1 \wedge \neg \text{bad}_3$ and the $2|f|$ extra queries, there are exact $q + 2|f| = q_\Sigma$ non-repeating queries. Moreover, $2|f|$ responses in $\cup_{\ell=1}^3 \mathcal{K}_\ell^R$ for the non-repeating extra queries are fixed by the conditioned values but the other responses are fixed by $\text{SimP}^{\pm 1}$ for other $q_\Sigma - 2|f| = q$ queries.

Let $\mathcal{Q}_{i-1}$ denote the list $\mathcal{Q}$ (which is maintained in internal state st_{sim}) when it includes $i - 1 \in [0, q_\Sigma - 1]$ pairs (note that $\mathcal{Q}$ finally includes q_Σ pairs given the q_Σ non-repeating queries), and $\mathcal{N} \subseteq [1, q_\Sigma]$ denote the index set of these q queries in q_Σ non-repeating queries to $\text{SimP}^{\pm 1}(\cdot)$ such that $|\mathcal{N}| = q$. We have

$$\begin{aligned}
& \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\omega \vdash (\mathcal{K}_1^R, \mathcal{K}_2^R, \mathcal{K}_3^R) \mid \omega \vdash (\hat{f}, \tilde{X}, \tilde{x}, \tilde{U}, \Delta)] \\
&= \prod_{i \in \mathcal{N}} \frac{1}{2^\lambda - |\mathcal{Q}_{i-1}|} = \prod_{i \in \mathcal{N}} \frac{1}{2^\lambda - (i-1)} \\
&\leq \frac{1}{2^\lambda - 2|f|} \times \frac{1}{2^\lambda - (2|f| + 1)} \times \cdots \times \frac{1}{2^\lambda - (q_\Sigma - 1)} = \frac{1}{(2^\lambda - 2|f|)_q}, \\
&\Rightarrow \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\omega \in \text{comp}_{\text{ideal}}(\tau)] \leq \frac{1}{(2^\lambda)^{|\mathcal{W}_{\text{in}}(f)|}} \cdot \frac{1}{(2^\lambda - 2|f|)_q} \cdot \frac{1}{(2^\lambda)^{4|f|}} \cdot \frac{1}{2^{\lambda-1}}.
\end{aligned}$$

So, we can have $\varepsilon_2 = 0$ since, for every $|f| \geq 0$ and $q \geq 0$,

$$\begin{aligned}
\frac{\Pr_{\omega \leftarrow \Omega_{\text{real}}} [\omega \in \text{comp}_{\text{real}}(\tau)]}{\Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\omega \in \text{comp}_{\text{ideal}}(\tau)]} &\geq \frac{(2^\lambda - 2|f|)_q \cdot (2^\lambda)^{4|f|}}{(2^\lambda)_{q+4|f|}} \\
&\geq \frac{(2^\lambda - 2|f|)_q \cdot (2^\lambda)^{2|f|} \cdot (2^\lambda)^{2|f|}}{(2^\lambda)_{q+4|f|}} \\
&= \frac{(2^\lambda)_{q+2|f|} \cdot (2^\lambda)^{2|f|}}{(2^\lambda)_{q+4|f|}} = \frac{(2^\lambda)^{2|f|}}{(2^\lambda - (q + 2|f|))_{2|f|}} \geq 1.
\end{aligned}$$

Bounding ε_1 . We bound the probabilities of the bad events in the *ideal world*. First, consider bad_1 . Note that each active label X_i can be written as the XOR of (i) some active circuit input labels, and/or (ii) some active output labels of the *precedent* AND gates, i.e., for $\mathcal{I}_i \ominus \mathcal{J}_i \neq \emptyset$,

$$X_i = \left(\bigoplus_{w \in \mathcal{I}_i \subseteq \mathcal{W}_{\text{in}}(f)} X_w \right) \oplus \left(\bigoplus_{w \in \mathcal{J}_i \subseteq \mathcal{W}_{\text{and}}(f)} X_w \right) = \bigoplus_{w \in \mathcal{I}_i \ominus \mathcal{J}_i} X_w \in \{0, 1\}^\lambda. \quad (24)$$

For (3), we can use (24) to rewrite it as

$$\bigoplus_{i \in (\mathcal{I}_w \ominus \mathcal{I}_{w'}) \ominus (\mathcal{J}_w \ominus \mathcal{J}_{w'})} X_i = k_u^g \oplus k_{u'}^{g'} \in \{0, 1\}^\lambda.$$

According to SimIn, each X_i , which is sampled in the step 1 or computed in the step 13, has at least $\lambda - 1$ random non-LSBs. Therefore, the equality holds with probability at most $2^{-(\lambda-1)}$ for some fixed distinct k_u^g and $k_{u'}^{g'}$ (or equivalently, (g, u) and (g', u')). If $\mathcal{I}_w = \mathcal{I}_{w'}$ and $\mathcal{J}_w = \mathcal{J}_{w'}$, this probability will be zero for the distinct k_u^g and $k_{u'}^{g'}$. For (4), the worst case is that U_u^g and $U_{u'}^{g'}$ are used for the same AND gate with an output wire in $\mathcal{Z}(f)$. In this case, the XOR $U_u^g \oplus U_{u'}^{g'}$ has at least $\lambda - 1$ random non-LSBs due to some X_c sampled in the step 1 of SimIn. Such non-LSBs are independent of other (previously fixed) active labels, including X_w and $X_{w'}$. So, the equality holds with probability at most $2^{-(\lambda-1)}$ for some fixed (g, u) and (g', u') .

In the same gate g , the upper $\lambda - 1$ bits of $U_0^g \oplus U_1^g$ are uniform so that (5) holds with probability at most $2^{-(\lambda-1)}$. Otherwise, the distinct $g \neq g'$ implies that, for every $u, u' \in \{0, 1\}$, $U_u^g \oplus U_{u'}^{g'} \in \{0, 1\}^\lambda$ is uniform. As a result, each of (6), (7), (8), and (9) holds with probability $2^{-\lambda}$.

Taking a union bound over the above cases, we have

$$\Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\text{bad}_1] \leq \frac{2|f|(2|f| - 1) + |f|^2}{2^{\lambda-1}} = \frac{5|f|^2 - 2|f|}{2^{\lambda-1}}. \quad (29)$$

Then, consider bad_2 . From (10) (resp., (11)), we have $\Delta = \alpha \oplus X_a \oplus k_0^g$ (resp., $\Delta = \alpha \oplus X_b \oplus k_1^g$), occurring with probability $2^{-(\lambda-1)}$ due to the randomness of Δ . In (12), if $s_b \oplus x_b = 0$, linear orthomorphism σ ensures that

$$\Delta = \sigma^{-1}(\beta \oplus G_0^g \oplus U_0^g \oplus \sigma(X_a \oplus k_0^g)),$$

which occurs with probability $2^{-(\lambda-1)}$; if $s_b \oplus x_b = 1$, according to permutation $\sigma'(x) := \sigma(x) \oplus x$ well-defined from σ , it holds with the same probability that

$$\Delta = \sigma'^{-1}(\beta \oplus G_0^g \oplus U_0^g \oplus \sigma(X_a \oplus k_0^g)).$$

Similar result holds for (13). Taking a union bound over all pairs, we have

$$\Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\text{bad}_2] \leq \frac{4|f| \cdot (q_1 + q_2 + q_3)}{2^{\lambda-1}}. \quad (30)$$

Finally, consider bad_3 . Recall that each X_i has at least $\lambda - 1$ random non-LSBs. Since these bits are independent of $\bigcup_{\ell=1}^2 \mathcal{K}_\ell$, each of (14) and (15) holds with probability at most $2^{-(\lambda-1)}$ for some fixed $(\alpha, \dots)$ and g . For each of (16) and (17), the mask sampled in the step 9 of SimIn (resp., the direct sampling in the step 9 or 12 of SimIn) ensures that $U_0^g \in \{0, 1\}^\lambda$ (resp., $U_1^g \in \{0, 1\}^\lambda$) is uniform and independent of X_a (resp., X_b) or $\bigcup_{\ell=1}^2 \mathcal{K}_\ell$. So, each equality holds with probability $2^{-\lambda} < 2^{-(\lambda-1)}$ for some fixed $(\dots, \beta)$ and g . It follows from a union bound that

$$\Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\text{bad}_3] \leq \frac{4|f| \cdot (q_1 + q_2)}{2^{\lambda-1}}. \quad (31)$$

We have a bound ε_1 from (29), (30), and (31):

$$\begin{aligned} & \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [Y(\omega) \in \mathcal{T}_{\text{bad}}] \\ &= \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\text{bad}_1 \vee \text{bad}_2 \vee \text{bad}_3] \\ &\leq \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\text{bad}_1] + \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\text{bad}_2] + \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\text{bad}_3] \\ &\leq \frac{5|f|^2 - 2|f| + 8|f| \cdot (q + 2|f|)}{2^{\lambda-1}} \\ &= \frac{8qs + 21s^2 - 2s}{2^{\lambda-1}} = \varepsilon_1. \end{aligned}$$

This lemma follows from the H-coefficient bound and $(\varepsilon_1, \varepsilon_2)$. $\square$

Public parameters: (See Appendix F for concrete instantiations)

- $M \in \mathbb{F}_2^{8 \times 6}$ with $K \in \mathbb{F}_2^{3 \times 8}$ from the co-kernel basis of M , i.e., $KM = \mathbf{0}_{3 \times 6}$.
- $V = [V_{00} \ V_{01} \ V_{10} \ V_{11}]^\top \in (\mathbb{F}_2^{2 \times 5})^4 \equiv \mathbb{F}_2^{8 \times 5}$ with left inverse $V^+ \in \mathbb{F}_2^{5 \times 8}$.
- Basis matrices $S_L, S_R \in \mathbb{F}_2^{2 \times 4}$.
- Control matrices $R_p = [R_{p,00} \ R_{p,01} \ R_{p,10} \ R_{p,11}]^\top \in (\mathbb{F}_2^{2 \times 4})^4 \equiv \mathbb{F}_2^{8 \times 4}$, and $R'_1, R'_2 \in \mathbb{F}_2^{4 \times 2}$.
- A distribution $\mathcal{R}_0$ over $(\mathbb{F}_2^{1 \times 2})^4 \equiv \mathbb{F}_2^{4 \times 2}$ such that, for every $R'_\S = [R'_{\S,00} \ R'_{\S,01} \ R'_{\S,10} \ R'_{\S,11}]^\top \leftarrow \mathcal{R}_0$, (i) $K [\hat{R}_{\S,00} \ \hat{R}_{\S,01} \ \hat{R}_{\S,10} \ \hat{R}_{\S,11}]^\top = \mathbf{0}_{3 \times 6}$, where $\hat{R}_{\S,ij} := (R'_{\S,ij} \otimes I_2) \begin{bmatrix} S_L \\ S_R \end{bmatrix} \left(\begin{bmatrix} 1 & 0 & i \\ 0 & 1 & j \end{bmatrix} \otimes I_2 \right)$ for every $i, j \in \mathbb{F}_2$, and (ii) for every $i, j \in \mathbb{F}_2$, the marginal distribution of $R'_{\S,ij}$ is uniform.

TH.SampleR(t):

- 1: $\zeta_1 := g(\overline{p_a}, p_b) \oplus g(\overline{p_a}, \overline{p_b})$, $\zeta_2 := g(p_a, \overline{p_b}) \oplus g(\overline{p_a}, \overline{p_b})$
- 2: $R'_\S \leftarrow \mathcal{R}_0$, $\begin{bmatrix} r_{00}^\top \\ r_{01}^\top \\ r_{10}^\top \\ r_{11}^\top \end{bmatrix} = \begin{bmatrix} r_{00L} & r_{00R} \\ r_{01L} & r_{01R} \\ r_{10L} & r_{10R} \\ r_{11L} & r_{11R} \end{bmatrix} := R'_\S \oplus \zeta_1 R'_1 \oplus \zeta_2 R'_2$
- 3: **for** $i, j \in \mathbb{F}_2$ **do**
- 4: $R_{ij} := (r_{ij}^\top \otimes I_2) \begin{bmatrix} S_L \\ S_R \end{bmatrix} \oplus R_{p,ij}$, $\hat{R}_{ij} := R_{ij} \left(\begin{bmatrix} 1 & 0 & i \\ 0 & 1 & j \end{bmatrix} \otimes I_2 \right)$
- 5: $\hat{R} := [\hat{R}_{00} \ \hat{R}_{01} \ \hat{R}_{10} \ \hat{R}_{11}]^\top$
- 6: $r := [r_{00L} \ r_{00R} \ r_{01L} \ r_{01R} \ r_{10L} \ r_{10R} \ r_{11L} \ r_{11R}]^\top$
- 7: **return** $(\hat{R}, r)$

TH.DecodeR(r_{ij}, i, j):

- 1: **return** $R_{ij} := (r_{ij}^\top \otimes I_2) \begin{bmatrix} S_L \\ S_R \end{bmatrix} \oplus R_{p,ij}$

Figure 8: Three-halves garbling scheme [RR21] (Part I).

D Adaptive Security of Three-Halves

We consider the three-halves scheme [RR21] with the computational optimization using both halves of hash outputs (see Section 6.2 therein). It uses a counter per wire rather than an identifier per gate to compute tweaks in hash computation. We begin with the implementation in the pRPM that sends decoding information d in the offline phase, followed by another implementation in the nRPM that postpones d to the online phase. Note that three-halves uses an equivalent invariant for each wire i : its active label $X_i = W_i \oplus (p_i \oplus x_i)\Delta$ for its wire label W_i of bit 0 with $\text{lsb}(W_i) = 0$, truth bit x_i , permuted bit p_i , and global key Δ with $\text{lsb}(\Delta) = 1$.

For some $\pi \in \mathcal{S}_\ell$ and $x = [x_L \ x_R]^\top \in \mathbb{F}_{2^{\ell/2}}^2$, we slightly abuse the notation $y := \pi(x)$ for the following computation: $(y_L \parallel y_R) := \pi(x_L \parallel x_R) \in \{0, 1\}^\ell$ and $y := [y_L \ y_R]^\top \in \mathbb{F}_{2^{\ell/2}}^2$. We refer to Appendix A for the additional notation.

D.1 Adaptive Security in pRPM

Theorem 3. Let $H(X, k) = \pi((0 \parallel X) \oplus k) \oplus \sigma((0 \parallel X) \oplus k)$ be a tweakable hash function where $X \in \mathbb{F}_{2^{\lambda/2}}^2$, $k \in \mathbb{F}_{2^{\lambda/2+1}}^2$, $\pi \in \mathcal{S}_{\lambda+2}$ is random permutation, and $\sigma : \mathbb{F}_{2^{\lambda/2+1}}^2 \rightarrow \mathbb{F}_{2^{\lambda/2+1}}^2$ is a linear orthomorphism for the function family

$$\mathcal{L} = \{L_{\xi_1, \xi_2, \xi_3, \xi_4} : \mathbb{F}_{2^{\lambda/2+1}}^2 \rightarrow \mathbb{F}_{2^{\lambda/2+1}}^2\}_{\xi_1, \xi_2, \xi_3, \xi_4 \in \mathbb{F}_2}, \quad L_{\xi_1, \xi_2, \xi_3, \xi_4} \left(\begin{bmatrix} x_L \\ x_R \end{bmatrix} \right) = \begin{bmatrix} \xi_1 x_L \oplus \xi_2 x_R \\ \xi_3 x_L \oplus \xi_4 x_R \end{bmatrix}.$$

Then, three-halves (Figure 8) is a $(\lambda + 2)$ -garbling scheme with (q, s, ε) -adaptive security in the pRPM, where $\varepsilon = (48qs + 189s^2)/2^{\lambda+1}$.

TH.Garble $^{\pi^{\pm 1}(\cdot)}(f)$:

```

1:  $\Delta \leftarrow \begin{bmatrix} \mathbb{F}_{2^{\lambda/2}} \\ \mathbb{F}_{2^{\lambda/2-1}} \parallel 1 \end{bmatrix}$ 
2: for  $i \in \mathcal{W}_{\text{in}}(f)$  do
3:    $p_i \leftarrow \mathbb{F}_2, \mathbf{W}_i \leftarrow \begin{bmatrix} \mathbb{F}_{2^{\lambda/2}} \\ \mathbb{F}_{2^{\lambda/2-1}} \parallel 0 \end{bmatrix}$ 
4: for  $i \in \mathcal{W}_{\cup}(f)$  do  $\text{ctr}_i := 0$ 
5: for  $g \in \mathcal{G}(f)$  in order do
6:    $(a, b, c) := (\text{in}_0(g), \text{in}_1(g), \text{out}(g))$ 
7:   if  $\text{type}(g) = \text{XOR}$  do  $p_c := p_a \oplus p_b, \mathbf{W}_c := \mathbf{W}_a \oplus \mathbf{W}_b$ 
8:   else if  $\text{type}(g) = \text{AND}$  do
9:      $\Gamma := \text{in}_2(g)$ 
10:     $(\chi_a, \rho_a) := (\lfloor \text{ctr}_a/2 \rfloor, \text{lsb}(\text{ctr}_a)), \text{ctr}_a := \text{ctr}_a + 1$ 
11:     $(\chi_b, \rho_b) := (\lfloor \text{ctr}_b/2 \rfloor, \text{lsb}(\text{ctr}_b)), \text{ctr}_b := \text{ctr}_b + 1$ 
12:     $(\chi_\Gamma, \rho_\Gamma) := (\lfloor \text{ctr}_\Gamma/2 \rfloor, \text{lsb}(\text{ctr}_\Gamma)), \text{ctr}_\Gamma := \text{ctr}_\Gamma + 1$ 
13:     $\mathbf{t}^g := [g(p_a, p_b) \ g(p_a, \overline{p_b}) \ g(\overline{p_a}, p_b) \ g(\overline{p_a}, \overline{p_b})]^\top \in \mathbb{F}_2^4$ 
14:     $(\hat{\mathbf{R}}^g, \mathbf{r}^g) \leftarrow \text{TH.SampleR}(\mathbf{t}^g)$ 
15:    Compute  $(\mathbf{c}^g, \mathbf{g}^g, \mathbf{z}^g) \in \mathbb{F}_{2^{\lambda/2}}^2 \times \mathbb{F}_{2^{\lambda/2}}^3 \times \mathbb{F}_2^5$  as follows: for

```

$$\mathbf{h}^g := \begin{bmatrix} \text{Half}_{\rho_a}(\text{H}(\mathbf{W}_a, a \parallel \chi_a)) \\ \text{Half}_{\rho_a}(\text{H}(\mathbf{W}_a \oplus \Delta, a \parallel \chi_a)) \\ \text{Half}_{\rho_b}(\text{H}(\mathbf{W}_b, b \parallel \chi_b)) \\ \text{Half}_{\rho_b}(\text{H}(\mathbf{W}_b \oplus \Delta, b \parallel \chi_b)) \\ \text{Half}_{\rho_\Gamma}(\text{H}(\mathbf{W}_a \oplus \mathbf{W}_b, \Gamma \parallel \chi_\Gamma)) \\ \text{Half}_{\rho_\Gamma}(\text{H}(\mathbf{W}_a \oplus \mathbf{W}_b \oplus \Delta, \Gamma \parallel \chi_\Gamma)) \end{bmatrix} \in \mathbb{F}_{2^{\lambda/2+1}}^6,$$

$$\left(\mathbf{z}^g \parallel \begin{bmatrix} \mathbf{c}^g \\ \mathbf{g}^g \end{bmatrix} \right) := \mathbf{V}^+ \left(\mathbf{M} \mathbf{h}^g \oplus \left(\mathbf{r}^g \parallel (\hat{\mathbf{R}}^g \oplus ([\mathbf{0}_{4 \times 2} \ \mathbf{t}^g] \otimes \mathbf{I}_2)) \begin{bmatrix} \mathbf{W}_a \\ \mathbf{W}_b \\ \Delta \end{bmatrix} \right) \right)$$

```

16:    $p_c := \text{lsb}(\mathbf{c}^g), \mathbf{W}_c := \mathbf{c}^g \oplus p_c \Delta$ 
17: for  $i \in \mathcal{W}_{\text{out}}(f)$  do  $d_i := p_i$ 
18: return  $\hat{f} := (f' := f, F := \{(\mathbf{g}^g, \mathbf{z}^g)\}_{g \in \mathcal{G}_{\text{and}}(f)}, d), k := (f, \Delta, p, \mathbf{W})$ 

```

TH.DecEval $^{\pi^{\pm 1}(\cdot)}(\hat{f}, \hat{x})$:

```

1: for  $i \in \mathcal{W}_{\cup}(f)$  do  $\text{ctr}_i := 0$ 
2: for  $g \in \mathcal{G}(f)$  in order do
3:    $(a, b, c) := (\text{in}_0(g), \text{in}_1(g), \text{out}(g))$ 
4:   if  $\text{type}(g) = \text{XOR}$  do  $\mathbf{X}_c := \mathbf{X}_a \oplus \mathbf{X}_b$ 
5:   else if  $\text{type}(g) = \text{AND}$  do
6:      $\Gamma := \text{in}_2(g)$ 
7:      $(\chi_a, \rho_a) := (\lfloor \text{ctr}_a/2 \rfloor, \text{lsb}(\text{ctr}_a)), \text{ctr}_a := \text{ctr}_a + 1$ 
8:      $(\chi_b, \rho_b) := (\lfloor \text{ctr}_b/2 \rfloor, \text{lsb}(\text{ctr}_b)), \text{ctr}_b := \text{ctr}_b + 1$ 
9:      $(\chi_\Gamma, \rho_\Gamma) := (\lfloor \text{ctr}_\Gamma/2 \rfloor, \text{lsb}(\text{ctr}_\Gamma)), \text{ctr}_\Gamma := \text{ctr}_\Gamma + 1$ 
10:     $s_a := \text{lsb}(\mathbf{X}_a), s_b := \text{lsb}(\mathbf{X}_b)$ 
11:     $(\mathbf{r}_{s_a s_b}^g \parallel \mathbf{m}_{s_a s_b}^g) := \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix} \begin{bmatrix} \text{Half}_{\rho_a}(\text{H}(\mathbf{X}_a, a \parallel \chi_a)) \\ \text{Half}_{\rho_b}(\text{H}(\mathbf{X}_b, b \parallel \chi_b)) \\ \text{Half}_{\rho_\Gamma}(\text{H}(\mathbf{X}_a \oplus \mathbf{X}_b, \Gamma \parallel \chi_\Gamma)) \end{bmatrix} \oplus \mathbf{V}_{s_a s_b} \left( \mathbf{z}^g \parallel \begin{bmatrix} \mathbf{0} \\ \mathbf{g}^g \end{bmatrix} \right)$ 
12:     $\mathbf{R}_{s_a s_b}^g := \text{TH.DecodeR}(\mathbf{r}_{s_a s_b}^g, s_a, s_b)$ 
13:     $\mathbf{X}_c := \mathbf{m}_{s_a s_b}^g \oplus \mathbf{R}_{s_a s_b}^g \begin{bmatrix} \mathbf{X}_a \\ \mathbf{X}_b \end{bmatrix}$ 
14: for  $i \in \mathcal{W}_{\text{out}}(f)$  do  $y_i := d_i \oplus \text{lsb}(\mathbf{X}_i)$ 
15: return  $y$ 

```

Figure 8: Three-halves garbling scheme [RR21] (Part II).

SimF(f):

- 1: $F \leftarrow (\mathbb{F}_{2^{\lambda/2}}^3 \times \mathbb{F}_2^5)^{|f|}$, $d \leftarrow \mathbb{F}_2^{|\mathcal{W}_{\text{out}}(f)|}$ such that
 - (i) For each $i \in [1, m]$ and each wire $j \in \mathcal{Y}_i(f)$, $d_j = d_{y_i(f)}$.
 - (ii) For each $(\mu_1, \dots, \mu_m) \in \mathbb{F}_2^m$ s.t. $\bigoplus_{i \in [1, m], \mu_i=1} \mathcal{X}_i(f) = \emptyset$, $\bigoplus_{i \in [1, m]} \mu_i \cdot d_{y_i(f)} = 0$.
- 2: **return** $\hat{f} := (f' := f, F, d)$, $\text{st}_{\text{sim}} := (\text{st}_{\text{sim}}, f, F, d)$

SimIn($f(x)$):

- 1: $\{\mathbf{X}_i\}_{i \in \mathcal{X}(f)} \leftarrow (\mathbb{F}_{2^{\lambda/2}}^2)^{|\mathcal{X}(f)|}$ such that, for each $j \in [1, m]$, $\text{lsb}(\bigoplus_{i \in \mathcal{X}_j(f)} \mathbf{X}_i) = d_{y_j(f)} \oplus f(x)_{y_j(f)}$.
- 2: **for** $i \in \mathcal{W}_{\cup}(f)$ **do** $\text{ctr}_i := 0$
- 3: **for** $g \in \mathcal{G}(f)$ **in order do**
- 4: $(a, b, c) := (\text{in}_0(g), \text{in}_1(g), \text{out}(g))$
- 5: **if** $\text{type}(g) = \text{XOR}$ **do** $\mathbf{X}_c := \mathbf{X}_a \oplus \mathbf{X}_b$
- 6: **else if** $\text{type}(g) = \text{AND}$ **do**
- 7: $\Gamma := \text{in}_2(g)$
- 8: $(\chi_a, \rho_a) := (\lfloor \text{ctr}_a/2 \rfloor, \text{lsb}(\text{ctr}_a))$, $\text{ctr}_a := \text{ctr}_a + 1$
- 9: $(\chi_b, \rho_b) := (\lfloor \text{ctr}_b/2 \rfloor, \text{lsb}(\text{ctr}_b))$, $\text{ctr}_b := \text{ctr}_b + 1$
- 10: $(\chi_\Gamma, \rho_\Gamma) := (\lfloor \text{ctr}_\Gamma/2 \rfloor, \text{lsb}(\text{ctr}_\Gamma))$, $\text{ctr}_\Gamma := \text{ctr}_\Gamma + 1$
- 11: $s_a := \text{lsb}(\mathbf{X}_a)$, $s_b := \text{lsb}(\mathbf{X}_b)$
- 12: **if** $c \in \mathcal{Z}(f)$ **do**
- 13: $\mathbf{r}_{s_a s_b}^g \leftarrow \mathbb{F}_2^2$, $\mathbf{R}_{s_a s_b}^g := \text{TH.DecodeR}(\mathbf{r}_{s_a s_b}^g, s_a, s_b)$
- 14: $\text{Half}_{\rho_\Gamma}(\mathbf{u}_\Gamma^{\chi_\Gamma}) \leftarrow \mathbb{F}_{2^{\lambda/2+1}}$
- 15: $\begin{bmatrix} \text{Half}_{\rho_a}(\mathbf{u}_a^{\chi_a}) \\ \text{Half}_{\rho_b}(\mathbf{u}_b^{\chi_b}) \end{bmatrix} := \begin{bmatrix} \text{Half}_{\rho_\Gamma}(\mathbf{u}_\Gamma^{\chi_\Gamma}) \\ \text{Half}_{\rho_\Gamma}(\mathbf{u}_\Gamma^{\chi_\Gamma}) \end{bmatrix} \oplus \left(\mathbf{r}_{s_a s_b}^g \parallel \mathbf{X}_c \oplus \mathbf{R}_{s_a s_b}^g \begin{bmatrix} \mathbf{X}_a \\ \mathbf{X}_b \end{bmatrix} \right) \oplus \mathbf{V}_{s_a s_b} \left(\mathbf{z}^g \parallel \begin{bmatrix} \mathbf{0} \\ \mathbf{g}^g \end{bmatrix} \right)$
- 16: **else if** $c \notin \mathcal{Z}(f)$ **do**
- 17: $\text{Half}_{\rho_a}(\mathbf{u}_a^{\chi_a})$, $\text{Half}_{\rho_b}(\mathbf{u}_b^{\chi_b})$, $\text{Half}_{\rho_\Gamma}(\mathbf{u}_\Gamma^{\chi_\Gamma}) \leftarrow \mathbb{F}_{2^{\lambda/2+1}}$
- 18: $(\mathbf{r}_{s_a s_b}^g \parallel \mathbf{m}_{s_a s_b}^g) := \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix} \begin{bmatrix} \text{Half}_{\rho_a}(\mathbf{u}_a^{\chi_a}) \\ \text{Half}_{\rho_b}(\mathbf{u}_b^{\chi_b}) \\ \text{Half}_{\rho_\Gamma}(\mathbf{u}_\Gamma^{\chi_\Gamma}) \end{bmatrix} \oplus \mathbf{V}_{s_a s_b} \left(\mathbf{z}^g \parallel \begin{bmatrix} \mathbf{0} \\ \mathbf{g}^g \end{bmatrix} \right)$
- 19: $\mathbf{R}_{s_a s_b}^g := \text{TH.DecodeR}(\mathbf{r}_{s_a s_b}^g, s_a, s_b)$
- 20: $\mathbf{X}_c := \mathbf{m}_{s_a s_b}^g \oplus \mathbf{R}_{s_a s_b}^g \begin{bmatrix} \mathbf{X}_a \\ \mathbf{X}_b \end{bmatrix}$
- 21: **for** $i \in \mathcal{W}_{\cup}(f)$ **do**
- 22: **for** $j \in [0, \lfloor n_i(f)/2 \rfloor - 1]$ **do**
- 23: (Programming) Add a pair $\left((\mathbf{0} \parallel \mathbf{X}_i) \oplus (i \parallel j), \mathbf{u}_i^j \oplus \sigma((\mathbf{0} \parallel \mathbf{X}_i) \oplus (i \parallel j)) \right)$ to the list $\mathcal{Q}$ kept in st_{sim} , if it causes no pre-image collision or image collision in $\mathcal{Q}$, to ensure

$$\underbrace{\pi((\mathbf{0} \parallel \mathbf{X}_i) \oplus (i \parallel j)) \oplus \sigma((\mathbf{0} \parallel \mathbf{X}_i) \oplus (i \parallel j))}_{\text{H}(\mathbf{X}_i, i \parallel j)} = \mathbf{u}_i^j = \begin{bmatrix} \text{Half}_1(\mathbf{u}_i^j) \\ \text{Half}_0(\mathbf{u}_i^j) \end{bmatrix}.$$
- 24: **if** $n_i(f)$ is odd **do**
- 25: $j := \lfloor n_i(f)/2 \rfloor$, $\text{Half}_1(\mathbf{u}_i^j) \leftarrow \mathbb{F}_{2^{\lambda/2+1}}$ and repeat the step 23 for this j
- 26: **return** $\hat{x} := \{\mathbf{X}_i\}_{i \in \mathcal{W}_{\text{in}}(f)}$, $\text{st}_{\text{sim}} := (\text{st}_{\text{sim}}, \tilde{\mathbf{X}}, \tilde{\mathbf{u}}, \tilde{\mathbf{r}})$ where st_{sim} on the right hand has an updated list $\mathcal{Q}$, $\tilde{\mathbf{X}} := \{\mathbf{X}_i\}_{i \in \mathcal{W}(f)}$, $\tilde{\mathbf{u}} := \{\mathbf{u}_i^j\}_{i \in \mathcal{W}_{\cup}(f), j \in [0, \lfloor n_i(f)/2 \rfloor - 1]}$, $\tilde{\mathbf{r}} := \{\mathbf{r}_{s_a s_b}^g\}_{g \in \mathcal{G}_{\text{and}}(f), (a,b) := (\text{in}_0(g), \text{in}_1(g))}$.

Figure 9: Our simulator for three-halves in the pRPM.

Proof. The correctness has been proved in the original work [RR21]. We only need to consider the simulation.

Our simulator Sim consists of (SimF, SimIn) in Figure 9 and $\text{SimP}^{\pm 1}$, which emulates the random permutation and its inverse on-the-fly. More specifically, there is a list $\mathcal{Q}$ of query-response pairs in internal state st_{sim} . Upon receiving forward query α (resp., backward query β) from $\mathcal{A}_i$ to SimP (resp., SimP^{-1}), it reads $\mathcal{Q}$ from st_{sim} and checks whether $\exists(\alpha, \gamma) \in \mathcal{Q}$ (resp., $\exists(\gamma, \beta) \in \mathcal{Q}$). If true, it returns γ as response; otherwise it samples $\gamma \leftarrow \{s \in \mathbb{F}_{2^{\lambda/2+1}} \mid (\dots, s) \notin \mathcal{Q}\}$ (resp., $\gamma \leftarrow \{s \in$

$\mathbb{F}_2^{2^{\lambda/2+1}} \mid (s, \dots) \notin Q\}$), adds (α, γ) (resp., (γ, β)) to Q , and returns γ as response. The programming is the step 23 of SimIn, where (α, β) is added to Q if and only if there is no pre-image collision with $(\alpha, \dots) \in Q$ or image collision with $(\dots, \beta) \in Q$.

To see that Sim is PPT, we note that the circuit-dependent notation can be efficiently computed by traversing the polynomial-size circuit f . Then, the crux is to show that the step 1 in both SimF and SimIn can be polynomial-time.

The runtime of the step 1 of SimF is dominated by the runtime of iterating through all qualified $(\mu_1, \dots, \mu_m) \in \mathbb{F}_2^m$. To find the qualified vectors, one can interpret each $X_i(f)$ as a one-hot *non-zero* column vector in the space $\mathbb{F}_2^{|\mathcal{W}(f)|}$ and derive a $|\mathcal{W}(f)|$ -by- m matrix E from these column vectors. One can check that all qualified vectors fall in the kernel of E . This kernel can be efficiently computed from the Gaussian elimination on E and is a subspace spanned by $m - \text{rank}(E)$ basis vectors. So, the step 1 of SimF only needs to iterate through these basis vectors, and the other qualified vectors must satisfy the constraints as they are in the subspace. As a result, the step 1 of SimF runs in polynomial time due to the Gaussian elimination plus a linear-time pass to assign random or constrained values to $d_{y_i(f)}$'s according to the $m - \text{rank}(E)$ basis vectors.

The step 1 of SimIn only requires one linear-time pass to assign constrained or random values to the active labels so it runs in polynomial time. The linear constraint on the LSBs of these active labels has rank $\text{rank}(E)$ and is satisfiable for the $d_{y_i(f)}$'s assigned in SimF.

Then, we fix z and $\mathcal{A}$. We will use the H-coefficient technique (Section 4.4) with transcript padding to bound the advantage of distinguishing between the real world (i.e., the adaptive experiment that uses the three-halves scheme) and the ideal world (i.e., the adaptive experiment that uses simulator Sim). In this technique, we consider the computationally unbounded non-uniform adversary $\mathcal{A} = \mathcal{A}(z)$ and compute $\varepsilon_1, \varepsilon_2$ as follows.

Transcript padding. In either world, $\mathcal{A}$ will interact with an integrated oracle that acts as the two-round challenger in the adaptive experiment and provides interfaces $\pi^{*\pm 1} \in \{\pi^{\pm 1}, \text{SimP}^{\pm 1}\}$ for forward/backward permutation queries. $\mathcal{A}$ can learn $\pi^*(\alpha) = \beta$ if and only if it sent forward query α to π^* and received response β , or sent backward query β to π^{*-1} and received response α .

To compute $\varepsilon_1, \varepsilon_2$ more easily, we ask the oracle to send more messages to $\mathcal{A}$ and $\mathcal{A}$ to make extra queries (in addition to the supposed q queries) in both two worlds. More specifically,

- Upon receiving x from $\mathcal{A}$, the oracle sends $\tilde{X} := \{X_i\}_{i \in \mathcal{W}(f)}$ instead of $\hat{x} := \{X_i\}_{i \in \mathcal{W}_{\text{in}}(f)}$ to $\mathcal{A}$. In addition to the active input labels in $\hat{x}$, the former also gives the active internal and output labels. In the real world, the oracle can run $\text{TH.DecEval}^{\pi^{\pm 1}(\cdot)}$, which determines other active labels in $\tilde{X}$. In the ideal world, this $\tilde{X}$ can be directly output by SimIn.
- Along with $\tilde{X}$, the oracle sends $\tilde{x} := \{x_i\}_{i \in \mathcal{W}(f)}$ to $\mathcal{A}$, which denote the wire truth values in the evaluation of $f(x)$. Both two oracles “echo” these values, which are self-evident to $\mathcal{A}$, to explicitly include them in transcripts. In the experiment, the real-world oracle uses $x = \{x_i\}_{i \in \mathcal{W}_{\text{in}}(f)}$ in $\text{TH.Encode}^{\pi^{\pm 1}(\cdot)}$, but the ideal-world oracle can only use $f(x) = \{x_i\}_{i \in \mathcal{W}_{\text{out}}(f)}$ in SimIn.
- Along with $\tilde{X}$, the oracle sends $\tilde{u} := \{u_i^j\}_{i \in \mathcal{W}_{\cup}(f), j \in [0, \lceil n_i(f)/2 \rceil - 1]}$ to $\mathcal{A}$. In the real world, the oracle computes $u_i^j := H(X_i, i \parallel j)$ for each $i \in \mathcal{W}_{\cup}(f)$ and $j \in [0, \lceil n_i(f)/2 \rceil - 1]$. In the ideal world, this $\tilde{u}$ is output by SimIn and gives the “hash outputs” fixed by the random tape, which is specified by the oracle to run the programming therein.
- Along with $\tilde{X}$, the oracle sends

$$\tilde{r} := \{r_{s_a s_b}^g\}_{g \in \mathcal{G}_{\text{and}}(f), (a,b) := (\text{in}_0(g), \text{in}_1(g))}$$

to $\mathcal{A}$. In the real world, the oracle computes $r_{s_a s_b}^g \in \mathbb{F}_2^2$ from two uniform bits (which span $R_{\S}^g \leftarrow \mathcal{R}_0$ and match $R_{\S, ij}^g$ for every $i, j \in \mathbb{F}_2$ as per distribution $\mathcal{R}_0$), the truth table t^g (expressed in terms of the truth values x_a, x_b in $\tilde{x}$ and the masked bits $s_a = \text{lsb}(X_a), s_b = \text{lsb}(X_b)$ in $\tilde{X}$), and the two

masked bits s_a and s_b . More specifically, it follows from TH.SampleR that

$$\begin{aligned} \mathbf{r}_{s_a s_b}^g{}^\top &= \mathbf{R}_{s, s_a s_b}'^g \oplus \left(g(\overline{p_a}, p_b) \oplus g(\overline{p_a}, \overline{p_b}) \right) \mathbf{R}_{1, s_a s_b}' \oplus \left(g(p_a, \overline{p_b}) \oplus g(\overline{p_a}, \overline{p_b}) \right) \mathbf{R}_{2, s_a s_b}' \\ &= \mathbf{R}_{s, s_a s_b}'^g \oplus \overline{s_a \oplus x_a} \cdot \mathbf{R}_{1, s_a s_b}' \oplus \overline{s_b \oplus x_b} \cdot \mathbf{R}_{2, s_a s_b}'. \end{aligned} \quad (32)$$

In the ideal world, this $\tilde{\mathbf{r}}$ is also output by SimIn as per the random tape of the oracle.

- Along with $\tilde{\mathbf{X}}$, the oracle sends $\tilde{T} := \{T_i\}_{i \in \mathcal{W}_\cup(f), n_i(f) \text{ is odd}}$ to $\mathcal{A}$. In the real world, the oracle computes $T_i := \text{Half}_1(\text{H}(\mathbf{X}_i \oplus \Delta, i \parallel \lfloor n_i(f)/2 \rfloor)) \in \mathbb{F}_{2^{\lambda/2+1}}$, i.e., the *unused* upper half of the $\lfloor n_i(f)/2 \rfloor$ -th Δ -related hash output of the wire i . In the ideal world, this $\tilde{T}$ is sampled by the oracle at random.
- (Extra queries) Upon receiving $(\tilde{\mathbf{X}}, \tilde{x}, \tilde{\mathbf{u}}, \tilde{\mathbf{r}}, \tilde{T})$ from the oracle, $\mathcal{A}$ will also make a forward permutation query $(\mathbf{0} \parallel \mathbf{X}_i) \oplus (i \parallel j)$ for every $i \in \mathcal{W}_\cup(f)$ and $j \in [0, \lceil n_i(f)/2 \rceil - 1]$, if it has never learned $\pi^*((\mathbf{0} \parallel \mathbf{X}_i) \oplus (i \parallel j)) = \mathbf{y}$ for some $\mathbf{y}$ in its interaction with $\pi^{*\pm 1} \in \{\pi^{\pm 1}, \text{SimP}^{\pm 1}\}$.
- At the end of the experiment (i.e., once all other transcripts are settled), the oracle sends Δ to $\mathcal{A}$. In the real world, the oracle gets this Δ from the output of TH.Garble $^{\pi^{\pm 1}(\cdot)}$. In the ideal world, Δ is dummy and sampled by the oracle at this time (note that Sim does not use Δ).

According to the two oracle constructions, real-world sample space

$$\Omega_{\text{real}} = \mathbb{F}_2^{\lambda-1} \times \mathbb{F}_2^{|\mathcal{W}_{\text{in}}(f)|\lambda} \times \underbrace{\mathbb{F}_2^{2|f|}}_{\substack{\text{random tape to run} \\ \text{TH.SampleR } |f| \text{ times}}} \times \mathcal{S}_{\lambda+2},$$

and ideal-world sample space

$$\begin{aligned} \Omega_{\text{ideal}} &= (\mathbb{F}_2^{3\lambda/2+5})^{|f|} \times \mathbb{F}_2^{\text{rank}(\mathbf{E})} \times \mathbb{F}_2^{(|\mathcal{W}_{\text{in}}(f)| + |\mathcal{Z}(f)|)\lambda - \text{rank}(\mathbf{E})} \times (\mathbb{F}_2^{\lambda/2+3})^{|\mathcal{Z}(f)|} \times (\mathbb{F}_2^{3\lambda/2+3})^{|f| - |\mathcal{Z}(f)|} \\ &\quad \times \underbrace{(\mathbb{F}_2^{\lambda+2})^M}_{\text{for the step 25 of SimIn and } \tilde{T}} \times \underbrace{\{0, 1\}^*}_{\text{random tape for the sampling in SimP}^{\pm 1}(\cdot)} \times \underbrace{\mathbb{F}_2^{\lambda-1}}_{\text{dummy } \Delta}. \end{aligned}$$

Given the oracle constructions, a transcript in the original adaptive experiment will be padded with more literal values. Note that transcript padding will not lower the advantage of $\mathcal{A}$ since $\mathcal{A}$ can discard the padding values at will. With the padding, a transcript is of the form:

$$\tau = (\mathcal{K}_1, (f, \hat{f}), \mathcal{K}_2, (x, (\tilde{\mathbf{X}}, \tilde{x}, \tilde{\mathbf{u}}, \tilde{\mathbf{r}}, \tilde{T})), \mathcal{K}_3, \Delta),$$

where $\mathcal{K}_1$, $\mathcal{K}_2$, and $\mathcal{K}_3$ are the ordered lists of query-response pairs seen in the interleaved interaction with permutation oracles. We do not explicitly consider query direction in these pairs. Given $\pi^{*\pm 1} \in \{\pi^{\pm 1}, \text{SimP}^{\pm 1}\}$, $\mathcal{A}$ is able to learn $\pi^*(\alpha) = \beta$ if and only if there exists $(\alpha, \beta) \in \cup_{\ell=1}^3 \mathcal{K}_\ell$.

Let $q_\ell := |\mathcal{K}_\ell|$ for every $\ell \in \{1, 2, 3\}$ and $q_\Sigma := \sum_{\ell=1}^3 q_\ell$. It follows from the extra queries that $q_\Sigma \leq q + N$. Without loss of generality, we can assume that $\mathcal{A}$ only makes *non-repeating* queries, i.e., it never makes forward query α to π^* or backward query β to π^{*-1} for any learned permutation entry (α, β) .

Bad transcripts. A transcript $\tau \in \mathcal{T}_{\text{bad}}$ if and only if it incurs at least one of the following events:

- bad_1 . There exist distinct $(i, j) \in \mathcal{W}_\cup(f) \times [0, \lceil n_i(f)/2 \rceil - 1]$, $(i', j') \in \mathcal{W}_\cup(f) \times [0, \lceil n_{i'}(f)/2 \rceil - 1]$ such that

$$(\mathbf{0} \parallel \mathbf{X}_i) \oplus (i \parallel j) = (\mathbf{0} \parallel \mathbf{X}_{i'}) \oplus (i' \parallel j') \quad \vee \quad (33)$$

$$\mathbf{u}_i^j \oplus \sigma((\mathbf{0} \parallel \mathbf{X}_i) \oplus (i \parallel j)) = \mathbf{u}_{i'}^{j'} \oplus \sigma((\mathbf{0} \parallel \mathbf{X}_{i'}) \oplus (i' \parallel j')) \quad \vee \quad (34)$$

$$\mathbf{y}_i^j \oplus \mathbf{u}_i^j \oplus \sigma((\mathbf{0} \parallel \mathbf{X}_i) \oplus (i \parallel j)) = \mathbf{y}_{i'}^{j'} \oplus \mathbf{u}_{i'}^{j'} \oplus \sigma((\mathbf{0} \parallel \mathbf{X}_{i'}) \oplus (i' \parallel j')) \quad (35)$$

where $\mathbf{y}_i^j \in \mathbb{F}_{2^{\lambda/2+1}}^2$ will be defined from τ as per (43) and (45).

In this case, $\mathcal{A}$ is able to check the consistency between the value of $\mathbf{y}_i^j \oplus \mathbf{y}_{i'}^{j'}$ and that of Δ at the end of experiment *without* sending any needed queries, which are computed from Δ , to a random permutation or its inverse. In the real world, the consistency certainly holds. However, the ideal-world garbled rows and Δ are independently sampled, leading to the consistency only with negligible probability. So, $\mathcal{A}$ has non-negligible advantage to distinguish the two worlds and the statistical distance, as an upper bound, also blows up.

More specifically, the real world is as follows in this case. The pre-image collision (33) leads to the syntactically same XOR of two hash masks in $\mathbf{y}_i^j, \mathbf{y}_{i'}^{j'}$, which can be XORed to cancel all hash masks to check the consistency with Δ without further queries. Furthermore, the image collision (34) also implies the pre-image collision (33) as π is permutation. The collision (35) results in the image collision $\pi((\mathbf{0} \parallel \mathbf{X}_i \oplus \Delta) \oplus (i \parallel j)) = \pi((\mathbf{0} \parallel \mathbf{X}_{i'} \oplus \Delta) \oplus (i' \parallel j'))$ for some distinct (i, j) and (i', j') . Since π is permutation, this collision implies the pre-image collision (33), which can be used to see the consistency. In contrast, the above cancelling of hash masks cannot give this consistency except with negligible probability in the ideal world.

- bad_2 . There exists $((\alpha, \beta), i, j) \in \cup_{\ell=1}^3 \mathcal{K}_\ell \times \mathcal{W}_\cup(f) \times [0, \lceil n_i(f)/2 \rceil - 1]$ such that

$$\alpha = (\mathbf{0} \parallel \mathbf{X}_i \oplus \Delta) \oplus (i \parallel j) \quad \vee \quad (36)$$

$$\beta = \sigma(\mathbf{0} \parallel \Delta) \oplus \mathbf{y}_i^j \oplus \mathbf{u}_i^j \oplus \sigma((\mathbf{0} \parallel \mathbf{X}_i) \oplus (i \parallel j)) \quad (37)$$

where $\mathbf{y}_i^j \in \mathbb{F}_{2^{\lambda/2+1}}^2$ will be defined from τ according to (43) and (45).

In this case, $\mathcal{A}$ essentially makes it to guess Δ before receiving this value. It allows $\mathcal{A}$ to distinguish the real world, where every $\mathbf{y}_i^j$ is consistent with Δ , and the ideal world with a dummy Δ . So, the statistical distance blows up.

- bad_3 . There exists $((\alpha, \beta), i, j) \in \cup_{\ell=1}^2 \mathcal{K}_\ell \times \mathcal{W}_\cup(f) \times [0, \lceil n_i(f)/2 \rceil - 1]$ such that

$$\alpha = (\mathbf{0} \parallel \mathbf{X}_i) \oplus (i \parallel j) \quad \vee \quad (38)$$

$$\beta = \mathbf{u}_i^j \oplus \sigma((\mathbf{0} \parallel \mathbf{X}_i) \oplus (i \parallel j)) \quad (39)$$

In this case, $\mathcal{A}$ can make forward/backward queries w.r.t. some active labels before receiving active input labels and computing other active ones.

This case is necessary to ensure successful programming in the ideal world as the values on the right hand should not be queried before the programming (otherwise it fails due to pre-image or image collision). If the programming fails in the ideal world, the two worlds can be distinguishable as the decoding consistency in the ideal world does not always hold as in the real world.

Bounding $1 - \varepsilon_2$. Without loss of generality, we can consider some fixed good transcript τ such that $\Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [Y(\omega) = \tau] > 0$. Using Lemma 1, we turn to analyze the sampled oracle's compatibility (Section 4.4) with such a transcript, instead of the interaction between $\mathcal{A}$ and the sampled oracle.

Note that there is a computationally unbounded non-uniform adversary $\mathcal{A}'$ such that, for every oracle ω , it sends the queries in τ in order in its interaction with ω (e.g., $\mathcal{A}'$ has auxiliary input τ and sends its ordered queries). Fix $\mathcal{A}'$ in the following compatibility analysis so that any real-world or ideal-world oracle will receive the queries in τ in order. For a response c recorded in a fixed τ , let $\omega \vdash c$ denote the event that, fixing the ordered queries as per τ , oracle ω produces c given the corresponding query. Let $\mathcal{K}^R$ denote the order-preserving list of the responses in an ordered list $\mathcal{K}$ of permutation query-response pairs.

In the real world, an oracle

$$\omega = (\Delta, \{(p_i, \mathbf{W}_i)\}_{i \in \mathcal{W}_{\text{in}}(f)}, \{(r_L^g, r_R^g)\}_{g \in \mathcal{G}_{\text{and}}(f)}, \pi) \in \Omega_{\text{real}}$$

in the three-halves. So,

$$\Pr_{\omega \leftarrow \Omega_{\text{real}}} [\omega \in \text{comp}_{\text{real}}(\tau)] = \Pr_{\omega \leftarrow \Omega_{\text{real}}} [\omega \vdash (\mathcal{K}_1^{\text{R}}, \hat{f}, \mathcal{K}_2^{\text{R}}, \tilde{\mathbf{X}}, \tilde{x}, \tilde{\mathbf{u}}, \tilde{\mathbf{r}}, \tilde{T}, \mathcal{K}_3^{\text{R}}, \Delta)].$$

To begin with, every real-world ω certainly produces f' in $\hat{f}$ and $\tilde{x}$ in τ , being consistent with $\Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [Y(\omega) = \tau] > 0$. This non-zero probability implies that $(f', \tilde{x})$ in τ is honestly and deterministically computed from the fixed queries (f, x) . Otherwise, no ideal-world oracle, which computes $(f', \tilde{x})$ from the same deterministic procedure, can produce this transcript, contradicting the non-zero probability. As every real-world ω will follow the same deterministic procedure, it certainly produces the two values.

Meanwhile, a real-world oracle ω should have the same literal value of Δ as its counterpart in τ . Then, the real-world sampling $\mathbf{R}_{\S}^g \leftarrow \mathcal{R}_0$ (in Appendix F) and $\mathbf{r}^g = [\mathbf{r}_{00}^g \ \mathbf{r}_{01}^g \ \mathbf{r}_{10}^g \ \mathbf{r}_{11}^g]^{\top} \in (\mathbb{F}_2^2)^4$ ensure that, for every $i, j \in \mathbb{F}_2$ and every $g \in \mathcal{G}_{\text{and}}(f)$ with $(a, b) := (\text{in}_0(g), \text{in}_1(g))$,

$$\begin{aligned} \mathbf{r}_{ij}^{g \top} &= [r_{ijL}^g \ r_{ijR}^g] \\ &= \mathbf{R}_{\S, ij}^g \oplus \overline{s_a \oplus x_a} \cdot \mathbf{R}'_{1, ij} \oplus \overline{s_b \oplus x_b} \cdot \mathbf{R}'_{2, ij} \\ &= [r_L^g \ r_R^g] \oplus \overline{s_a \oplus x_a} \cdot \mathbf{R}'_{1, ij} \oplus \overline{s_b \oplus x_b} \cdot \mathbf{R}'_{2, ij}. \end{aligned} \quad (40)$$

In particular, (40) holds for $i = s_a$ and $j = s_b$ (i.e., the real-world compatibility between ω and $\tilde{\mathbf{r}}$ in (32) for this g) with probability 2^{-2} , which comes from two uniform coins $(r_L^g, r_R^g) \in \mathbb{F}_2^2$ in ω and independent of the literal values of x_a, x_b, s_a , and s_b . Conditioned on the compatibility so far, the probability

$$\begin{aligned} &\Pr_{\omega \leftarrow \Omega_{\text{real}}} [\omega \in \text{comp}_{\text{real}}(\tau)] \\ &= \Pr_{\omega \leftarrow \Omega_{\text{real}}} [\omega \vdash (\mathcal{K}_1^{\text{R}}, F, d, \mathcal{K}_2^{\text{R}}, \tilde{\mathbf{X}}, \tilde{\mathbf{u}}, \tilde{T}, \mathcal{K}_3^{\text{R}}) \mid \omega \vdash (f', \tilde{x}) \wedge \omega \vdash (\tilde{\mathbf{r}}, \Delta)] \\ &\quad \cdot \Pr_{\omega \leftarrow \Omega_{\text{real}}} [\omega \vdash (f', \tilde{x}) \wedge \omega \vdash (\tilde{\mathbf{r}}, \Delta)] \\ &= \Pr_{\omega \leftarrow \Omega_{\text{real}}} [\omega \vdash (\mathcal{K}_1^{\text{R}}, F, d, \mathcal{K}_2^{\text{R}}, \tilde{\mathbf{X}}, \tilde{\mathbf{u}}, \tilde{T}, \mathcal{K}_3^{\text{R}}) \mid \omega \vdash (f', \tilde{x}) \wedge \omega \vdash (\tilde{\mathbf{r}}, \Delta)] \\ &\quad \cdot \Pr_{\omega \leftarrow \Omega_{\text{real}}} [\omega \vdash (f', \tilde{x})] \cdot \Pr_{\omega \leftarrow \Omega_{\text{real}}} [\omega \vdash \tilde{\mathbf{r}}] \cdot \Pr_{\omega \leftarrow \Omega_{\text{real}}} [\omega \vdash \Delta] \\ &= \Pr_{\omega \leftarrow \Omega_{\text{real}}} [\omega \vdash (\mathcal{K}_1^{\text{R}}, F, d, \mathcal{K}_2^{\text{R}}, \tilde{\mathbf{X}}, \tilde{\mathbf{u}}, \tilde{T}, \mathcal{K}_3^{\text{R}}) \mid \omega \vdash (f', \tilde{x}) \wedge \omega \vdash (\tilde{\mathbf{r}}, \Delta)] \cdot \frac{1}{2^{2|f|+(\lambda-1)}}. \end{aligned}$$

Conditioned on the compatibility with $(f', \tilde{x}, \tilde{\mathbf{r}}, \Delta)$, a real-world ω should be compatible with $(\cup_{\ell=1}^3 \mathcal{K}_{\ell}^{\text{R}}, F, \tilde{\mathbf{u}}, \tilde{T})$ and some active labels in $\tilde{\mathbf{X}}$ such that

- (i) $\pi^{\pm 1}$ respects the query-response pairs fixed in $\cup_{\ell=1}^3 \mathcal{K}_{\ell}^{\text{R}}$.
- (ii) For each $i \in \mathcal{W}_{\text{in}}(f)$, it holds that $\mathbf{X}_i = \mathbf{W}_i \oplus (p_i \oplus x_i) \Delta$.
- (iii) For each $i \in \mathcal{W}_{\cup}(f)$ and $j \in [0, \lceil n_i(f)/2 \rceil - 1]$,

$$\mathbf{H}(\mathbf{X}_i, i \parallel j) := \pi((\mathbf{0} \parallel \mathbf{X}_i) \oplus (i \parallel j)) \oplus \sigma((\mathbf{0} \parallel \mathbf{X}_i) \oplus (i \parallel j)) = \mathbf{u}_i^j = \begin{bmatrix} \text{Half}_1(\mathbf{u}_i^j) \\ \text{Half}_0(\mathbf{u}_i^j) \end{bmatrix}. \quad (41)$$

- (iv) For each $g \in \mathcal{G}_{\text{and}}(f)$ with

$$(a, b, \Gamma, c) := (\text{in}_0(g), \text{in}_1(g), \text{in}_2(g), \text{out}(g))$$

and those non-repeating counters $(\chi_a \parallel \rho_a)$, $(\chi_b \parallel \rho_b)$, and $(\chi_\Gamma \parallel \rho_\Gamma)$ as per a topology order of f , it holds that

$$\begin{aligned}
& \left(\mathbf{r}_{s_a s_b}^g \parallel \mathbf{X}_c \oplus \mathbf{R}_{s_a s_b}^g \begin{bmatrix} \mathbf{X}_a \\ \mathbf{X}_b \end{bmatrix} \right) \\
&= \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix} \begin{bmatrix} \text{Half}_{\rho_a}(\text{H}(\mathbf{X}_a, a \parallel \chi_a)) \\ \text{Half}_{\rho_b}(\text{H}(\mathbf{X}_b, b \parallel \chi_b)) \\ \text{Half}_{\rho_\Gamma}(\text{H}(\mathbf{X}_\Gamma, \Gamma \parallel \chi_\Gamma)) \end{bmatrix} \oplus \mathbf{V}_{s_a s_b} \left(\mathbf{z}^g \parallel \begin{bmatrix} \mathbf{0} \\ \mathbf{g}^g \end{bmatrix} \right), \\
& \left(\mathbf{z}_{\text{bot}}^g \parallel \mathbf{g}^g \right) = \mathbf{V}_{\text{bot}}^+ \left(\mathbf{M} \mathbf{h}^g \oplus \left(\mathbf{r}^g \parallel (\hat{\mathbf{R}}^g \oplus ([\mathbf{0}_{4 \times 2} \ t^g] \otimes \mathbf{I}_2)) \begin{bmatrix} \mathbf{W}_a \\ \mathbf{W}_b \\ \Delta \end{bmatrix} \right) \right) \\
&= \mathbf{V}_{\text{bot}}^+ \mathbf{M} \begin{bmatrix} \overline{s_a} & s_a \\ s_a & \overline{s_a} \\ \overline{s_b} & s_b \\ s_b & \overline{s_b} \\ \overline{s_a \oplus s_b} & s_a \oplus s_b \\ s_a \oplus s_b & \overline{s_a \oplus s_b} \end{bmatrix} \begin{bmatrix} \text{Half}_{\rho_a}(\text{H}(\mathbf{X}_a, a \parallel \chi_a)) \\ \text{Half}_{\rho_a}(\text{H}(\mathbf{X}_a \oplus \Delta, a \parallel \chi_a)) \\ \text{Half}_{\rho_b}(\text{H}(\mathbf{X}_b, b \parallel \chi_b)) \\ \text{Half}_{\rho_b}(\text{H}(\mathbf{X}_b \oplus \Delta, b \parallel \chi_b)) \\ \text{Half}_{\rho_\Gamma}(\text{H}(\mathbf{X}_\Gamma, \Gamma \parallel \chi_\Gamma)) \\ \text{Half}_{\rho_\Gamma}(\text{H}(\mathbf{X}_\Gamma \oplus \Delta, \Gamma \parallel \chi_\Gamma)) \end{bmatrix} \\
&\oplus \underbrace{\left(\begin{bmatrix} \overline{s_a \oplus x_a} \\ \overline{s_b \oplus x_b} \\ s_a \oplus x_a \oplus s_b \oplus x_b \end{bmatrix} \parallel \mathbf{V}_{\text{bot}}^+ (\hat{\mathbf{R}}^g \oplus ([\mathbf{0}_{4 \times 2} \ t^g] \otimes \mathbf{I}_2)) \begin{bmatrix} \mathbf{X}_a \oplus s_a \Delta \\ \mathbf{X}_b \oplus s_b \Delta \\ \Delta \end{bmatrix} \right)}_{\begin{bmatrix} \text{Half}_{\rho_a}(\mathbf{e}_a^{\chi_a}) & \text{Half}_{\rho_b}(\mathbf{e}_b^{\chi_b}) & \text{Half}_{\rho_\Gamma}(\mathbf{e}_\Gamma^{\chi_\Gamma}) \end{bmatrix}^\top \in \mathbb{F}_{2^{\lambda/2+1}}^3} \\
&\Leftrightarrow \underbrace{\left(\mathbf{z}_{\text{bot}}^g \parallel \mathbf{g}^g \right) \oplus \begin{bmatrix} \text{Half}_{\rho_a}(\mathbf{e}_a^{\chi_a}) & \text{Half}_{\rho_b}(\mathbf{e}_b^{\chi_b}) & \text{Half}_{\rho_\Gamma}(\mathbf{e}_\Gamma^{\chi_\Gamma}) \end{bmatrix}^\top}_{\begin{bmatrix} \text{Half}_{\rho_a}(\mathbf{y}_a^{\chi_a}) & \text{Half}_{\rho_b}(\mathbf{y}_b^{\chi_b}) & \text{Half}_{\rho_\Gamma}(\mathbf{y}_\Gamma^{\chi_\Gamma}) \end{bmatrix}^\top \in \mathbb{F}_{2^{\lambda/2+1}}^3} \\
&= \begin{bmatrix} \text{Half}_{\rho_a}(\text{H}(\mathbf{X}_a, a \parallel \chi_a)) \\ \text{Half}_{\rho_b}(\text{H}(\mathbf{X}_b, b \parallel \chi_b)) \\ \text{Half}_{\rho_\Gamma}(\text{H}(\mathbf{X}_\Gamma, \Gamma \parallel \chi_\Gamma)) \end{bmatrix} \oplus \begin{bmatrix} \text{Half}_{\rho_a}(\text{H}(\mathbf{X}_a \oplus \Delta, a \parallel \chi_a)) \\ \text{Half}_{\rho_b}(\text{H}(\mathbf{X}_b \oplus \Delta, b \parallel \chi_b)) \\ \text{Half}_{\rho_\Gamma}(\text{H}(\mathbf{X}_\Gamma \oplus \Delta, \Gamma \parallel \chi_\Gamma)) \end{bmatrix},
\end{aligned} \tag{42}$$

where the bits $x_a, x_b, s_a = \text{lsb}(\mathbf{X}_a), s_b = \text{lsb}(\mathbf{X}_b)$ are given in τ , and in (43),

$$\begin{aligned}
\mathbf{V}^+ &= [\mathbf{V}_{\text{top}}^+ \ \mathbf{V}_{\text{bot}}^+]^\top \in (\mathbb{F}_2^{1 \times 8})^2 \times (\mathbb{F}_2^{1 \times 8})^3, \\
\forall g \in \mathcal{G}_{\text{and}}(f) : \mathbf{z}^g &= [\mathbf{z}_{\text{top}}^g \ \mathbf{z}_{\text{bot}}^g]^\top \in \mathbb{F}_2^2 \times \mathbb{F}_2^3, \\
\forall g \in \mathcal{G}_{\text{and}}(f) : \mathbf{t}^g &= \begin{bmatrix} g(s_a \oplus x_a, s_b \oplus x_b) \\ g(s_a \oplus x_a, \overline{s_b \oplus x_b}) \\ g(\overline{s_a \oplus x_a}, s_b \oplus x_b) \\ g(s_a \oplus x_a, \overline{s_b \oplus x_b}) \end{bmatrix},
\end{aligned}$$

and for each $g \in \mathcal{G}_{\text{and}}(f)$, $\hat{\mathbf{R}}^g \in \mathbb{F}_2^{8 \times 6}$ is fixed by running TH.SampleR with random coins (r_L^g, r_R^g) in ω , which have been compatible (according to the condition) with $\tilde{\mathbf{r}}$ as per the (40) for this g and $(i, j) = (s_a, s_b)$.

(v) For each $i \in \{i \in \mathcal{W}_\cup(f) \mid n_i(f) \text{ is odd}\}$, it holds that

$$\text{Half}_1(\text{H}(\mathbf{X}_i \oplus \Delta, i \parallel \lfloor n_i(f)/2 \rfloor)) = T_i. \tag{44}$$

Conditioned on the compatibility so far, every real-world oracle ω is always compatible with the leftover values in τ , i.e., decoding information d and other active labels in $\tilde{\mathbf{X}}$, which are deterministically computed from XOR combination. The reason is that, for τ ensuring $\Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [Y(\omega) = \tau] > 0$, these values should be honestly determined by the conditioned values as in the real world. Otherwise, this probability will be zero for an ideal-world oracle, which obtains them from a consistent deterministic computation as per the conditioned values. As every real-world oracle ω honestly follows the

real-world computation, this “leftover” compatibility must hold conditioned on the previous compatibility.

It remains to compute the conditional probabilities for (i) to (v). Consider (iii), (iv), and (v). We claim that every good τ with $\Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [Y(\omega) = \tau] > 0$ already implies (41) and (42). To see this, we use that condition $\neg \text{bad}_3$ for good transcripts and the extra queries ensure that $\mathcal{K}_3$ fixes the pairs of permutation pre-images and images for hash values

$$\{H(\mathbf{X}_i, i \parallel j)\}_{i \in \mathcal{W}_{\cup}(f), j \in [0, \lceil n_i(f)/2 \rceil - 1]}.$$

These values are consistent with $\tilde{\mathbf{u}}$ fixed in τ as per (41). Otherwise, $\Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [Y(\omega) = \tau] > 0$ cannot be satisfied by τ since $\neg \text{bad}_1 \wedge \neg \text{bad}_3$ for every good transcript implies successful programming in the ideal world so that $H(\mathbf{X}_i, i \parallel j) = \mathbf{u}_i^j$. As a corollary, (42) holds for every good transcript due to this consistency and the step 15 and 18 of SimIn.

Consider (43) and (44), the leftover parts of (iii), (iv), and (v). If we define from the fixed τ that

$$\text{Half}_1(\mathbf{y}_i^{\lceil n_i(f)/2 \rceil}) := \text{Half}_1(\mathbf{u}_i^{\lceil n_i(f)/2 \rceil}) \oplus T_i \in \mathbb{F}_{2^{\lambda/2+1}} \quad (45)$$

for each $i \in \{i \in \mathcal{W}_{\cup}(f) \mid n_i(f) \text{ is odd}\}$, then we can unify (43) and (44) as:

$$\mathcal{V} := \left\{ \begin{array}{l} i \in \mathcal{W}_{\cup}(f), j \in [0, \lceil n_i(f)/2 \rceil - 1] : \\ \mathbf{y}_i^j = H(\mathbf{X}_i, i \parallel j) \oplus H(\mathbf{X}_i \oplus \Delta, i \parallel j) \\ \quad = \underbrace{\pi((\mathbf{0} \parallel \mathbf{X}_i) \oplus (i \parallel j))}_{P_{i \parallel j, 0}} \oplus \underbrace{\pi((\mathbf{0} \parallel \mathbf{X}_i \oplus \Delta) \oplus (i \parallel j))}_{P_{i \parallel j, 1}} \oplus \sigma(\mathbf{0} \parallel \Delta) \end{array} \right\}$$

since (43) and (44) iterate through both halves of all well-defined $\mathbf{y}_i^j \in \mathbb{F}_{2^{\lambda/2+1}}^2$ for $i \in \mathcal{W}_{\cup}(f)$ and $j \in [0, \lceil n_i(f)/2 \rceil - 1]$. Since τ is a good transcript, there are exactly $2N$ pairwise distinct permutation pre-images on the right hand (otherwise, there will be a pair of permutation pre-images leading to (33) in bad_1 or a permutation pre-image leading to (36) in bad_2 given the extra queries). Due to this pairwise distinctness, $\mathcal{V}$ has $2N$ syntactically different variables $\mathcal{P} := \{P_{i \parallel j, 0}, P_{i \parallel j, 1}\}_{i \in \mathcal{W}_{\cup}(f), j \in [0, \lceil n_i(f)/2 \rceil - 1]}$. They can fix the same number of the entries of permutation π in a real-world ω *if and only if their literal values fixed by τ are also pairwise distinct*. We note that every good transcript τ does fix *exact one* such assignment of these variables for the following reasons:

- (41) already holds for τ , i.e., for $i \in \mathcal{W}_{\cup}(f)$ and $j \in [0, \lceil n_i(f)/2 \rceil - 1]$,

$$P_{i \parallel j, 0} := \pi((\mathbf{0} \parallel \mathbf{X}_i) \oplus (i \parallel j)) = \mathbf{u}_i^j \oplus \sigma((\mathbf{0} \parallel \mathbf{X}_i) \oplus (i \parallel j)). \quad (46)$$

The literal values of $\{P_{i \parallel j, 0}\}_{i \in \mathcal{W}_{\cup}(f), j \in [0, \lceil n_i(f)/2 \rceil - 1]}$ are immediate from the responses in $\mathcal{K}_3^R$ given the extra queries and will be pairwise distinct due to the impossible (34) from $\neg \text{bad}_1$.

- For $i \in \mathcal{W}_{\cup}(f)$ and $j \in [0, \lceil n_i(f)/2 \rceil - 1]$, the literal value of $P_{i \parallel j, 1}$ is fixed by τ according to $\mathcal{V}$ and (46):

$$P_{i \parallel j, 1} := \pi((\mathbf{0} \parallel \mathbf{X}_i \oplus \Delta) \oplus (i \parallel j)) = \sigma(\mathbf{0} \parallel \Delta) \oplus \mathbf{y}_i^j \oplus \mathbf{u}_i^j \oplus \sigma((\mathbf{0} \parallel \mathbf{X}_i) \oplus (i \parallel j)). \quad (47)$$

The literal values of $\{P_{i \parallel j, 1}\}_{i \in \mathcal{W}_{\cup}(f), j \in [0, \lceil n_i(f)/2 \rceil - 1]}$ will be pairwise distinct according to the impossible (35) from $\neg \text{bad}_1$.

- The goodness of τ also ensures that there do not exist

$$\begin{aligned} P' &\in \{P_{i \parallel j, 0}\}_{i \in \mathcal{W}_{\cup}(f), j \in [0, \lceil n_i(f)/2 \rceil - 1]}, \\ P'_{\Delta} &\in \{P_{i \parallel j, 1}\}_{i \in \mathcal{W}_{\cup}(f), j \in [0, \lceil n_i(f)/2 \rceil - 1]} \end{aligned}$$

such that $P' = P'_\Delta$. Otherwise, this equality and (47) ensure that there exist $(i, j) \in \mathcal{W}_\cup(f) \times [0, \lceil n_i(f)/2 \rceil - 1]$ and $(i', j') \in \mathcal{W}_\cup(f) \times [0, \lceil n_{i'}(f)/2 \rceil - 1]$ such that

$$\begin{aligned} \pi((\mathbf{0} \parallel \mathbf{X}_i) \oplus (i \parallel j)) &= \pi((\mathbf{0} \parallel \mathbf{X}_{i'} \oplus \Delta) \oplus (i' \parallel j')) \\ &= \sigma(\mathbf{0} \parallel \Delta) \oplus \mathbf{y}_{i'}^{j'} \oplus \mathbf{u}_{i'}^{j'} \oplus \sigma((\mathbf{0} \parallel \mathbf{X}_{i'}) \oplus (i' \parallel j')). \end{aligned} \quad (48)$$

We have $((\mathbf{0} \parallel \mathbf{X}_i) \oplus (i \parallel j), \pi((\mathbf{0} \parallel \mathbf{X}_i) \oplus (i \parallel j))) \in \mathcal{K}_3$ according to $\neg\text{bad}_3$ and the extra queries. So, (48) contradicts the impossible (37) from $\neg\text{bad}_2$.

Putting these cases together, we can see that τ yields a value assignment of $\mathcal{P}$, and this assignment fixes exact $2N$ entries of real-world permutation π .

Based on the condition $\neg\text{bad}_1 \wedge \neg\text{bad}_3$ of good transcript τ , N extra queries are non-repeating and the number of non-repeating queries is $q + N = q_\Sigma$. As a result, N responses in $\cup_{\ell=1}^3 \mathcal{K}_\ell^R$ for the non-repeating extra queries are fixed by the values in $\mathcal{P}$ while the other $q_\Sigma - N = q$ responses are fixed by real-world π (conditioned on the values in $\mathcal{P}$). Based on (i), (ii), (iii), (iv), and (v) together with the “leftover” compatibility, we have in the real world that

$$\begin{aligned} &\Pr_{\omega \leftarrow \Omega_{\text{real}}} [\omega \vdash (\mathcal{K}_1^R, F, d, \mathcal{K}_2^R, \tilde{\mathbf{X}}, \tilde{\mathbf{u}}, \tilde{T}, \mathcal{K}_3^R) \mid \omega \vdash (f', \tilde{x}) \wedge \omega \vdash (\tilde{\mathbf{r}}, \Delta)] \\ &= \frac{1}{(2^\lambda)^{|\mathcal{W}_{\text{in}}(f)|}} \cdot \frac{(2^{\lambda+2} - 2N - (q_\Sigma - N))!}{(2^{\lambda+2})!} \\ &= \frac{1}{(2^\lambda)^{|\mathcal{W}_{\text{in}}(f)|}} \cdot \frac{1}{(2^{\lambda+2})_{q+2N}} \\ &\Rightarrow \Pr_{\omega \leftarrow \Omega_{\text{real}}} [\omega \in \text{comp}_{\text{real}}(\tau)] = \frac{1}{(2^\lambda)^{|\mathcal{W}_{\text{in}}(f)|}} \cdot \frac{1}{(2^{\lambda+2})_{q+2N}} \cdot \frac{1}{2^{2|f|+(\lambda-1)}}. \end{aligned}$$

In the ideal world, we can use a similar argument to show

$$\begin{aligned} &\Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\omega \in \text{comp}_{\text{ideal}}(\tau)] \\ &= \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\omega \vdash (\mathcal{K}_1^R, \mathcal{K}_2^R, \mathcal{K}_3^R) \mid \omega \vdash (\hat{f}, \tilde{\mathbf{X}}, \tilde{x}, \tilde{\mathbf{u}}, \tilde{\mathbf{r}}, \tilde{T}, \Delta)] \\ &\quad \cdot \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\omega \vdash (\hat{f}, \tilde{\mathbf{X}}, \tilde{x}, \tilde{\mathbf{u}}, \tilde{\mathbf{r}}, \tilde{T}, \Delta)] \\ &= \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\omega \vdash (\mathcal{K}_1^R, \mathcal{K}_2^R, \mathcal{K}_3^R) \mid \omega \vdash (\hat{f}, \tilde{\mathbf{X}}, \tilde{x}, \tilde{\mathbf{u}}, \tilde{\mathbf{r}}, \tilde{T}, \Delta)] \\ &\quad \cdot \frac{1}{2^{|\mathcal{W}_{\text{in}}(f)|\lambda + (3\lambda+8)|f| + (\lambda+2)M + (\lambda-1)}}. \end{aligned}$$

According to the condition $\neg\text{bad}_1 \wedge \neg\text{bad}_3$ and the N extra queries, there are exact $q + N = q_\Sigma$ non-repeating queries. Here, N responses in $\cup_{\ell=1}^3 \mathcal{K}_\ell^R$ for the non-repeating extra queries are fixed by the conditioned values while the other responses are fixed by $\text{SimP}^{\pm 1}$ for other $q_\Sigma - N = q$ queries.

Let $\mathcal{Q}_{i-1}$ denote the list $\mathcal{Q}$ (which is maintained in internal state st_{sim}) when it includes $i - 1 \in [0, q_\Sigma - 1]$ pairs (note that $\mathcal{Q}$ finally includes q_Σ pairs given the q_Σ non-repeating queries), and $\mathcal{N} \subseteq [1, q_\Sigma]$ denote the index set of these q queries in q_Σ non-repeating queries to $\text{SimP}^{\pm 1}(\cdot)$ such that

$|\mathcal{N}| = q$. We have

$$\begin{aligned}
& \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\omega \vdash (\mathcal{K}_1^R, \mathcal{K}_2^R, \mathcal{K}_3^R) \mid \omega \vdash (\hat{f}, \tilde{\mathbf{X}}, \tilde{x}, \tilde{\mathbf{u}}, \tilde{r}, \tilde{T}, \boldsymbol{\Delta})] \\
&= \prod_{i \in \mathcal{N}} \frac{1}{2^{\lambda+2} - |\mathcal{Q}_{i-1}|} = \prod_{i \in \mathcal{N}} \frac{1}{2^{\lambda+2} - (i-1)} \\
&\leq \frac{1}{2^{\lambda+2} - N} \times \frac{1}{2^{\lambda+2} - (N+1)} \times \cdots \times \frac{1}{2^{\lambda+2} - (q_\Sigma - 1)} \\
&= \frac{1}{(2^{\lambda+2} - N)_q}, \\
&\Rightarrow \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\omega \in \text{comp}_{\text{ideal}}(\tau)] \\
&\leq \frac{1}{(2^\lambda)^{|\mathcal{W}_{\text{in}}(f)|}} \cdot \frac{1}{(2^{\lambda+2} - N)_q} \cdot \frac{1}{(2^{\lambda+2})^{3|f|+M}} \cdot \frac{1}{2^{2|f|+(\lambda-1)}}.
\end{aligned}$$

As iterating an AND gate increases three counters in either world, we have

$$\begin{aligned}
3|f| &= \sum_{i \in \mathcal{W}_{\cup}(f)} n_i(f) \\
&= \sum_{\substack{i \in \mathcal{W}_{\cup}(f), \\ n_i(f) \text{ is even}}} 2 \cdot \left\lceil \frac{n_i(f)}{2} \right\rceil + \sum_{\substack{i \in \mathcal{W}_{\cup}(f), \\ n_i(f) \text{ is odd}}} 2 \cdot \left(\left\lceil \frac{n_i(f)}{2} \right\rceil - \frac{1}{2} \right) = 2N - M.
\end{aligned} \tag{49}$$

So, we can have $\varepsilon_2 = 0$ since, for every $N \geq 0$ and $q \geq 0$,

$$\begin{aligned}
& \frac{\Pr_{\omega \leftarrow \Omega_{\text{real}}} [\omega \in \text{comp}_{\text{real}}(\tau)]}{\Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\omega \in \text{comp}_{\text{ideal}}(\tau)]} \\
&\geq \frac{(2^{\lambda+2} - N)_q \cdot (2^{\lambda+2})^{2N}}{(2^{\lambda+2})_{q+2N}} \\
&\geq \frac{(2^{\lambda+2} - N)_q \cdot (2^{\lambda+2})_N \cdot (2^{\lambda+2})^N}{(2^{\lambda+2})_{q+2N}} \\
&= \frac{(2^{\lambda+2})_{q+N} \cdot (2^{\lambda+2})^N}{(2^{\lambda+2})_{q+2N}} = \frac{(2^{\lambda+2})^N}{(2^{\lambda+2} - (q+N))_N} \geq 1.
\end{aligned}$$

Bounding ε_1 . We bound the probabilities of the bad events in the *ideal world*. First, consider bad_1 . Note that each active label $\mathbf{X}_i$ can be written as the XOR of (i) some active circuit input labels, and/or (ii) some active output labels of the *precedent* AND gates, i.e., for $\mathcal{I}_i \ominus \mathcal{J}_i \neq \emptyset$,

$$\begin{aligned}
\mathbf{X}_i &= \left(\bigoplus_{w \in \mathcal{I}_i \subseteq \mathcal{W}_{\text{in}}(f)} \mathbf{X}_w \right) \oplus \left(\bigoplus_{w \in \mathcal{J}_i \subseteq \mathcal{W}_{\text{and}}(f)} \mathbf{X}_w \right) \\
&= \bigoplus_{w \in \mathcal{I}_i \ominus \mathcal{J}_i} \mathbf{X}_w \in \mathbb{F}_{2^{\lambda/2}}^2.
\end{aligned} \tag{50}$$

For (33), we can use (50) to rewrite it as

$$\left(\mathbf{0}_{2 \times 1} \parallel \left(\bigoplus_{w \in (\mathcal{I}_i \ominus \mathcal{I}_{i'}) \ominus (\mathcal{J}_i \ominus \mathcal{J}_{i'})} \mathbf{X}_w \right) \right) = (i \parallel j) \oplus (i' \parallel j') \in \mathbb{F}_{2^{\lambda/2+1}}^2.$$

According to SimIn, each $\mathbf{X}_w$, which is sampled in the step 1 or computed in the step 19, has at least $\lambda - 1$ random non-LSBs. Therefore, the equality holds with probability at most $2^{-(\lambda-1)}$ for some fixed distinct (i, j) and (i', j') , and, if (i) $\mathcal{I}_i = \mathcal{I}_{i'}$ and $\mathcal{J}_i = \mathcal{J}_{i'}$ or (ii) the right-hand XOR does not give two leading zero bits, this probability is zero for the distinct (i, j) and (i', j') . For (34), the worst case is that both halves of $\mathbf{u}_i^j \oplus \mathbf{u}_{i'}^{j'} \in \mathbb{F}_{2^{\lambda/2+1}}^2$ respectively serve (in the step 14 of SimIn) as the lower-half masks of two AND gates both with output wires in $\mathcal{Z}(f)$. In this case, each half of this XOR will have at least

$1 + (\lambda/2 - 1) = \lambda/2$ random non-LSBs using the lower-half randomness of $(\mathbf{r}_{s_a s_b}^g \parallel \mathbf{X}_c) \in \mathbb{F}_{2^{\lambda/2+1}}^2$ in each of the two AND gates. Note that such $2 \cdot \lambda/2 = \lambda$ bits are independent of other (previously fixed) active labels, including $\mathbf{X}_i$ and $\mathbf{X}_{i'}$. Thus, the equality holds with probability at most $2^{-\lambda} < 2^{-(\lambda-1)}$ for some fixed (i, j) and (i', j') .

Similarly, the worst case for (35) is that both halves of $\mathbf{u}_i^j \oplus \mathbf{u}_{i'}^{j'} \in \mathbb{F}_{2^{\lambda/2+1}}^2$ are respectively the lower-half masks of two AND gates both with output wires in $\mathcal{Z}(f)$. Otherwise, at least one half of $(\mathbf{y}_i^j \oplus \mathbf{u}_i^j) \oplus (\mathbf{y}_{i'}^{j'} \oplus \mathbf{u}_{i'}^{j'})$ is uniform for:

- (i) The $\mathbf{u}_i^j \oplus \mathbf{u}_{i'}^{j'}$ masked with the uniform upper half of some $(\mathbf{r}_{s_a s_b}^g \parallel \mathbf{X}_c)$, which cannot be cancelled by $\mathbf{y}_i^j$ or $\mathbf{y}_{i'}^{j'}$ defined as per (43) and (45), or
- (ii) The uniform $\mathbf{u}_i^j$ or $\mathbf{u}_{i'}^{j'}$ sampled in the step 16 of SimIn and independent of $\mathbf{y}_i^j$ or $\mathbf{y}_{i'}^{j'}$, or
- (iii) The uniform $T_i = \text{Half}_1(\mathbf{y}_i^j \oplus \mathbf{u}_i^j)$ (resp., $T_{i'} = \text{Half}_1(\mathbf{y}_{i'}^{j'} \oplus \mathbf{u}_{i'}^{j'})$) when the upper half of the XOR is considered, $n_i(f)$ (resp., $n_{i'}(f)$) is odd, and $j = \lceil n_i(f)/2 \rceil - 1$ (resp., $j' = \lceil n_{i'}(f)/2 \rceil - 1$).

In this worst case, each half of $\mathbf{u}_i^j \oplus \mathbf{u}_{i'}^{j'}$ includes at least $1 + (\lambda/2 - 1) = \lambda/2$ uniform bits for the lower-half non-LSBs of some $(\mathbf{r}_{s_a s_b}^g \parallel \mathbf{X}_c)$. These non-LSBs are independent of $\mathbf{y}_i^j$ and $\mathbf{y}_{i'}^{j'}$ defined as per (43) and (45), or other (previously fixed) active labels, including $\mathbf{X}_i$ and $\mathbf{X}_{i'}$. Thus, (35) holds with probability $2^{-\lambda}$ for some fixed distinct (i, j) and (i', j') .

Taking a union bound over the above cases, we have

$$\Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\text{bad}_1] \leq \frac{5N(N-1)}{2^{\lambda+1}}. \quad (51)$$

Then, consider bad_2 . For (36), it is clear that $(\mathbf{0} \parallel \mathbf{\Delta}) = \alpha \oplus (\mathbf{0} \parallel \mathbf{X}_i) \oplus (i \parallel j)$, which occurs with probability $2^{-(\lambda-1)}$ according to the randomness of $\mathbf{\Delta}$. Let $\mathbf{\Delta} = [\Delta_L \ \Delta_R]^\top \in \mathbb{F}_{2^{\lambda/2}}^2$. We note that each $\mathbf{y}_i^j \in \mathbb{F}_{2^{\lambda/2+1}}^2$ defined from (43) and (45) includes an additive term of the form

$$L_{\xi_1, \xi_2, \xi_3, \xi_4}(\mathbf{0} \parallel \mathbf{\Delta}) = \begin{bmatrix} 0 \parallel \xi_1 \Delta_L \oplus \xi_2 \Delta_R \\ 0 \parallel \xi_3 \Delta_L \oplus \xi_4 \Delta_R \end{bmatrix} \in \mathbb{F}_{2^{\lambda/2+1}}^2$$

for some $\xi_1, \xi_2, \xi_3, \xi_4 \in \mathbb{F}_2$, while other additive terms in $\mathbf{y}_i^j$ are independent of $\mathbf{\Delta}$. Linear orthomorphism σ for every $L_{\xi_1, \xi_2, \xi_3, \xi_4}$ turns (37) into

$$\begin{aligned} \sigma(\mathbf{0} \parallel \mathbf{\Delta}) \oplus L_{\xi_1, \xi_2, \xi_3, \xi_4}(\mathbf{0} \parallel \mathbf{\Delta}) &= \beta \oplus \mathbf{u}_i^j \oplus \sigma((\mathbf{0} \parallel \mathbf{X}_i) \oplus (i \parallel j)) \\ &\quad \oplus \text{“other additive terms in } \mathbf{y}_i^j \text{”}, \end{aligned}$$

which is invertible to compute $(\mathbf{0} \parallel \mathbf{\Delta})$. This implies that the equality holds with probability $2^{-(\lambda-1)}$ due to the randomness of $\mathbf{\Delta}$. Taking a union bound, we have

$$\Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\text{bad}_2] \leq \frac{2N \cdot (q_1 + q_2 + q_3)}{2^{\lambda-1}}. \quad (52)$$

Finally, consider bad_3 . Recall that each $\mathbf{X}_w$ has at least $\lambda - 1$ random non-LSBs. Since these bits are independent of $\cup_{\ell=1}^2 \mathcal{K}_\ell$, (38) holds with probability at most $2^{-(\lambda-1)}$ for some fixed $(\alpha, \dots)$ and (i, j) . For (39), the mask sampled in the step 13 or the direct sampling in the step 13 or 16 of SimIn ensures that both halves of $\mathbf{u}_i^j \in \mathbb{F}_{2^{\lambda/2+1}}^2$ are uniform and independent of $\mathbf{X}_i$ or $\cup_{\ell=1}^2 \mathcal{K}_\ell$. It holds with probability $2^{-(\lambda+2)} < 2^{-(\lambda-1)}$ for some fixed $(\dots, \beta)$ and (i, j) . So, we can take a union bound to have that

$$\Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\text{bad}_3] \leq \frac{2N \cdot (q_1 + q_2)}{2^{\lambda-1}}. \quad (53)$$

```

SimFπ±1(·)(f):
1:  $F := \{(g^g, z^g)\}_{g \in \mathcal{G}_{\text{and}}(f)} \leftarrow (\mathbb{F}_{2^{\lambda/2}}^3 \times \mathbb{F}_2^5)^{|f|}$ 
2:  $\{\mathbf{X}_i\}_{i \in \mathcal{W}_{\text{in}}(f)} \leftarrow (\mathbb{F}_{2^{\lambda/2}}^2)^{|\mathcal{W}_{\text{in}}(f)|}$ 
3: for  $i \in \mathcal{W}_{\text{in}}(f)$  do  $\text{ctr}_i := 0$ 
4: for  $g \in \mathcal{G}(f)$  in order do
5:    $(a, b, c) := (\text{in}_0(g), \text{in}_1(g), \text{out}(g))$ 
6:   if  $\text{type}(g) = \text{XOR}$  do  $\mathbf{X}_c := \mathbf{X}_a \oplus \mathbf{X}_b$ 
7:   else if  $\text{type}(g) = \text{AND}$  do
8:      $\Gamma := \text{in}_2(g)$ 
9:      $(\chi_a, \rho_a) := (\lfloor \text{ctr}_a/2 \rfloor, \text{lsb}(\text{ctr}_a)), \text{ctr}_a := \text{ctr}_a + 1$ 
10:     $(\chi_b, \rho_b) := (\lfloor \text{ctr}_b/2 \rfloor, \text{lsb}(\text{ctr}_b)), \text{ctr}_b := \text{ctr}_b + 1$ 
11:     $(\chi_\Gamma, \rho_\Gamma) := (\lfloor \text{ctr}_\Gamma/2 \rfloor, \text{lsb}(\text{ctr}_\Gamma)), \text{ctr}_\Gamma := \text{ctr}_\Gamma + 1$ 
12:     $s_a := \text{lsb}(\mathbf{X}_a), s_b := \text{lsb}(\mathbf{X}_b)$ 
13:     $\begin{bmatrix} \text{Half}_{\rho_a}(\mathbf{u}_a^{\chi_a}) \\ \text{Half}_{\rho_b}(\mathbf{u}_b^{\chi_b}) \\ \text{Half}_{\rho_\Gamma}(\mathbf{u}_\Gamma^{\chi_\Gamma}) \end{bmatrix} := \begin{bmatrix} \text{Half}_{\rho_a}(\pi((\mathbf{0} \parallel \mathbf{X}_a) \oplus (a \parallel \chi_a)) \oplus \sigma((\mathbf{0} \parallel \mathbf{X}_a) \oplus (a \parallel \chi_a))) \\ \text{Half}_{\rho_b}(\pi((\mathbf{0} \parallel \mathbf{X}_b) \oplus (b \parallel \chi_b)) \oplus \sigma((\mathbf{0} \parallel \mathbf{X}_b) \oplus (b \parallel \chi_b))) \\ \text{Half}_{\rho_\Gamma}(\pi((\mathbf{0} \parallel \mathbf{X}_a \oplus \mathbf{X}_b) \oplus (\Gamma \parallel \chi_\Gamma)) \oplus \sigma((\mathbf{0} \parallel \mathbf{X}_a \oplus \mathbf{X}_b) \oplus (\Gamma \parallel \chi_\Gamma))) \end{bmatrix}$ 
14:     $(\mathbf{r}_{s_a s_b}^g \parallel \mathbf{m}_{s_a s_b}^g) := \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix} \begin{bmatrix} \text{Half}_{\rho_a}(\mathbf{u}_a^{\chi_a}) \\ \text{Half}_{\rho_b}(\mathbf{u}_b^{\chi_b}) \\ \text{Half}_{\rho_\Gamma}(\mathbf{u}_\Gamma^{\chi_\Gamma}) \end{bmatrix} \oplus \mathbf{V}_{s_a s_b} \left( z^g \parallel \begin{bmatrix} \mathbf{0} \\ g^g \end{bmatrix} \right)$ 
15:     $\mathbf{R}_{s_a s_b}^g := \text{TH.DecodeR}(\mathbf{r}_{s_a s_b}^g, s_a, s_b)$ 
16:     $\mathbf{X}_c := \mathbf{m}_{s_a s_b}^g \oplus \mathbf{R}_{s_a s_b}^g \begin{bmatrix} \mathbf{X}_a \\ \mathbf{X}_b \end{bmatrix}$ 
17: return  $\hat{f} := (f, F), \text{st}_{\text{sim}} := (f, \tilde{\mathbf{X}} := \{\mathbf{X}_i\}_{i \in \mathcal{W}(f)}, \tilde{\mathbf{u}} := \{\mathbf{u}_i^j\}_{i \in \mathcal{W}_{\text{in}}(f), j \in [0, \lceil n_i(f)/2 \rceil - 1]}, \tilde{\mathbf{r}} := \{\mathbf{r}_{s_a s_b}^g\}_{g \in \mathcal{G}_{\text{and}}(f), (a,b) := (\text{in}_0(g), \text{in}_1(g))})$ 

SimInπ±1(·)(f(x)):
1: Parse  $\text{st}_{\text{sim}} = (f, \tilde{\mathbf{X}} = \{\mathbf{X}_i\}_{i \in \mathcal{W}(f)}, \tilde{\mathbf{u}}, \tilde{\mathbf{r}})$ 
2: for  $i \in \mathcal{W}_{\text{out}}(f)$  do  $d_i := f(x)_i \oplus \text{lsb}(\mathbf{X}_i)$ 
3: return  $\hat{x} := (\{\mathbf{X}_i\}_{i \in \mathcal{W}_{\text{in}}(f)}, d), \tilde{\mathbf{X}}, \tilde{\mathbf{u}}, \tilde{\mathbf{r}}.$ 

```

Figure 10: Our simulator for three-halves in the npRPM.

We have a bound ε_1 from (51), (52), and (53):

$$\begin{aligned}
& \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [Y(\omega) \in \mathcal{T}_{\text{bad}}] \\
&= \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\text{bad}_1 \vee \text{bad}_2 \vee \text{bad}_3] \\
&\leq \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\text{bad}_1] + \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\text{bad}_2] + \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\text{bad}_3] \\
&\leq \frac{5N(N-1) + 16N(q+N)}{2^{\lambda+1}} \\
&\leq \frac{48qs + 189s^2 - 15s}{2^{\lambda+1}} = \varepsilon_1,
\end{aligned}$$

where the last inequality comes from (49), $M \leq |\mathcal{W}_{\text{in}}(f)| \leq 3|f|$, and $|f| = s$.

This theorem holds for the H-coefficient bound from $(\varepsilon_1, \varepsilon_2)$. $\square$

D.2 Adaptive Security in npRPM

We consider a slightly different three-halves implementation from the original one of [RR21] in that decoding information d is transferred from $\hat{f}$ to k in TH.Garble^{π^{±1}(·)} and then included in garbled input $\hat{x}$ in the online phase. This implementation also follows the lower bound in Theorem 5 and satisfies the adaptive security in the npRPM as per the following theorem.

Theorem 4. Let $H(\mathbf{X}, k) = \pi((\mathbf{0} \parallel \mathbf{X}) \oplus k) \oplus \sigma((\mathbf{0} \parallel \mathbf{X}) \oplus k)$ be a tweakable hash function where $\mathbf{X} \in \mathbb{F}_{2^{\lambda/2}}^2, k \in \mathbb{F}_{2^{\lambda/2+1}}^2, \pi \in \mathcal{S}_{\lambda+2}$ is random permutation, and $\sigma : \mathbb{F}_{2^{\lambda/2+1}}^2 \rightarrow \mathbb{F}_{2^{\lambda/2+1}}^2$ is a linear

orthomorphism for the function family

$$\mathcal{L} = \{L_{\xi_1, \xi_2, \xi_3, \xi_4} : \mathbb{F}_2^{2^{\lambda/2+1}} \rightarrow \mathbb{F}_2^{2^{\lambda/2+1}}\}_{\xi_1, \xi_2, \xi_3, \xi_4 \in \mathbb{F}_2}, \quad L_{\xi_1, \xi_2, \xi_3, \xi_4} \left(\begin{bmatrix} x_L \\ x_R \end{bmatrix} \right) = \begin{bmatrix} \xi_1 x_L \oplus \xi_2 x_R \\ \xi_3 x_L \oplus \xi_4 x_R \end{bmatrix}.$$

Then, *three-halves* (sketched above) is a $(\lambda + 2)$ -garbling scheme with (q, s, ε) -adaptive security in the $npRPM$, where $\varepsilon = (69qs + 234s^2)/2^{\lambda+2}$.

Proof (sketch). The correctness can be proved as [RR21] as postponing decoding information d does not affect correctness. We only need to consider the simulation.

Our simulator $\text{Sim} = (\text{SimF}, \text{SimIn})$ is presented in Figure 10 and is obviously PPT. Then, we prove this theorem using the following three hybrids:

- Hybrid_0 . This is the adaptive experiment with simulator Sim .
- Hybrid_1 . This is identical to the previous hybrid, except that we replace $\pi^{\pm 1}$ (which can be equivalently emulated on-the-fly as in Figure 5) by an approximation $\tilde{\pi}^{\pm 1}$ (given in Figure 6). This approximation is the same as random permutation except that, for a new query of the simulator, it returns a fresh random string as response and records this query-response pair. This hybrid is used to simplify probability analysis.
- Hybrid_2 . This is the adaptive experiment with three-halves.

According to Lemma 2, every adaptive adversary $\mathcal{A}$ can distinguish Hybrid_0 and Hybrid_1 with advantage at most $(q + N) \cdot N/2^{\lambda+1}$ up to the supposed q queries of $\mathcal{A}$ and the N queries of Sim .

Then, we prove the negligible statistical distance between the transcripts in Hybrid_1 and Hybrid_2 , which upper bounds the advantage of adaptive adversary $\mathcal{A}$. To simplify probability analysis, we also use the H-coefficient technique and the same transcript padding as in the proof of Theorem 3. We regard Hybrid_1 (resp., Hybrid_2) and the associated oracle as the ideal (resp., real) world in the H-coefficient technique. The real-world sample space is the same as that in the proof of Theorem 3, but the ideal-world sample space is defined as

$$\begin{aligned} \Omega_{\text{ideal}} = & (\mathbb{F}_2^{3\lambda/2+5})^{|f|} \times (\mathbb{F}_2^\lambda)^{|\mathcal{W}_{\text{in}}(f)|} \times \underbrace{(\mathbb{F}_2^{\lambda/2+1})^M}_{\text{for the sampling of } \tilde{T}} \\ & \times \underbrace{\{0, 1\}^*}_{\text{random tape for the sampling in } \tilde{\pi}^{\pm 1}(\cdot)} \times \underbrace{\mathbb{F}_2^{\lambda-1}}_{\text{dummy } \Delta}. \end{aligned}$$

We will consider the same bad transcripts as in the proof of Theorem 3 (where bad_3 is for probability analysis instead of programming, as its counterpart in the proof of Theorem 1) and assume that, without loss of generality, $\mathcal{A}$ only makes non-repeating queries. Let $q_\ell := |\mathcal{K}_\ell|$ for every $\ell \in \{1, 2, 3\}$ and $q_\Sigma := \sum_{\ell=1}^3 q_\ell$.

Bounding $1 - \varepsilon_2$. We can follow a similar argument (with the difference that the consistency (41) and (42) trivially hold for good transcripts due to the step 12 to 15 in SimF) in the proof of Theorem 3 to show that, for some fixed good transcript τ such that $\Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [Y(\omega) = \tau] > 0$ and $q_\Sigma = q + N$

(which comes from the condition $\neg \text{bad}_1 \wedge \neg \text{bad}_3$ for good transcripts),

$$\begin{aligned}
& \Pr_{\omega \leftarrow \Omega_{\text{real}}} [\omega \in \text{comp}_{\text{real}}(\tau)] \\
&= \frac{1}{(2^\lambda)^{|\mathcal{W}_{\text{in}}(f)|}} \cdot \frac{(2^{\lambda+2} - 2N - (q_\Sigma - N))!}{(2^{\lambda+2})!} \cdot \frac{1}{2^{2|f|+(\lambda-1)}} \\
&= \frac{1}{(2^\lambda)^{|\mathcal{W}_{\text{in}}(f)|}} \cdot \frac{1}{(2^{\lambda+2})_{q+2N}} \cdot \frac{1}{2^{2|f|+(\lambda-1)}}, \\
& \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\omega \in \text{comp}_{\text{ideal}}(\tau)] \\
&\leq \frac{1}{(2^{\lambda+2} - N)_q} \cdot \frac{1}{2^{|\mathcal{W}_{\text{in}}(f)|\lambda + (3\lambda/2+5)|f| + (\lambda/2+1)M + (\lambda-1)}} \cdot \frac{1}{(2^{\lambda+2})^N} \\
&= \frac{1}{(2^\lambda)^{|\mathcal{W}_{\text{in}}(f)|}} \cdot \frac{1}{(2^{\lambda+2} - N)_q} \cdot \frac{1}{(2^{\lambda/2+1})^{3|f|+M+2N}} \cdot \frac{1}{2^{2|f|+(\lambda-1)}} \\
&= \frac{1}{(2^\lambda)^{|\mathcal{W}_{\text{in}}(f)|}} \cdot \frac{1}{(2^{\lambda+2} - N)_q} \cdot \frac{1}{(2^{\lambda+2})^{2N}} \cdot \frac{1}{2^{2|f|+(\lambda-1)}}, \quad (\text{By (49)}) \\
&\Rightarrow \frac{\Pr_{\omega \leftarrow \Omega_{\text{real}}} [\omega \in \text{comp}_{\text{real}}(\tau)]}{\Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\omega \in \text{comp}_{\text{ideal}}(\tau)]} \geq 1.
\end{aligned}$$

That is, we have $\varepsilon_2 = 0$.

Bounding ε_1 . First, we consider $\text{bad}_1 \vee \text{bad}_3 = (33) \vee (34) \vee (35) \vee (38) \vee (39)$. We have a similar induction in the proof of Theorem 1 to prove the probability of bad values of some forward queries:

$$\Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [(33) \vee (38)] \leq \frac{N(N-1) + 2N \cdot (q_1 + q_2)}{2^{\lambda+1}}. \quad (54)$$

We consider (34), (35), and (39) conditioned on $\neg((33) \vee (38))$. In each of them, $\mathbf{u}_i^j \oplus \sigma((\mathbf{0} \parallel \mathbf{X}_i) \oplus (i \parallel j))$ is the response for query $(\mathbf{0} \parallel \mathbf{X}_i) \oplus (i \parallel j)$ to $\tilde{\pi}^{\pm 1}(\cdot)$. It follows from this condition that these queries are pairwise distinct so that their responses are taken from uniform $c_1, \dots, c_{n(\lambda)}$ in $\tilde{\pi}^{\pm 1}(\cdot)$, where $\ell(\lambda) = \lambda + 2$, and pairwise independent given the pairwise distinct queries. So, each of them occurs with probability $1/2^{\lambda+2}$ for some fixed quantifier. Taking a union bound over all quantifiers, we have

$$\Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [(34) \vee (35) \vee (39) \mid \neg((33) \vee (38))] \leq \frac{N(N-1) + N \cdot (q_1 + q_2)}{2^{\lambda+2}}. \quad (55)$$

Using (54) and (55), we have

$$\begin{aligned}
& \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\text{bad}_1 \vee \text{bad}_3] \\
&= \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [(33) \vee (34) \vee (35) \vee (38) \vee (39)] \\
&= \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [(33) \vee (38)] + \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [(34) \vee (35) \vee (39) \mid \neg((33) \vee (38))] \\
&\leq \frac{3N(N-1) + 5N \cdot (q_1 + q_2)}{2^{\lambda+2}}.
\end{aligned} \quad (56)$$

Then, consider bad_2 . It is easy to see from the randomness of Δ that

$$\Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\text{bad}_2] \leq \frac{2N \cdot (q_1 + q_2 + q_3)}{2^{\lambda-1}}. \quad (57)$$

We have a bound ε_1 from (56) and (57):

$$\begin{aligned}
& \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [Y(\omega) \in \mathcal{T}_{\text{bad}}] \\
&= \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\text{bad}_1 \vee \text{bad}_2 \vee \text{bad}_3] \\
&\leq \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\text{bad}_1 \vee \text{bad}_3] + \Pr_{\omega \leftarrow \Omega_{\text{ideal}}} [\text{bad}_2] \\
&\leq \frac{3N(N-1) + 21N(q+N)}{2^{\lambda+2}} \\
&\leq \frac{63qs + 216s^2 - 9s}{2^{\lambda+2}} = \varepsilon_1,
\end{aligned}$$

where the last inequality comes from (49), $M \leq |\mathcal{W}_{\cup}(f)| \leq 3|f|$, and $|f| = s$.

By using the H-coefficient technique with the above ε_1 and ε_2 , we have that any adaptive adversary can distinguish Hybrid_1 and Hybrid_2 with advantage at most $(63qs + 216s^2 - 9s)/2^{\lambda+2}$.

Putting the three hybrids together, we prove this theorem. $\square$

E Separation between npRPM and pRPM

We separate the adaptively secure garbling schemes in the two RPMs since the programmability can make a difference in online complexity. In Theorem 5, we prove that the online-complexity lower bound [AIKW13, HW15] of the adaptively secure garbling schemes in the standard model can be extended to the npRPM. However, it is implied by the definition of the adaptively secure garbling schemes in the pRPM that this lower bound can be bypassed if the random permutation is allowed to be programmed by the simulator to embed messages. The proof of Theorem 5 is extended from [HW15], which is based on Yao entropy.

Definition 3 (Yao entropy [Yao82, BSW03, HLR07]). *A distribution $\mathcal{X}$ has Yao entropy at least $\rho \in \mathbb{N}^+$, denoted by $H^{\text{Yao}}(\mathcal{X}) \geq \rho$, if there exists a negligible function ε such that, for every pair of polynomial-size circuits $(C, \mathcal{D})$ where C has output bit-length at most $\rho - 1$, it holds that*

$$\Pr_{x \leftarrow \mathcal{X}} [\mathcal{D}(C(x)) = x] \leq \frac{1}{2} + \varepsilon(\lambda).$$

E.1 High-level Overview

In the npRPM, the simulator of adaptive garbling consists of two probabilistic polynomial-time (PPT) algorithms: $\text{SimF}^{\pi^{\pm 1}(\cdot)}$ and $\text{SimIn}^{\pi^{\pm 1}(\cdot)}$, which have oracle access to a random permutation π and its inverse π^{-1} . The former simulates garbled circuit $\hat{f}$ (without online input x) and outputs an internal state, while the latter simulates garbled input $\hat{x}$ given the internal state and circuit output $f(x)$. Intuitively, since the simulated $\hat{x}$ is the only message that can depend on $f(x)$, $\hat{x}$ should consist of “adequate” information of $f(x)$. Otherwise, we cannot reconstruct $f(x)$ from the simulated $(\hat{f}, \hat{x})$ in the ideal world, making the real world and ideal world trivially distinguishable.

Following prior works [AIKW13, HW15], we will also evaluate this “adequacy” based on the Yao entropy [Yao82, BSW03, HLR07] of $f(x)$. This entropy is defined as the minimal bit-length of an efficiently computable compressed form of $f(x)$, which can be decompressed to $f(x)$ with probability significantly more than $1/2$. Similar to the standard-model lower bound in the prior works, we can prove that the bit-length of $\hat{x}$ should be at least the Yao entropy of $f(x)$ in the npRPM. The high-level idea is that the simulator can compress $f(x)$ into $\hat{x}$, which can be decompressed using the evaluation algorithm of garbling scheme with overwhelming probability. Otherwise, the ideal world is trivially distinguishable from the real one. Recall that the simulator and the evaluation algorithm are PPT and the adversary also makes a polynomial number of queries to $\pi^{\pm 1}$. The total number of permutation queries in such compression and decompression is polynomial, and their responses can be approximated by uniform strings up to negligible statistical distance. As a result, the above compression and

decompression have polynomial-size circuits as required by Yao entropy, where the circuits hardcode the uniform responses used to approximate the real ones from $\pi^{\pm 1}$.

More specifically, we first define an intermediate hybrid by replacing a random permutation and its inverse with “coarse” approximation $\pi^{\pm 1}$, which outputs a fresh random string upon a fresh query but ensures the query-response consistency. Clearly, the advantage of any adaptive adversary to distinguish this hybrid and the ideal world is at most twice the birthday bound, which is negligible up to a polynomial number of queries to the permutation and its inverse. Since the garbling scheme is adaptively secure in the npRPM, the ideal world is indistinguishable from the real one. As a result, $\text{SimF}^{\pi^{\pm 1}(\cdot)}$ and $\text{SimIn}^{\pi^{\pm 1}(\cdot)}$ give an approximate simulation indistinguishable from the real world.

This indistinguishability implies that the approximate simulation results in $(\hat{f}, \hat{x})$ that can be decoded to $f(x)$ with overwhelming probability. If the bit-length of this $\hat{x}$ is less than the Yao entropy of $f(x)$, we will show that the approximate simulation contradicts this entropy. We can construct a compression circuit that sequentially runs $\text{SimF}^{\pi^{\pm 1}(\cdot)}$ and $\text{SimIn}^{\pi^{\pm 1}(\cdot)}$ over some hardcoded random tape to output such an $\hat{x}$. The decompression circuit uses the same random tape to recompute $\hat{f}$ output by $\text{SimF}^{\pi^{\pm 1}(\cdot)}$, and runs the deterministic evaluation algorithm to compute $f(x)$ from $(\hat{f}, \hat{x})$. According to the coarse approximate permutation (which is hardcoded in the circuits), both circuits are of polynomial sizes. Thus, these circuits contradict the Yao entropy of $f(x)$, leading to a lower bound of the online complexity in the npRPM.

However, this lower bound does not hold in the pRPM, where the simulator has another PPT algorithm $\text{SimP}^{\pm 1}$ to emulate a random permutation and its inverse for queries (a) before $\hat{f}$, (b) after $\hat{f}$ but before $\hat{x}$, and (c) after $\hat{x}$. It is notable that SimF , SimIn , and $\text{SimP}^{\pm 1}$ maintain an internal state. In particular, given $f(x)$ and the internal state, SimIn outputs simulated $\hat{x}$ and *an updated internal state to be used in $\text{SimP}^{\pm 1}$ for case (c)*. Although we can prove that the output bit-length of SimIn should exceed the Yao entropy of $f(x)$, a part of the information of $f(x)$ can be transferred into the updated internal state so that the bit-length of the simulated $\hat{x}$ can be lower than the Yao entropy. As a result, the lower bound does not hold for a real-world $\hat{x}$ since the real and ideal worlds are indistinguishable for the adaptive security in the pRPM. This result is confirmed by our proofs for the adaptive security of half-gates and three-halves in the pRPM, where programming is performed to embed the decoding consistency, e.g., $\text{lsb}(X_c) = x_c \oplus d_c$, into the internal state of the simulator.

E.2 Detailed Proof

Theorem 5 (A lower bound of the online complexity of adaptive garbling in the npRPM). *For every polynomial-size circuit $f : \{0, 1\}^{\ell_{\text{in}}} \rightarrow \{0, 1\}^{\ell_{\text{out}}}$ with a distribution $\mathcal{X}_f$ on $\{0, 1\}^{\ell_{\text{in}}}$ such that distribution $f(\mathcal{X}_f)$ on $\{0, 1\}^{\ell_{\text{out}}}$ has Yao entropy at least $\rho > 0$, and every $\ell(\lambda)$ -garbling scheme that is $(\text{poly}(\lambda), \text{poly}'(\lambda), \text{negl}(\lambda))$ -adaptively secure in the npRPM, the output length of $\text{Encode}^{\pi^{\pm 1}(\cdot)}(k, \cdot)$ is at least ρ bits for $(\cdot, k) \leftarrow \text{Garble}^{\pi^{\pm 1}(\cdot)}(f)$.*

Proof. For the sake of contradiction, assume that there exist a polynomial-size circuit f with an input distribution $\mathcal{X}_f$ inducing Yao entropy $H^{\text{Yao}}(f(\mathcal{X}_f)) \geq \rho$, and an adaptively secure garbling scheme in the npRPM, such that the output of $\text{Encode}^{\pi^{\pm 1}(\cdot)}(k, \cdot)$ has at most $\rho - 1$ bits for $(\cdot, k) \leftarrow \text{Garble}^{\pi^{\pm 1}(\cdot)}(f)$. We prove that there exists a pair of polynomial-size compressor and decompressor circuits that contradicts the Yao entropy $H^{\text{Yao}}(f(\mathcal{X}_f)) \geq \rho$.

We specify auxiliary input $z = (f, \mathcal{X}_f, \rho)$ and the following computationally unbounded $\mathcal{A}$:

- $\mathcal{A}_1^{\pi^{\pm 1}(\cdot)}(z)$ outputs f in z and defines $\text{st}_1 := z$.
- $\mathcal{A}_2^{\pi^{\pm 1}(\cdot)}(\text{st}_1, \hat{f})$ samples $x \leftarrow \mathcal{X}_f$ and defines $\text{st}_2 := (\text{st}_1, x)$.
- $\mathcal{A}_3^{\pi^{\pm 1}(\cdot)}(\text{st}_2, \hat{f}, \hat{x})$ outputs 1 if $\text{DecEval}^{\pi^{\pm 1}(\cdot)}(\hat{f}, \hat{x}) = f(x)$ and $\hat{x}$ has at most $\rho - 1$ bits, or outputs 0 otherwise. Recall that running $\text{DecEval}^{\pi^{\pm 1}(\cdot)}$ requires a polynomial number of oracle queries to $\pi^{\pm 1}$ as this algorithm is PPT.

The correctness and the contradiction assumption ensure that, for this $(z, \mathcal{A})$,

$$\Pr \left[\begin{array}{l} \pi \leftarrow \mathcal{S}_{\ell(\lambda)}, \pi^{-1} := \text{inv}(\pi), \\ (f, \text{st}_1) \leftarrow \mathcal{A}_1^{\pi^{\pm 1}(\cdot)}(z), \\ (\hat{f}, k) \leftarrow \text{Garble}^{\pi^{\pm 1}(\cdot)}(f), : \mathcal{A}_3^{\pi^{\pm 1}(\cdot)}(\text{st}_2, \hat{f}, \hat{x}) = 1 \\ (x, \text{st}_2) \leftarrow \mathcal{A}_2^{\pi^{\pm 1}(\cdot)}(\text{st}_1, \hat{f}), \\ \hat{x} := \text{Encode}^{\pi^{\pm 1}(\cdot)}(k, x) \end{array} \right] = 1. \quad (58)$$

Meanwhile, the adaptive security in the npRPM guarantees that there exists a PPT simulator $\text{Sim} = (\text{SimF}, \text{SimIn})$ such that, for this $(z, \mathcal{A})$, it holds that

$$\left| \Pr \left[\begin{array}{l} \pi \leftarrow \mathcal{S}_{\ell(\lambda)}, \pi^{-1} := \text{inv}(\pi), \\ (f, \text{st}_1) \leftarrow \mathcal{A}_1^{\pi^{\pm 1}(\cdot)}(z), \\ (\hat{f}, k) \leftarrow \text{Garble}^{\pi^{\pm 1}(\cdot)}(f), : \mathcal{A}_3^{\pi^{\pm 1}(\cdot)}(\text{st}_2, \hat{f}, \hat{x}) = 1 \\ (x, \text{st}_2) \leftarrow \mathcal{A}_2^{\pi^{\pm 1}(\cdot)}(\text{st}_1, \hat{f}), \\ \hat{x} := \text{Encode}^{\pi^{\pm 1}(\cdot)}(k, x) \end{array} \right] - \Pr \left[\begin{array}{l} \pi \leftarrow \mathcal{S}_{\ell(\lambda)}, \pi^{-1} := \text{inv}(\pi), \\ (f, \text{st}_1) \leftarrow \mathcal{A}_1^{\pi^{\pm 1}(\cdot)}(z), \\ \hat{f} \leftarrow \text{SimF}^{\pi^{\pm 1}(\cdot)}(\Phi(f)), : \mathcal{A}_3^{\pi^{\pm 1}(\cdot)}(\text{st}_2, \hat{f}, \hat{x}) = 1 \\ (x, \text{st}_2) \leftarrow \mathcal{A}_2^{\pi^{\pm 1}(\cdot)}(\text{st}_1, \hat{f}), \\ \hat{x} \leftarrow \text{SimIn}^{\pi^{\pm 1}(\cdot)}(f(x)) \end{array} \right] \right| \leq \text{negl}(\lambda). \quad (59)$$

Using (58) and (59), we have for the specified $(z, \mathcal{A})$ that

$$\Pr \left[\begin{array}{l} \pi \leftarrow \mathcal{S}_{\ell(\lambda)}, \pi^{-1} := \text{inv}(\pi), \\ (f, \text{st}_1) \leftarrow \mathcal{A}_1^{\pi^{\pm 1}(\cdot)}(z), \\ \hat{f} \leftarrow \text{SimF}^{\pi^{\pm 1}(\cdot)}(\Phi(f)), : \mathcal{A}_3^{\pi^{\pm 1}(\cdot)}(\text{st}_2, \hat{f}, \hat{x}) = 1 \\ (x, \text{st}_2) \leftarrow \mathcal{A}_2^{\pi^{\pm 1}(\cdot)}(\text{st}_1, \hat{f}), \\ \hat{x} \leftarrow \text{SimIn}^{\pi^{\pm 1}(\cdot)}(f(x)) \end{array} \right] \geq 1 - \text{negl}(\lambda), \quad (60)$$

where the probability is taken over the random choice of $\pi \in \mathcal{S}_{\ell(\lambda)}$, the random tape of Sim , and the random tape used by $\mathcal{A}_2$ to sample $x \leftarrow \mathcal{X}_f$.

Recall that both the adversary $\mathcal{A}$ and the PPT simulator Sim can only make a polynomial number of oracle queries. Let $n(\lambda) \in \mathbb{N}^+$ denote the number of oracle queries jointly made by $\mathcal{A}$ and Sim in the simulation (60). For the $n(\lambda)$ oracle queries, permutation π and its inverse π^{-1} can be emulated on-the-fly as in Figure 5 on the uniform distribution of random tape (r_π, r_π^*) . This on-the-fly emulation requires exact $n(\lambda) \cdot \ell(\lambda)$ bits of r_π but possibly exponentially large r_π^* . To remove r_π^* , we can consider a coarse approximation $\overset{\circ}{\pi}^{\pm 1}$ (see Figure 11), which differs from $\pi^{\pm 1}$ in that it always returns a fresh c_i as the response for a new query. Similar to Lemma 2, it can be proved that replacing $\pi^{\pm 1}$ with $\overset{\circ}{\pi}^{\pm 1}$ only incurs additional statistical distance at most $n(\lambda)^2/2^{\ell(\lambda)}$. So, we have for $\text{negl}'(\lambda) := \text{negl}(\lambda) + n(\lambda)^2/2^{\ell(\lambda)}$ and the distribution $\overset{\circ}{\mathcal{P}}_{\ell(\lambda)}$ of $\overset{\circ}{\pi}^{\pm 1}(\cdot)$ that

$$\Pr \left[\begin{array}{l} \overset{\circ}{\pi}^{\pm 1} \leftarrow \overset{\circ}{\mathcal{P}}_{\ell(\lambda)}, \\ (f, \text{st}_1) \leftarrow \mathcal{A}_1^{\overset{\circ}{\pi}^{\pm 1}(\cdot)}(z), \\ \hat{f} \leftarrow \text{SimF}^{\overset{\circ}{\pi}^{\pm 1}(\cdot)}(\Phi(f)), : \mathcal{A}_3^{\overset{\circ}{\pi}^{\pm 1}(\cdot)}(\text{st}_2, \hat{f}, \hat{x}) = 1 \\ (x, \text{st}_2) \leftarrow \mathcal{A}_2^{\overset{\circ}{\pi}^{\pm 1}(\cdot)}(\text{st}_1, \hat{f}), \\ \hat{x} \leftarrow \text{SimIn}^{\overset{\circ}{\pi}^{\pm 1}(\cdot)}(f(x)) \end{array} \right] \geq 1 - \text{negl}'(\lambda), \quad (61)$$

where the probability is taken over the random tape r_π to emulate $\overset{\circ}{\pi}^{\pm 1}(\cdot)$, the random tape $(r_{\text{SimF}}, r_{\text{SimIn}})$ of Sim , and the random tape used by $\mathcal{A}_2$ to sample $x \leftarrow \mathcal{X}_f$. We stress that r_π and $(r_{\text{SimF}}, r_{\text{SimIn}})$ are polynomial-size as r_π carries $n(\lambda) \cdot \ell(\lambda)$ uniform bits and the simulator is PPT.

```

1: Initialize a list  $\mathcal{Q}_0 = \emptyset$ .
2:  $c_1, \dots, c_{n(\lambda)} \leftarrow \{0, 1\}^{\ell(\lambda)}$  a.
3: for  $i \in [1, n(\lambda)]$  do
4:   if  $\overset{\circ}{\pi}$  is queried with  $\alpha_i \in \{0, 1\}^{\ell(\lambda)}$  do
5:     if  $\exists (\alpha_i, \gamma_i) \in \mathcal{Q}_{i-1}$  do Return  $\gamma_i$ .
6:     Return  $\gamma_i := c_i$  and define  $\mathcal{Q}_i := \mathcal{Q}_{i-1} \cup \{(\alpha_i, \gamma_i)\}$ .
7:   else if  $\overset{\circ}{\pi}^{-1}$  is queried with  $\beta_i \in \{0, 1\}^{\ell(\lambda)}$  do
8:     if  $\exists (\gamma_i, \beta_i) \in \mathcal{Q}_{i-1}$  do Return  $\gamma_i$ .
9:     Return  $\gamma_i := c_i$  and define  $\mathcal{Q}_i := \mathcal{Q}_{i-1} \cup \{(\gamma_i, \beta_i)\}$ .

```

^aA uniform random tape r_π is used.

Figure 11: The workflow of coarse approximate oracle $\overset{\circ}{\pi}^{\pm 1}(\cdot)$ up to $n(\lambda)$ queries.

We point out that (61) implies the existence of a circuit pair violating $H^{\text{Yao}}(f(\mathcal{X}_f)) \geq \rho$. To see this, let $\mathcal{C}_{\ell, \rho, f, r}$ (resp., $\mathcal{D}_{\ell, \rho, f, r}$) denote the compressor (resp., decompressor) circuit, which hardcodes (ℓ, ρ, f) and a random tape $r = (r_\pi, r_{\text{SimF}}, r_{\text{SimIn}})$ with the following functionality:

- Given $f(x) \leftarrow f(\mathcal{X}_f)$, the compressor $\mathcal{C}_{\ell, \rho, f, r}$ invokes

$$\text{SimF}^{\overset{\circ}{\pi}^{\pm 1}(\cdot)}(f; r_{\text{SimF}})$$

(to maintain internal state st_{sim}) and computes

$$\hat{x} := \text{SimIn}^{\overset{\circ}{\pi}^{\pm 1}(\cdot)}(f(x); r_{\text{SimIn}}).$$

In the end, it outputs $\hat{x}$ if $\hat{x}$ has at most $\rho - 1$ bits, or a symbol $\perp$ of $\rho - 1$ bits otherwise. Clearly, the output of $\mathcal{C}_{\ell, \rho, f, r}$ has at most $\rho - 1$ bits.

- Given $\hat{x}'$, the decompressor $\mathcal{D}_{\ell, \rho, f, r}$ recomputes

$$\hat{f} := \text{SimF}^{\overset{\circ}{\pi}^{\pm 1}(\cdot)}(f; r_{\text{SimF}}).$$

In the end, it outputs $y := \text{DecEval}^{\overset{\circ}{\pi}^{\pm 1}(\cdot)}(\hat{f}, \hat{x}')$ if $\hat{x}' \neq \perp$, or $\perp$ otherwise.

- If an oracle query α to $\overset{\circ}{\pi}$ (resp., β to $\overset{\circ}{\pi}^{-1}$) is needed, the hardcoded random tape r_π is used to emulate the oracle response according to Figure 11.

Let correct denote the event $\mathcal{D}_{\ell, \rho, f, r}(\mathcal{C}_{\ell, \rho, f, r}(f(x))) = f(x)$, and fail denote the event that $\hat{x}$ has at least ρ bits. These events are taken over the distribution of $(r, f(x))$. We consider the events in the same probability space where $(r, f(x))$ takes the same literal values, to observe that $\text{correct} \wedge \neg \text{fail}$ occurs if and only if $\mathcal{A}_3^{\overset{\circ}{\pi}^{\pm 1}(\cdot)}(\text{st}_2, f, \hat{x}) = 1$ occurs in the adaptive experiment (61), i.e.,

$$\Pr_{r, f(x) \leftarrow f(\mathcal{X}_f)} [\text{correct} \wedge \neg \text{fail}] = (61) \geq 1 - \text{negl}'(\lambda),$$

implying

$$\Pr_{r, f(x) \leftarrow f(\mathcal{X}_f)} [\text{correct}] \geq \Pr_{r, f(x) \leftarrow f(\mathcal{X}_f)} [\text{correct} \wedge \neg \text{fail}] \geq 1 - \text{negl}'(\lambda).$$

So, there exists a random tape r_0 , which gives a circuit pair $(\mathcal{C}_{\ell, \rho, f, r_0}, \mathcal{D}_{\ell, \rho, f, r_0})$, such that

$$\mathcal{D}_{\ell, \rho, f, r_0}(\mathcal{C}_{\ell, \rho, f, r_0}(f(x))) = f(x)$$

occurs with probability at least $1 - \text{negl}'(\lambda)$ for $f(x) \leftarrow f(\mathcal{X}_f)$. Otherwise,

$$\begin{aligned} \Pr_{r, f(x) \leftarrow f(\mathcal{X}_f)} [\text{correct}] &= \sum_r \Pr_{f(x) \leftarrow f(\mathcal{X}_f)} [\text{correct} \mid r] \cdot \Pr_r [r] \\ &< (1 - \text{negl}'(\lambda)) \cdot \sum_r \Pr_r [r] = 1 - \text{negl}'(\lambda), \end{aligned}$$

leading to a contradiction. Note the both $\mathcal{C}_{\ell, \rho, f, r_0}$ and $\mathcal{D}_{\ell, \rho, f, r_0}$ are polynomial-size circuits since the hardcoded random tape r_0 is polynomial-size and the two circuits run in polynomial time given this r_0 . Thus, $(\mathcal{C}_{\ell, \rho, f, r_0}, \mathcal{D}_{\ell, \rho, f, r_0})$ contradicts the Yao entropy $H^{\text{Yao}}(f(\mathcal{X}_f)) \geq \rho$. $\square$

F Public Parameters of Three-Halves

The following public constants are used by three-halves [RR21].

$$\begin{aligned}
M &= \left[\begin{array}{cc|cc|cc} 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 \\ \hline 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 1 \\ \hline 0 & 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 1 \\ \hline 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 \end{array} \right] \in \mathbb{F}_2^{8 \times 6}, \\
K &= \left[\begin{array}{cc|cc|cc} 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ \hline 0 & 0 & 0 & 1 & 1 & 0 & 1 & 1 \end{array} \right] \in \mathbb{F}_2^{3 \times 8}, \\
V &= \begin{bmatrix} V_{00} \\ V_{01} \\ V_{10} \\ V_{11} \end{bmatrix} = \left[\begin{array}{cc|cc|cc} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ \hline 1 & 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 1 & 1 \\ \hline 1 & 0 & 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 1 \\ \hline 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 \end{array} \right] \in (\mathbb{F}_2^{2 \times 5})^4 \equiv \mathbb{F}_2^{8 \times 5}, \\
V^+ &= \left[\begin{array}{cc|cc|cc} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ \hline 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ \hline 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 \end{array} \right] \in \mathbb{F}_2^{5 \times 8}, \\
S_L &= \left[\begin{array}{cc|cc} 1 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \end{array} \right] \in \mathbb{F}_2^{2 \times 4}, \quad S_R = \left[\begin{array}{cc|cc} 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 1 \end{array} \right] \in \mathbb{F}_2^{2 \times 4}, \\
R'_1 &= \begin{bmatrix} 0 & 0 \\ 1 & 1 \\ 0 & 1 \\ 1 & 0 \end{bmatrix} \in \mathbb{F}_2^{4 \times 2}, \quad R'_2 = \begin{bmatrix} 0 & 0 \\ 1 & 0 \\ 1 & 1 \\ 0 & 1 \end{bmatrix} \in \mathbb{F}_2^{4 \times 2}, \\
R_p &= \begin{bmatrix} R_{p,00} \\ R_{p,01} \\ R_{p,10} \\ R_{p,11} \end{bmatrix} = \left[\begin{array}{cc|cc} 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ \hline 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ \hline 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ \hline 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{array} \right] \in (\mathbb{F}_2^{2 \times 4})^4 \equiv \mathbb{F}_2^{8 \times 4}.
\end{aligned}$$

The distribution $\mathcal{R}_0$ is defined as sampling two uniform bits $[r_L \ r_R] \leftarrow \mathbb{F}_2^2$ and returning

$$R'_\$ = \begin{bmatrix} R'_{\$,00} \\ R'_{\$,01} \\ R'_{\$,10} \\ R'_{\$,11} \end{bmatrix} := r_L \cdot \begin{bmatrix} 1 & 0 \\ 1 & 0 \\ 1 & 0 \\ 1 & 0 \end{bmatrix} \oplus r_R \cdot \begin{bmatrix} 0 & 1 \\ 0 & 1 \\ 0 & 1 \\ 0 & 1 \end{bmatrix} \in (\mathbb{F}_2^{1 \times 2})^4 \equiv \mathbb{F}_2^{4 \times 2}.$$